

Depths of Induction and Formalized Self-Awareness

Version 49

Generalized Board Games, Strategic Horizons,
and Clocked Simulation Degrees

Brian Tenneson

September 24, 2026

Scope freeze. Version 49 retains the Version 48 core and closes the present development cycle by integrating: (1) the settled order theory of clocked simulation degrees; (2) generalized-board-game operationalizations of proof and search; (3) the separation of proof, discovery, and forced-settlement horizons; (4) a GBG interpretation of scheduled simulation degrees; and (5) a Hilbert-addressed ATP case study. Later themes belong to a subsequent edition rather than being allowed to make Version 49 a moving target.

Abstract

Version 49 retains the mathematical core of Version 48 and integrates two later developments. First, the clocked-simulation preorder on semi-ideal computers is sharpened through its quotient degree order: the degree poset contains a countably infinite antichain of finite, finitely describable machines, fails to be a lattice even when relevant common bounds exist, admits synchronized dynamic invariants, and retains these phenomena under effective simulation. Because only countably many finite descriptions exist, the countably infinite antichain has the largest possible cardinality among antichains restricted to finitely describable machines.

Second, generalized board games (GBGs) supply an operational state-transition interpretation of formal inference, ATP search, and cooperative AMLD settlement. Finitary inference can be lifted to unary moves on complete derivation states; theoremhood becomes finite target-region reachability; an inference-step proof horizon becomes directed distance; and future-complete operational states admit transposition quotienting. The resulting geometry separates intrinsic proof horizon, implementation-dependent discovery effort, and a strategic or scheduler-robust forced-settlement horizon. Conservative BANK growth can shorten certified routes without changing consequence closure.

The final integration defines a scheduled/deterministic GBG simulation-degree viewpoint and uses the Hilbert-addressed inference-space architecture as a case study. Different schedulers may preserve theoremhood and proof depth while inducing different operational chronologies; whether particular schedulers occupy the same or distinct clocked degrees is posed as a precise comparison problem rather than assumed.

Version 49 integration map

Component			Version 49 status
Retained	Version 48	core	Preserved as the documentary mathematical baseline, with Version 49 running identification.
Clocked simulation	degrees		Settlements of former Open Problems 9.1, 9.3, 9.4, and 9.5 integrated as proved results.
Former Open Problem	9.2		Greatest-degree question remains open in this edition.
Generalized board games			Used as an operationalization of finitary proof and search states, not merely as analogy.
Strategic horizons			Proof horizon, discovery effort, and forced/scheduler-robust settlement horizon are separated.
AMLD/BANK			Nonstarvation is treated as an admissibility restriction on external scheduling; BANK growth as conservative road-building.
GBG simulation	degrees		The old clocked-simulation preorder is reinterpreted for deterministic scheduled GBG chronologies.
Hilbert case study			Hilbert-, lexicographic-, random-, or learned-order search on the same action fibers becomes a concrete degree-comparison problem.

Contents

Version 49 integration map

i

Retained Version 48 Core

1

Version 49 Integration A: Clocked Simulation Degrees	166
1 Standing clocked-simulation definitions	166
2 Settlement of former Open Problem 9.1: infinite antichains	166
3 Settlement of former Open Problem 9.3: joins and meets	167
4 Settlement of former Open Problem 9.4: synchronized invariants	168
5 Settlement of former Open Problem 9.5: effective degrees	168
6 The one former open problem intentionally left open	168
 Version 49 Integration B: Generalized Board Games and Strategic Settlement	 169
7 Purpose of the synthesis	169
8 Standing formal framework from Depths v48	169
8.1 Formal systems and partial inference rules	169
8.2 The Version 48 separation of metrics	170
9 The GBG layer	170
10 Operational lifting: from k -ary inference to unary proof-state moves	171
10.1 What has and has not been shown	171
11 Proof horizon as directed distance	172
12 Histories, states, and transpositions	172
13 Proof-count growth and match-count growth	173
14 Three horizons	173
14.1 Intrinsic proof horizon	173
14.2 Discovery effort	174
14.3 Strategic or forced-settlement horizon	174
15 A layer diagram	175
16 AMLD global states are already rule-complete candidates	175
17 Nonstarvation as a strategic admissibility condition	176
18 BANK growth as conservative road-building	177
18.1 Macro shortcuts versus expanded primitive cost	177
19 When BANK also improves a forced horizon	178
20 Learned legal-move ranking in GBG language	178
21 Causal BANK depth and path compression	179

22 A minimal worked example	179
23 The five AMLD roles as settlement regions	180
24 A proposed strategic settlement hierarchy for Depths	180
25 What this adds to the induction story	181
26 Boundaries and nonclaims	181
27 Research consequences	182
 Version 49 Integration C: Scheduled GBG Clocked Simulation Degrees	 183
28 Scheduled GBG clocked simulation degrees	183
29 Degree structure versus theoremhood	183
30 Deterministic hitting-time distortion under a clocked embedding	184
31 Case study: Hilbert-addressed operational ATPs	184
32 Why the infinite-antichain theorem matters for GBGs	185
33 Strategic extension: beyond deterministic clocked degrees	185
34 Version 49 research boundary	186
35 Version 49 conclusion	186
Version 49 references	187

Retained Version 48 Core

The following pages retain the Version 48 core. They are carried forward as the baseline on which the editable Version 49 integrations are built. The Version 49 additions begin immediately after the retained core.

Abstract

This edition consolidates three previously diverged branches of the manuscript. The corrected Version 46.2 core supplies the paper’s formal-system, SIC/ATP, proof-complexity, induction, and formal-self-awareness framework. The material unique to Version 47 is retained as a separate integration on learned proof-search guidance: a learned ranker changes allocation of finite search attention among legal candidate applications while an independent verifier remains the mathematical trust boundary. The reported Predator 8.002 `prcom` run is preserved as a finite-instance, right-censored search-effort comparison rather than an asymptotic speedup theorem. A second integration revises the Version 46.3 AMLD material. It formalizes a cooperative five-role architecture with typed verifier-governed BANK memory, distinguishes object-theory channels from metatheoretic dependence and independence certifiers, proves conservative BANK invariants and cooperative birth-time induction through expanded certificate DAGs, separates first settlement from later settlement refinement, proves object-level proof-horizon monotonicity, states resource-qualified ATP-emulation results, and sharpens the AMLD potential method with a causal dependency-depth bound. Formal self-awareness remains a syntactic and proof-theoretic notion; no phenomenological claim is intended.

Version 48 consolidation and authority statement

Version 48 is the successor to both Version 47 and Version 46.3. It resolves the numbering split by taking the corrected Version 46.2 core as the common foundation, importing only the material unique to Version 47, and replacing the Version 46.3 AMLD supplement by the revised chapter in this edition.

The corrected core historically used a facsimile-plus-replacement architecture because the underlying merged Parts I and II were not available as one editable LaTeX source. Version 48 preserves that documentary core rather than silently reconstructing mathematical text from PDF extraction. The following reading rule is therefore part of the edition:

Authority rule. A retained facsimile sentence that conflicts with an adjacent or later correction insert is historical source text only and is not a Version 48 mathematical claim. The correction insert is authoritative. No theorem, corollary, remark, priority claim, or complexity claim is asserted in Version 48 merely because it remains visible on a superseded facsimile page.

Every imported core page carries a Version 48 footer repeating this rule. This avoids the ambiguity of an unmarked facsimile while preserving documentary provenance. The new ML and AMLD integrations are directly editable LaTeX and contain no superseded alternate statements.

Version 48 consolidation changes

1. Resolves the version split: Version 48 explicitly supersedes both Version 47 and Version 46.3.
2. Retains the Version 47 learned-search contribution without duplicating its induction material already present in the corrected core.
3. States the `prcom` result only as a reported finite-instance search-effort separation under a frozen protocol; independent reproduction remains an empirical obligation.
4. Makes the AMLD refutation target T^- and contradiction target $\perp$ explicit members of the object language rather than assuming primitive negation syntax.

5. Places D and I in a declared metatheory with sound certificate interfaces and explicitly rejects timeout or search failure as an independence certificate.
6. Repairs cooperative BANK birth-time induction by distinguishing external BANK leaves from internal certificate formulas and inducting through the finite certificate DAG.
7. Replaces an absorbing single settlement class by a monotone settlement record with an absorbing first-settlement time, so an early dependence certificate may later be refined by proof or refutation.
8. Retains the cumulative weighted AMLD potential theorem and adds a sharper causal dependency-depth theorem that does not charge irrelevant deposits merely because they occurred earlier in the schedule.
9. Preserves the distinction among proof length, logical stage, search expansions, wall-clock time, and aggregate computational work throughout the new integrations.

Contents

Abstract	1
Version 48 consolidation and authority statement	1
Version 48 consolidation changes	1
 Corrected Core: Version 46.2 Atlas Edition	 4
 Version 48 Integration A: ML-Guided Proof Search	 154
Purpose and provenance of the ML-guided integration	154
1 Proof-count bounds and learned search allocation	154
2 The Predator 8.002 controlled prcom experiment	155
3 Illustrative counting proxy and its limitation	156
4 Relationship to induction	156
 Version 48 Integration B: AMLDs, Cooperative Induction, and Settlement Speed	 157
Scope of the AMLD integration	157
1 Formal AMLD setting	157
2 Global-state induction and conservative BANK growth	159
3 Channel discovery, BANK birth, and certificate induction	159
4 Settlement time and refinement	160

5	Proof horizons under cooperative BANK growth	160
6	Comparing a stand-alone ATP with an AMLD	161
7	Wall-clock speed and total work	162
8	Weighted and causal AMLD potential bounds	162
9	AMLDS and the induction hierarchy	163
10	Monotone growth and monotone object-level horizons	163
11	Conclusions	163
Version 48	Closing Synthesis	165

Corrected Core: Version 46.2 Atlas Edition

The following pages are the corrected common core inherited by Version 48. They include their own detailed contents and correction inserts. The Version 48 authority rule printed above and in the footer of every imported page governs all retained facsimile wording.

Version 46.2 treats the remaining audit findings as dependency obligations rather than isolated edits. The exact finite-reflection-layer result and the induction supplement are retained. The following replacements repair the narrowest common ancestors of the affected downstream claims:

- All Version 46.1 corrections remain authoritative except where one of C37–C45 below expressly supersedes them.

Abstract

This paper studies formal systems, semi-ideal computers (SICs), automated theorem provers (ATPs), simulation preorders, proof search, and formally self-aware structures. Version 46 adds a systematic account of induction at four distinct levels. First, structural induction on well-formed formulas is available when the formula set is the least set generated by an explicit finitary grammar; it is not a consequence of merely declaring an arbitrary set of strings to be the well-formed formulas. Second, induction on finite derivations is equivalent to the least-inference-closure principle already underlying the consequence operator. Third, rule-driven ATPs admit stage induction, birth-time induction, and invariant induction, yielding uniform soundness principles and quantitative lower bounds on the first stage at which a target can appear. Fourth, the reflection tower used to grade formal self-awareness is an ordinary inductive sequence at finite levels, while its proposed transfinite extension requires genuinely separate limit-stage clauses.

The added results show that induction applies to ATPs in a mathematically substantive way: it proves preservation of formula invariants across a run, characterizes the union of exhaustive closure stages, propagates halting and freezing properties, and converts slowly changing formula ranks into lower bounds on ATP birth times. Applied to the reflection-closure machine, the method computes the exact proof layer of every finite reflection formula. Version 46.1 also repairs the self-awareness material where the submitted edition overreached: admissible translations are made injective and type-correct; finite reflection is separated from transfinite reflection; the Picard and Discovery claims are kept explicitly conditional; and the IIT comparison is recast as a conditional comparison requiring an explicit mechanism-level realization and calculation.

Version 46.1 changelog

- Added a formal distinction among *formula-formation induction*, *proof or rule induction*, *ATP stage induction*, and *reflection-sequence induction*.
- Added a grammatical extension of the *xyz* formal-system model. Structural induction on WFFs is stated only under the necessary least-generation hypothesis.
- Restated the consequence set as a least inference-closed set and proved the corresponding rule-induction scheme in predicate form.
- Added state-invariant, formula-invariant, and birth-time induction theorems for rule-driven ATPs.
- Added a quantitative ATP potential theorem: a bounded increase per inference layer forces a lower bound on the birth stage of a target.
- Added an induction theorem for persistent halting and frozen states.
- Connected the induction principles to the finite reflection tower and calculated the reflection-closure machine's exact layers $b_R(\theta_k) = k + 2$ on its designated base input.
- Clarified that the self-theory $\text{Self}(M)$ need not itself be inference-closed, even when the full consequence set is.
- Revised Part II to repair admissible translations, non-degeneracy, reflection, Picard, phenomenology, and IIT claims; conditional and conjectural statements are now labeled accordingly.

Editorial and pagination note

The submitted LaTeX source contains editable front matter and the Version 46 induction supplement, but it embeds Parts I and II as PDF facsimile pages rather than as editable LaTeX. Version 46.1 therefore preserves that source architecture. Each affected facsimile page is followed immediately by authoritative replacement text, and every old running header is covered by the Version 46.1 header. The replacement text controls wherever it conflicts with the facsimile. Printed page numbers inside the facsimile remain those of the merged source; the induction supplement retains page labels of the form V46-1, V46-2, and so on.

Reader's map

Location	Content
Corrected Part I, pp. 9–53	Formal systems, consequence and inference closure; SICs and ATPs; simulation; self-reference and recoding; heuristic proof search; theorem graphs; ground-up ATPs; proof counting; computability; Cook–Reckhow proof systems; algebraic proof systems; conclusion.
Version 46.1 Supplement	Structural induction on WFFs; proof induction; ATP stage and birth-time induction; invariant and potential methods; exact finite reflection layers; the boundary at transfinite limit stages.
Corrected Part II, pp. 54–94	Internal representations; tags; self-descriptive formulas; self-models and self-theories; levels of formal self-awareness; the self-awareness preorder and quotient; Tegmark; the Discovery Thesis; Picard; settlement of conjectures; integrated information.
Corrected appendices, pp. 94–100	Examples of ATPs; Turing machines, computation, and SICs.
Corrected closing matter, pp. 101–102	Acknowledgements and bibliography.

Detailed Contents

Abstract	i
Version 46.1 changelog	i
Editorial and pagination note	ii
Reader’s map	ii
How to read the corrected facsimile	vii
Part I: Computers, Formal Systems, Simulations, and Automated Theorem Proving	9
Global notation for functions	9
The intersection of nonstandard analysis and proof theory	9
1 Formal Systems, Consequence, and Inference Closure	10
2 Semi-Ideal Computers, Shortest Proofs, and Canonical Search	12
3 Weak, Synchronous, Clocked, and Effective Simulation	15
3.1 Synchronous deterministic simulation	15
3.2 Tests of synchronous over-rigidity	16
3.3 Clocked deterministic simulation	17
3.4 Effective witnesses	18
3.5 Canonical state abstractions	19
3.6 Finite-horizon comparison	20
4 Self-reference, recoding, and translated copies of SICs	20
Correction C10: Set-sized coding universes	21
4.1 The Picard Protocol and a minimal self-certifying ATP	24
Correction C01: Self-audit compression with explicit complexity hypotheses	28
Correction C02: Finite metadata self-description over the expanded language	31
5 Nondeterministic Search Policies and Heuristic Proof Search	32
6 Theorem Graphs, Maze Search, and Graph-Theoretic ATPs	33
7 Ground-Up ATPs, Weak Set Theory, Python, and Speedup	35
7.1 A weak set-theoretic setting for the successor-set identity	35
7.2 Executable ATPs and a Python point of view	36
7.3 Transactions, derived rules, and speedup	36
Correction C03: Executable finite-fragment ATP theorem	37
8 Counting Proofs, Shortest Proofs, and the Password Analogy	39
9 Universality, Finite Effectiveness, and Computability	40
Correction C11: Computability on effectively presented state domains	41
Correction C11: Computational effectiveness and the internal Church–Turing theorem	43
10 Propositional Proof Systems, p-Simulation, and Feasible Interpolation	44
10.1 Proof size as a refinement of proof length	44
10.2 p-simulation: a second preorder on formal systems	46
Correction C04: Polynomial equivalence of Frege systems	47
10.3 The pigeonhole principle: an unconditional instance of asymptotic speedup	48
10.4 Automatizability	49
Correction C05: Two-system size speedup for proof-producing realizations	50
10.5 Feasible interpolation as a proof-complexity barrier theorem	51
Correction C06 and C33: Interpolation lower bounds and current random 3-CNF status	52

10.6 An open problem, restated	53
10.7 Algebraic Proof Systems: Nullstellensatz, Polynomial Calculus, and Groebner Bases	53
Arithmetizing a formula	53
Two algebraic formal systems	54
The negative direction	55
Correction C07: Proof-producing Gröbner lower bound	56
The positive direction	57
A Collection of Examples of Objects Discussed Herein	58
Correction C08 and C09: Correct comparison of resolution and Polynomial Calculus	59
11 Conclusion	63
Correction Consequential revision: Qualified Part I conclusions	65
Version 46.1 Supplement: Induction, Closure, and ATP Invariants	V46-1
1. Scope and the four meanings of induction	V46-1
2. Structural induction on well-formed formulas	V46-1
3. Inference closure and induction on proofs	V46-2
4. Induction on ATP runs	V46-2
5. Induction and exhaustive closure	V46-3
6. Quantitative induction: potentials and ATP birth stages	V46-4
7. Induction on halting and freezing	V46-4
8. Induction and formal self-awareness	V46-5
9. The transfinite boundary	V46-6
10. Consolidated hierarchy of induction principles	V46-6
11. Conclusion of the supplement	V46-6
Part II: Formalized Self-Awareness and Self-Aware Structures	54
12 Introduction	54
13 Internal Representations	54
13.1 Internal Representations	54
13.2 Tagging	55
13.3 Self-Descriptive Formulas	55
13.4 Self-Models	55
13.5 Self-Theories	56
14 A Minimal ATP Proving Its Own Kind	56
15 Levels of Formal Self-Awareness	57
15.1 Level 0 and Level 1	57
15.2 Names for Formulas	57
15.3 The Internal Provability Predicate	58
15.4 The Reflection Sequence	58
15.5 Level n	59
15.6 Cumulativity Fails Without Hypotheses	59
15.7 A Minimal Level-2 Machine	60
Correction C14: Picard Protocol cross-reference	61
15.8 Level omega	62
15.9 Strictness of the Hierarchy	62
Correction C15 and C16: Internal necessitation and the transfinite boundary	63
16 The Order-Theoretic Structure of Formal Self-Awareness	64
16.1 The Formal Self-Awareness Preorder	64

16.2	A Defect in Definition 16.1, and Its Repair	65
	Correction C12: Admissible translations as genuine language embeddings	66
16.3	The Induced Equivalence Relation	67
16.4	The Quotient Set	67
16.5	The Level Function on the Quotient	68
16.6	Relationship with the Simulation Preorder	69
16.7	Ramifications and Open Problems	69
	Correction C13: Simulation notation	70
17	Self-Aware Substructures and the Mathematical Universe	71
17.1	Tegmark’s Implicit Definition	71
17.2	Satisfiability, and Why It Is a Problem	72
17.3	Grading the Criterion	72
17.4	Bird and Frog	73
17.5	What Is and Is Not Claimed	73
	Correction C14 and C20: Picard reference; stage and thought as models	74
18	The Discovery Thesis	75
18.1	Statement of the Thesis	75
18.2	The Existential Claim Is a Theorem, Not a Conjecture	75
18.3	First Objection: Mere Assertion	76
18.4	Second Objection: Use and Mention	76
18.5	Third Objection: The Level Is Too Low	77
18.6	Fourth Objection: Tegmark’s Complexity Requirement	77
18.7	Fifth Objection: No Phenomenology	78
	Correction C17: Non-degenerate witnesses without defective cardinal comparison	79
18.8	Non-Degenerate Witnesses Already in the Literature	80
	Correction C18: Milawa and Lean4Lean as candidates, not completed classifications	81
18.9	An Algebraic Prover Against the Four Clauses	82
	Correction C18 and C19: Candidate classifications and the canonical Polynomial Calculus machine	83
	Correction C19: Exhaustive closure, target freezing, and shortest-proof layers	85
18.10	The Parity Argument	86
18.11	Tegmark’s 1998 Challenge	86
18.12	Scope: What Is and Is Not Being Claimed	87
18.13	What Would Refute the Thesis	87
18.14	Open Problems Specific to the Discovery Thesis	88
19	The Philosophy of Formalized Self-Awareness	88
20	The Picard Theorem	89
20.1	The Subject and the Idealization	89
20.2	The Theorem	90
	Correction C21: Correct source reference in the Picard idealization	91
20.3	Corollaries	92
	Correction C22: Picard non-degeneracy under the corrected criterion	93
	Correction C23 and C24: Conditional Picard parity and injective incomparability	95
20.4	Conjectures, and Their Settlement	96
20.5	What Is and Is Not Claimed	98
21	Settlement of the Conjectures	99
21.1	The Reflection-Closure Machine	99

21.2 Universality (Conjecture 20.18)	100
21.3 Competence and Performance (Conjecture 20.20)	101
21.4 Equivalence and the Non-Invariance of Degeneracy (Conjecture 20.23)	101
21.5 Phenomenology (Conjecture 17.14)	102
21.6 The Transfinite Extension (Conjecture 20.22)	102
Correction C25 and C26: Phenomenology theorem and expressive limitation	103
21.7 New Conjectures	104
Correction C27: Transfinite reflection as a specified-calculus question	105
22 Integrated Information and the Two Senses of a Map	106
22.1 The Asymmetry	106
Correction C28 and C32: Practical provers and accurate IIT terminology	107
22.2 Two Senses of Mapping Experience	108
22.3 Mechanism Properties and Self-Theory Properties	108
22.4 The Placement of Phi	108
Correction C29, C30, and C32: Full mechanism presentations and conditional IIT comparison	109
22.5 What Is and Is Not Claimed	110
Appendix A: Examples of Automated Theorem Provers	110
Correction C31 and C32: The formal-phenomenal gap remains conjectural	111
Appendix B: Turing Machines, Computation, and SICs	112
B.1 Deterministic Turing Machines and Configurations	112
B.2 The Configuration Formal System	113
B.3 Trace SICs	113
B.4 Computability of SICs	114
B.5 The Correspondence and Its Boundary	115
B.6 Discussion: Computational Power and Proof-Search Suitability	116
Acknowledgements	118
References	118
Correction C34 and C36: Corrected references and acknowledgements	119
Version 46.1 source-integrity note	121
Version 46.1 source-integrity note	121
External references checked for Version 46.1	121

How to read the corrected facsimile

The source supplied for this edition embeds the established Parts I and II as PDF pages. Version 46.1 therefore uses a page-local correction method: an affected facsimile page is followed by an authoritative replacement page. The replacement page identifies the numbered item being corrected and supplies the corrected definition, theorem, proof, remark, conjecture, citation, or scope qualification. Unaffected facsimile pages remain unchanged except that their obsolete July 27 running header is covered by the Version 46.1 header.

Computers, Formal Systems, Simulations, and Automated Theorem Proving

Here *constructive standard mathematics* means ordinary mathematics stated without nonstandard vocabulary and proved in a constructive base theory, so that existence proofs carry witness information rather than relying only on classical excluded-middle reasoning. *Computationally explicit standard mathematics* means ordinary mathematics stated without nonstandard vocabulary in which the output of the proof is not just “there exists an object” but an algorithm, term, bound, modulus, finite search procedure, or other data that can in principle be computed.

There is an important contrast with the self-reference section below. A proof interpretation from F^{ns} to F may be a content-extracting translation that loses some nonstandard vocabulary and is not simply an isomorphism. A recoding map, by contrast, is deliberately invertible. It is a translator between two presentations that are mathematically the same language up to isomorphism. Thus proof extraction studies how much standard content hides in a nonstandard proof, while recoding studies how a system can recognize a coded, renamed, or quoted copy of itself without changing theoremhood.

The proof-theoretic background is standard; see, for example, [16, 25, 2]. Let $\mathbf{N}^+$ denote the positive integers. For each $m \in \mathbf{N}^+$, let $F_m := \{1, \dots, m\}$. If x is a set, then x^m denotes the set of all functions from F_m to x , equivalently the set of ordered m -tuples from x .

Definition 1.2. A formal system is an ordered triple $F = (x, y, z)$ such that:

- The elements of x are the symbols of F , the elements of y are the well-formed formulas of F , and the elements of z are the inference rules of F . For each $k \in \mathbb{N}^+$, write $z_k(F) := \{r \in z : \text{ar}(r) = k\}$.

C37: Target-search SICs freeze at the proof horizon

Authoritative replacement for: Definition 2.11, Theorem 2.12(2), Corollary 2.13.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

Let

$$h_\varphi(\Gamma) := \|\varphi\|_F(\Gamma) \in \mathbb{N}_+ \cup \{\infty\}, \quad \min(m, \infty) := m.$$

Keep the target-free breadth-first SIC $B_{F,Y}$ unchanged. Replace the target-search state map by

$$C_{F,Y}^\varphi(\Gamma, m) := \{\psi \in Y : \|\psi\|_F(\Gamma) \leq \min(m, h_\varphi(\Gamma))\},$$

and put

$$H_{F,Y}^\varphi(\Gamma, m) = 1 \iff h_\varphi(\Gamma) \leq m.$$

Then $B_{F,Y,\varphi} := (C_{F,Y}^\varphi, H_{F,Y}^\varphi)$ is a SIC. Before the target is reached its state agrees with ordinary breadth-first proof-length search; once $H = 1$, the state is frozen, so Definition 2.3(6) is satisfied.

Corrected Proof-Horizon conclusion. If $\Gamma \vdash_F \varphi$, the first halting stage is exactly $h_\varphi(\Gamma) = \|\varphi\|_F(\Gamma)$; if $\Gamma \not\vdash_F \varphi$, it never halts. Consequently the target transaction count remains

$$\tau_{B_{F,Y,\varphi}}(\Gamma, \varphi) = \|\varphi\|_F(\Gamma).$$

Thus the proof-horizon theorem survives; only the post-halt state evolution is corrected.

Proof. The SIC axioms follow directly from the definition. If $H_{F,Y,\varphi}(\Gamma, m) = 1$, then $\varphi \in C_{F,Y,\varphi}(\Gamma, m)$ by definition.

Conversely, suppose $\Gamma \vdash_F \varphi$. By Theorem 2.17, the formula φ appears at some stage of the enumerating search from Γ . The target-search SIC follows the same search until the first stage at which φ appears and then freezes. Hence $H_{F,Y,\varphi}(\Gamma, m) = 1$ for some $m \in \mathbf{N}^+$. $\square$

3 Weak, Synchronous, Clocked, and Effective Simulation

The original relational notion is retained as a boundary case. The exact stage-for-stage functional notion is retained as a strong faithful embedding. The principal comparison relation is clocked deterministic simulation, which allows a simulator to run faster, slower, or with idle stages while preserving states and halting observations at corresponding times.

Definition 3.1. Let $M = (C_M, H_M)$ be a SIC over $Y_M \subseteq y$, and let $N = (C_N, H_N)$ be a SIC over $Y_N \subseteq y$. A relation $R \subseteq S_M \times S_N$ is a *weak SIC-simulation witness* from M to N if:

1. for every $(p, q) \in R$ and every $m \in \mathbf{N}^+$, there exists $n \in \mathbf{N}^+$ such that $(C_M(p, m), C_N(q, n)) \in R$;
2. for every $\Gamma \subseteq Y_M$, there exists $\Delta \subseteq Y_N$ such that $(C_M(\Gamma, 1), C_N(\Delta, 1)) \in R$.

If such a relation exists, write $M \preceq_w N$ and say that M *weakly simulates into* N .

Theorem 3.2 (Collapse of weak simulation). *Every SIC weakly simulates into every other SIC.*

Proof. Let M and N be SICs and take $R := S_M \times S_N$. If $(p, q) \in R$ and $m \in \mathbf{N}^+$, choose $n = 1$. Then $C_M(p, m) \in S_M$ and $C_N(q, 1) = q \in S_N$, so $(C_M(p, m), C_N(q, 1)) \in R$. The anchoring condition is also immediate because every pair of initial states belongs to R . $\square$

Corollary 3.3. *Weak simulation has a single equivalence class. It cannot support a substantive theory of simulation depth.*

Proposition 3.4 (State-space saturation). *If M is a SIC over Y , then $S_M = \mathcal{P}(Y)$.*

Proof. By definition $S_M \subseteq \mathcal{P}(Y)$. Conversely, for every $\Gamma \subseteq Y$, the stage-one axiom gives $C_M(\Gamma, 1) = \Gamma$. Hence every subset of Y belongs to S_M . $\square$

3.1 Synchronous deterministic simulation

Definition 3.5. A *synchronous deterministic simulation* from M to N is a function

$$f \in [S_M \rightarrow S_N]$$

such that:

1. f is injective;
2. for every $p \in S_M$ and every $m \in \mathbf{N}^+$,

$$f(C_M(p, m)) = C_N(f(p), m);$$

3. for every $p \in S_M$ and every $m \in \mathbf{N}^+$,

$$H_M(p, m) = H_N(f(p), m).$$

Remark 3.6. This is a faithful synchronous state embedding. It is intentionally stronger than behavioral similarity: injectivity preserves every source-state distinction, and the same stage parameter appears on both sides. The earlier term “observable simulation” is avoided because the definition preserves the entire SIC state rather than merely a separately specified observation.

Proof. If $(p, f(p)) \in R_f$, then for every m the synchronous evolution equation gives $(C_M(p, m), C_N(f(p), m)) = (C_M(p, m), f(C_M(p, m))) \in R_f$. For anchoring, use $(\Gamma, f(\Gamma))$ and Proposition 3.4. $\square$

Proof. The identity is injective and preserves both C_M and H_M exactly. $\square$

Proof. The composite is injective. For every p and m ,

and halting preservation composes in the same way. $\square$

3.2 Tests of synchronous over-rigidity

Proof. By Proposition 3.4, the witness is an injection $\mathcal{P}(Y_M) \rightarrow \mathcal{P}(Y_N)$. In the finite case, $2^{|Y_M|} \leq 2^{|Y_N|}$, hence $|Y_M| \leq |Y_N|$. $\square$

$$C_M(p, m) = p, \quad H_M(p, m) = 0, \quad C_N(q, m) = q, \quad H_N(q, m) = 0.$$

Theorem 3.13 (One-stage-delay obstruction). *There are SICs performing the same update with a one-stage delay that are incomparable under synchronous deterministic simulation.*

Proof. Let $Y = \{a\}$ and let both halting maps be constantly zero. Define M by

$$C_M(\Gamma, 1) = \Gamma, \quad C_M(\Gamma, m) = \Gamma \cup \{a\} \quad (m \geq 2),$$

and define N by

$$C_N(\Gamma, 1) = \Gamma, \quad C_N(\Gamma, 2) = \Gamma, \quad C_N(\Gamma, m) = \Gamma \cup \{a\} \quad (m \geq 3).$$

Both satisfy the SIC axioms. Suppose first that $f \in [S_M \rightarrow S_N]$ witnesses $M \preceq_{\text{syn}} N$. At stage 2 and input $\emptyset$,

$$f(\{a\}) = f(C_M(\emptyset, 2)) = C_N(f(\emptyset), 2) = f(\emptyset),$$

contradicting injectivity. Conversely, if f witnessed $N \preceq_{\text{syn}} M$, then at stage 2,

$$f(p) = f(C_N(p, 2)) = C_M(f(p), 2) = f(p) \cup \{a\} = \{a\}$$

for every $p \in S_N$, where the last equality holds because $f(p) \subseteq \{a\}$; again this contradicts injectivity, since $|S_N| = 2$. $\square$

3.3 Clocked deterministic simulation

Definition 3.14. An *admissible clock* is a function

$$\rho \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$$

such that $\rho(1) = 1$, ρ is nondecreasing, and ρ is unbounded. Repeated values permit the source to take an idle stage relative to the target; skipped values permit the target to take extra stages between represented source stages.

Definition 3.15. A *clocked deterministic simulation* from M to N is a pair (f, ρ) such that $f \in [S_M \rightarrow S_N]$ is injective, ρ is an admissible clock, and for every $p \in S_M$ and $m \in \mathbf{N}^+$,

$$f(C_M(p, m)) = C_N(f(p), \rho(m))$$

and

$$H_M(p, m) = H_N(f(p), \rho(m)).$$

If such a pair exists, write $M \preceq_{\text{clk}} N$. Because this is the principal simulation relation in the remainder of the paper, the unadorned notation $M \preceq N$ abbreviates $M \preceq_{\text{clk}} N$.

Remark 3.16. Clocked deterministic simulation retains faithful injective state preservation but removes the artificial requirement that corresponding computations use the same stage number. It repairs timing rigidity, not cardinality rigidity. Lossy many-to-one comparisons remain state abstractions rather than simulations.

Proposition 3.17. If $M \preceq_{\text{syn}} N$, then $M \preceq N$.

Proof. Use the identity clock $\rho(m) = m$. $\square$

Proposition 3.18. For every SIC M , the pair $(\text{id}_{S_M}, \text{id}_{\mathbf{N}^+})$ witnesses $M \preceq M$.

Proposition 3.19 (Composition of clocked simulations). Suppose $M \preceq N$ via (f, ρ) and $N \preceq P$ via (g, σ) . Then $M \preceq P$ via

$$(g \circ f, \sigma \circ \rho).$$

$$(g \circ f)(C_M(p, m)) = g(C_N(f(p), \rho(m))) = C_P(g(f(p)), \sigma(\rho(m))),$$

and the halting equations compose identically.

Corollary 3.20. *The principal clocked deterministic simulation relation $\preceq$ is a preorder on SICs.*

Definition 3.21. Write $M \sim N$ if and only if $M \preceq N$ and $N \preceq M$.

Corollary 3.22. *The relation $\sim$ is an equivalence relation, and $\preceq$ induces a partial order on the set of $\sim$ -equivalence classes.*

Theorem 3.23 (Clocked repair of the delay example). *The SICs M and N from Theorem 3.13 satisfy $M \sim N$.*

Proof. For $M \preceq N$, use $f = \text{id}$ and

$$\rho(1) = 1, \quad \rho(m) = m + 1 \quad (m \geq 2).$$

For $N \preceq M$, use $f = \text{id}$ and

$$\sigma(1) = 1, \quad \sigma(2) = 1, \quad \sigma(m) = m - 1 \quad (m \geq 3).$$

Both clocks are admissible, and direct substitution verifies the state and halting equations.

3.4 Effective witnesses

The notation $[A \rightarrow B]$ denotes all functions, so neither synchronous nor clocked simulation requires a computable witness. This is appropriate for a set-theoretic preorder, but executable comparison requires a refinement.

Definition 3.24. An *effective presentation* of a SIC M is an injective coding $e_M \in [S_M \rightarrow \mathbb{N}]$ with decidable image such that the coded stage and halting maps are total computable functions. Given effective presentations of M and N , a clocked simulation (f, ρ) is *effective* if ρ is computable and the induced map

$$e_N \circ f \circ e_M^{-1}$$

is computable on $e_M[S_M]$. Write $M \preceq_{\text{eff}} N$ when such a witness exists.

Remark 3.25. By state-space saturation, a full SIC over a countably infinite Y has state space $\mathcal{P}(Y)$ and therefore has no coding of all states into $\mathbf{N}$. The effective definition applies directly to finite SICs and to other countably presented state spaces. Infinite executable applications should specify a restricted effective state domain, such as finite or reachable states, rather than silently coding the full power set.

Proposition 3.26. *For fixed effective presentations, $\preceq_{\text{eff}}$ is a preorder and implies $\preceq$.*

Proof. The identity maps and identity clock are computable. Computable state maps and clocks are closed under composition, and Proposition 3.19 provides the underlying clocked simulation. $\square$

Theorem 3.27 (Noncomputable simulation witnesses exist). *The set-theoretic synchronous simulation definition permits witnesses that are not computable even on finitely described singleton states.*

$$\pi_K(2e) = \begin{cases} 2e+1, & e \in K, \\ 2e, & e \notin K, \end{cases} \quad \pi_K(2e+1) = \begin{cases} 2e, & e \in K, \\ 2e+1, & e \notin K. \end{cases}$$
$$f_K(C_I(A, m)) = f_K(A) = C_I(f_K(A), m), \quad H_I(A, m) = H_I(f_K(A), m) = 0.$$

Remark 3.28. The identity is also a computable self-simulation of I . The theorem does not claim that the pair is related only by noncomputable witnesses. It shows that effectivity is not automatic at the witness level and must be stated when it matters.

$$a(C_M(p, m)) = C_N(a(p), m)$$
$$a_M \in [S_M \rightarrow S_{E_{F,Y}}] \quad \text{by} \quad a_M(p) := \text{Con}_F(p) \cap Y.$$
$$C_{F,Y}(a_M(p), m) = a_M(p).$$
$$\text{Con}_F(C(p, m)) \cap Y = \text{Con}_F(p) \cap Y.$$
$$a_M(C(p, m)) = C_{F,Y}(a_M(p), m).$$

Documentary core: Retained Versailles recording edit adds Version 40 integration, following insert that correction insert of the status in the Version 48 claim.

Theorem 3.32 (Target-search closure-abstraction theorem). *Let $Y \subseteq y$ be inference-closed, let $\varphi \in Y$, and let $M = (C, H)$ be a rule-driven target-search ATP over Y . Define*

$$a_{M,\varphi}(p) := \text{Con}_F(p) \cap Y.$$

Then $a_{M,\varphi} \in [S_M \rightarrow S_{E_{F,Y,\varphi}}]$ is a state abstraction.

Proof. If $q = \text{Con}_F(p) \cap Y$, then q is inference-closed. Whether or not $\varphi \in q$, the canonical target-search SIC remains at q . The same closure argument as in Theorem 3.30 gives

$$a_{M,\varphi}(C(p, m)) = C_{F,Y,\varphi}(a_{M,\varphi}(p), m).$$

□

Remark 3.33. The same distinction applies to proof extraction from nonstandard analysis. A proof interpretation is always an extraction or abstraction map. It becomes a synchronous deterministic simulation only when it preserves exact stages and halting through an injective state map, and it becomes a clocked deterministic simulation when those properties hold after an admissible time rescaling.

3.6 Finite-horizon comparison

Definition 3.34. Let $K \in \mathbf{N}^+$. A *depth- K clocked deterministic simulation* from M to N is an injective function $f \in [S_M \rightarrow S_N]$ together with a nondecreasing function

$$\rho_K \in [\{1, \dots, K\} \rightarrow \mathbf{N}^+]$$

such that $\rho_K(1) = 1$ and, for every $p \in S_M$ and $m \leq K$,

$$f(C_M(p, m)) = C_N(f(p), \rho_K(m)) \quad \text{and} \quad H_M(p, m) = H_N(f(p), \rho_K(m)).$$

It is *synchronous* when $\rho_K(m) = m$ for every $m \leq K$.

Remark 3.35. Finite-horizon simulation is useful for bounded experiments but is not used for the full preorder and quotient results, because composition requires compatible target horizons.

4 Self-reference, recoding, and translated copies of SICs

This section gives a formal treatment of the motivating question: can a SIC speak about, prove things about, or verify a translated copy of itself? The answer is yes, but only after separating three levels. First, there is the ordinary mathematical object $M = (C, H)$. Second, there is a code, name, or quotation of that object inside some formal language. Third, there is a proof-search SIC whose states contain formulas of that language. The safe form of self-reference is therefore not an unrestricted claim that a machine contains its own truth predicate. It is the more controlled claim that a formal system can code syntax, can define an invertible recoding map on that syntax, and can prove syntactic facts about the recoded code of a machine.

Definition 4.1. A coding universe is a set U whose elements are codes for the objects currently under discussion: symbols, finite strings, well-formed formulas, inference rules, finite proofs, SICs, ATPs, weak simulation witnesses, synchronous simulation maps, clocked simulation pairs, and effective simulation witnesses. A uniform recoding on U is a bijection $t \in [U \rightarrow U]$. For $A \subseteq U$, write $t[A] = \{t(a) : a \in A\}$.

C10: Set-sized coding universes

Authoritative replacement for: Definition 4.1, printed p. 20.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Definition 4.1 (Coding universe, corrected). Fix a set-sized family $\mathcal{O}$ of symbols, finite strings, well-formed formulas, inference rules, finite proofs, SICs, ATPs, and simulation witnesses under discussion. A coding universe for $\mathcal{O}$ is a set U together with specified injective coding maps whose domains cover the relevant members of $\mathcal{O}$. A uniform recoding on U is a bijection $t: U \rightarrow U$. For $A \subseteq U$, write $t[A] = \{t(a) : a \in A\}$.

All later coding and recoding claims are relative to the fixed set-sized family $\mathcal{O}$. No claim is made that one set contains codes for a proper class of all possible machines or witnesses.

Remark 4.2. The phrase “uniform” is meant to capture the idea that symbols, formulas, and rules are not recoded by unrelated tricks. There is one invertible operation at the level of codes, and the recoding of larger objects is induced by applying that same operation to their constituents or to their defining graphs.

Definition 4.3. Let $F = (x, y, z)$ be a formal system whose symbols, formulas, and rules are coded in U . A syntactic recoding for F is a uniform recoding t such that $t[x]$ is an alphabet, $t[y]$ is the corresponding set of recoded well-formed formulas, and each k -ary rule $r \in z$ has a recoded rule r^t defined by $r^t(t(a_1), \dots, t(a_k)) = t(r(a_1, \dots, a_k))$ whenever $r(a_1, \dots, a_k)$ is defined. The recoded formal system is $F^t := (x^t, y^t, z^t)$, where $x^t = t[x]$, $y^t = t[y]$, and $z^t = \{r^t : r \in z\}$.

Definition 4.4. The map t is a translator between F and F^t . Since t is invertible, F and F^t are not two essentially different languages. They are two presentations of one language up to isomorphism.

Proposition 4.5 (Where recoded copies live). *Let t be a syntactic recoding for $F = (x, y, z)$, and let $\varphi \in y$.*

1. *The recoded copy $t(\varphi)$ is an element of $y^t = t[y]$, the set of well-formed formulas of the recoded formal system F^t , and $y^t \subseteq U$. The existence of $t(\varphi)$ therefore requires nothing beyond the coding universe itself.*
2. *In general $t(\varphi) \notin y$: the formula sets y and y^t are two subsets of U related by the bijection t , and nothing in the definition places one inside the other.*
3. *One has $t(\varphi) \in y$ for every $\varphi \in y$ if and only if $t[y] \subseteq y$. Call such a t an endorecoding of F . If moreover y is finite, then $t[y] = y$, so an endorecoding of a finite system permutes its own formulas.*

Proof. Claims (1) and (2) restate the definition of a syntactic recoding: y^t is defined as the image $t[y]$, a subset of U . For (3), the displayed condition is exactly $t[y] \subseteq y$; if y is finite, injectivity of t gives $|t[y]| = |y|$, and a subset of y of full cardinality equals y . $\square$

Remark 4.6 (One mechanism, one vocabulary). The proposition answers a question every self-reference framework should be asked: a formula $\varphi \in y$ does not automatically have a recoded copy inside y . What always exists is $t(\varphi)$ inside U , and it lands back inside y exactly in the endorecoding case. This is also why, as of this version, the paper uses a single word family. A recoding is nothing over and above a change of coding: if the objects under discussion are coded by an injection e into U and t is a recoding, then $t \circ e$ is another coding of the very same objects, and the “recoded copy” of an object is that same object presented under the coding $t \circ e$. Objects never change; codes do. Through Version 43 recodings were called tags; the older word survives only in labels and in sentences describing the renaming.

Theorem 4.7 (Formal-system transport theorem). *If $F = (x, y, z)$ is a formal system and t is a syntactic recoding for F , then F^t is a formal system. Moreover, t is an isomorphism from F to F^t : it sends symbols to symbols, formulas to formulas, and rule applications in F exactly to rule applications in F^t .*

Proof. By definition, x^t is the recoded alphabet and y^t is the set of recoded well-formed formulas. If $r \in z$ is k -ary, then r^t is a partial function from $(y^t)^k$ to y^t . Thus z^t is a set of finitary rules on y^t , so F^t is a formal system. The defining equation for r^t says exactly that t preserves and reflects rule applications. $\square$

Theorem 4.8 (Proof transport and proof-length invariance). *Let F be a formal system, let t be a syntactic recoding for F , let $\Gamma \subseteq y$, and let $\varphi \in y$. Then $\Gamma \vdash_F \varphi$ if and only if $t[\Gamma] \vdash_{F^t} t(\varphi)$. When these equivalent conditions hold, shortest proof length is preserved: $\|t(\varphi)\|_{F^t}(t[\Gamma]) = \|\varphi\|_F(\Gamma)$.*

Proof. If $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ is a proof of φ from Γ , then $t(\pi) := \langle t(\pi(1)), \dots, t(\pi(n)) \rangle$ is a proof of $t(\varphi)$ from $t[\Gamma]$, because each hypothesis line and each rule application is transported by Theorem 4.7. Applying t^{-1} gives the converse. The two maps $\pi \mapsto t(\pi)$ and $\rho \mapsto t^{-1}(\rho)$ preserve the number of proof lines, so the minimum possible proof length is the same on both sides. $\square$

Definition 4.9. Let $M = (C, H)$ be a SIC over Y , and let t be a uniform recoding with $Y^t = t[Y]$. The recoded SIC $M^t = (C^t, H^t)$ over Y^t is defined by $C^t(\Delta, m) := t[C(t^{-1}[\Delta], m)]$ and $H^t(\Delta, m) := H(t^{-1}[\Delta], m)$ for every $\Delta \subseteq Y^t$ and every $m \in \mathbf{N}^+$.

Theorem 4.10 (SIC transport theorem). *If $M = (C, H)$ is a SIC over Y , then M^t is a SIC over Y^t . If M is deductively effective, deductively finitely effective, rule-driven, an enumerating ATP, or a target-search ATP for φ , then M^t has the corresponding property over Y^t , with target $t(\varphi)$.*

Proof. The stage-one condition is preserved because $C^t(\Delta, 1) = t[C(t^{-1}[\Delta], 1)] = t[t^{-1}[\Delta]] = \Delta$. Monotonicity is preserved because images under t preserve inclusion. The two halting axioms are preserved because H^t is defined by pulling the input back along t^{-1} , applying H , and then transporting the state forward again. Thus M^t is a SIC.

For deductive and ATP properties, use Theorem 4.8. For rule-driven behavior, a new recoded formula at stage $m + 1$ has the form $t(\beta)$, where β was a new formula in the original machine and hence a direct consequence of the previous original state. The recoded rule then makes $t(\beta)$ a direct consequence of the previous recoded state. $\square$

Corollary 4.11 (Transaction invariance under recoding). *If M is a target-search ATP for φ , then $\tau_{M^t}(t[\Gamma], t(\varphi)) = \tau_M(\Gamma, \varphi)$ whenever either side is defined, with both sides equal to ∞ when the target is not reached.*

Proof. For every stage m , one has $t(\varphi) \in C^t(t[\Gamma], m)$ if and only if $\varphi \in C(\Gamma, m)$. Therefore the first successful stage is the same. $\square$

Theorem 4.12 (Recoded copies are synchronously and clockedly simulation-equivalent). *Let M be a SIC over Y . Then $M \sim_{\text{syn}} M^t$. Consequently, $M \sim M^t$ under the principal clocked relation.*

Proof. Define

$$f_t \in [S_M \rightarrow S_{M^t}] \quad \text{by} \quad f_t(p) := t[p].$$

Because t is bijective, f_t is bijective and hence injective. For every $p \in S_M$ and every $m \in \mathbf{N}^+$,

$$f_t(C(p, m)) = t[C(p, m)] = C^t(t[p], m) = C^t(f_t(p), m),$$

and

$$H(p, m) = H^t(t[p], m) = H^t(f_t(p), m).$$

Thus f_t is a synchronous deterministic simulation. Its inverse $f_t^{-1}(q) = t^{-1}[q]$ is synchronous in the other direction. Using the identity clock in each direction gives clocked equivalence. $\square$

Definition 4.13. Let T be a formal system capable of coding finite syntax. Write $\langle A \rangle_{\text{code}}$ for the code of an object A . Let $\text{SIC}(u)$ be a formula of T meaning “ u codes a SIC,” let $\text{ATP}(u)$ mean “ u codes an ATP,” and let $\text{Recode}(u, v, w)$ mean “ w codes the recoded copy of the object coded by u under the recoding coded by v .”

Definition 4.14. A SIC M over a formula set Y is proof-bearing for T if members of Y include formulas of T , and if saying that M proves a formula θ from hypotheses Γ means that $\theta \in \bigcup_{m \in \mathbb{N}^+} C(\Gamma, m)$.

Definition 4.15. A proof-bearing SIC M is self-descriptive relative to T and t if T contains codes for M , t , and M^t and proves the relevant coding equations, for example $\text{Recode}(\langle M \rangle_{\text{code}}, \langle t \rangle_{\text{code}}, \langle M^t \rangle_{\text{code}})$.

Theorem 4.16 (Self-certification theorem). *Let $M = (C, H)$ be a deductively effective proof-bearing ATP for T over Y , and let Γ contain the axioms or background assumptions of T used for syntax coding. Suppose that T proves $\text{Recode}(\langle M \rangle_{\text{code}}, \langle t \rangle_{\text{code}}, \langle M^t \rangle_{\text{code}}) \wedge \text{SIC}(\langle M^t \rangle_{\text{code}})$. Then M eventually enumerates that formula from Γ . If M is run as a target-search ATP for that formula, then M halts on that target.*

Proof. The displayed formula is a theorem of T from the chosen background assumptions. Since M is deductively effective for T , every theorem of T from Γ appears in some stage of M on input Γ . The target-search claim is the same statement with that displayed formula chosen as the target. $\square$

Corollary 4.17. *Under the hypotheses of Theorem 4.16, a SIC can prove that a recoded copy of itself is a SIC. If T also proves the corresponding formula $\text{ATP}(\langle M^t \rangle_{\text{code}})$, then the SIC can prove that the recoded copy is an ATP.*

Proof. Apply Theorem 4.16 to the relevant coded verification statement. $\square$

Remark 4.18. This is the precise sense in which a SIC can “say” or “prove” things about a translated copy of itself. The object-level formula is not the machine itself. It is a coded assertion about the recoded code of the machine. This distinction prevents the section from confusing syntax, semantics, and truth.

4.1 The Picard Protocol and a minimal self-certifying ATP

Theorem 4.16 says that a sufficiently strong machine eventually certifies its recoded copy. This subsection turns that guarantee into a named success criterion and then exhibits the smallest machine meeting it.

Definition 4.19 (Picard Protocol). Let F be a formal system, let $Y \subseteq y$ be inference-closed, let t be a syntactic recoding, and let $\Gamma \subseteq Y$. A SIC M over Y satisfies the Picard Protocol for (t, Γ) if there is a target formula $\varphi^* \in Y \setminus \Gamma$ such that:

1. under a declared coding of the machines under discussion — one for which the code of a recoded object is the recoding of its code — φ^* asserts that M^t is an ATP;
2. every member of Γ is true, and every rule of F preserves truth, under that declared interpretation;
3. M is a rule-driven target-search ATP for φ^* , and M halts on input Γ .

The requirement $\varphi^* \notin \Gamma$ excludes the degenerate protocol in which the machine is handed its conclusion as a hypothesis, and rule-drivenness excludes machines that adjoin the conclusion without inference.

Example 4.20 (The Picard machine M_P). The construction proceeds syntax first, machine second, interpretation last.

Step 1: syntax. Let $F_P = (x_P, y_P, z_P)$ with $x_P = \{A, a, b\}$, with $y_P = \{Aa, Ab\}$ (two strings of length two), and with $z_P = \{r\}$, where r is the unary rule defined by $r(Aa) = Ab$ and $r(Ab) = Aa$. Set $Y := y_P$; it is inference-closed because both values of r lie in y_P .

Step 2: recoding. Let U consist of the finite strings over x_P . Let t swap $a \leftrightarrow b$ letterwise and fix A . Then t is an involutive bijection of U with $t[y_P] = y_P$, so t is an endorecoding in the sense of Proposition 4.5: every recoded formula lives inside y_P itself. Moreover $r^t = r$, hence $F_P^t = F_P$: the system is its own recoded copy.

Step 3: machine. Let $M_P := E_{F_P, Y, Ab}$ be the canonical target-search SIC for the target Ab (Definition 2.19).

Step 4: coding declaration. Declare $\text{code}(M_P) := a$. Since the code of a recoded object is the recoding of its code, $\text{code}(M_P^t) = t(a) = b$. Interpret the formula $A\beta$, for $\beta \in \{a, b\}$, as “the machine coded by β is an ATP.” Under this interpretation the target Ab asserts precisely that M_P^t is an ATP.

Step 5: input. Let $\Gamma := \{Aa\}$.

Theorem 4.21 (The Picard machine satisfies the Picard Protocol). M_P satisfies the Picard Protocol for $(t, \{Aa\})$ with target Ab , and

$$\tau_{M_P}(\{Aa\}, Ab) = 2.$$

Proof. Clause (3) of Definition 4.19: Y is inference-closed, so M_P is a rule-driven target-search ATP for Ab by Theorem 2.20. On input $\Gamma = \{Aa\}$ the run is

$$C(\Gamma, 1) = \{Aa\} \not\supseteq Ab, \quad C(\Gamma, 2) = D_{F_P}(\{Aa\}) = \{Aa, Ab\} \supseteq Ab,$$

so the machine halts at stage 2 and $\tau_{M_P}(\{Aa\}, Ab) = 2$.

Clause (2): the hypothesis Aa asserts that M_P is an ATP, and this is true by the theorem just cited. Note the order of construction: M_P was fully defined in Step 3, before the interpretation was declared in Step 4, so this is a verification, not a circle. The rule r sends $A\beta$ to $At(\beta)$, since $r(Aa) = Ab = At(a)$ and $r(Ab) = Aa = At(b)$. It is therefore the syntactic shadow of the SIC transport theorem (Theorem 4.10): if the machine coded by β is an ATP, then its recoded copy, coded by $t(\beta)$, is an ATP. Hence r preserves truth under the declared interpretation.

Clause (1) holds by Step 4. □

Proposition 4.22 (Minimality of the Picard machine). M_P is extremal in three simultaneous senses.

1. Every SIC satisfying the Picard Protocol has transaction count at least 2.
2. No formal system with a single well-formed formula supports the protocol.
3. At least one inference rule is required.

M_P attains all three bounds: two formulas, one rule, one hypothesis, and transaction count exactly 2.

Proof. For (1), $H(\Gamma, 1) = 1$ would force $\varphi^* \in C(\Gamma, 1) = \Gamma$, contradicting $\varphi^* \notin \Gamma$. For (2), if $y = \{\varphi^*\}$ and $\varphi^* \notin \Gamma$, then $\Gamma = \emptyset$; a proof from $\emptyset$ has no admissible first line, since a hypothesis line is impossible and every finitary rule needs at least one premise, so $\text{Con}_F(\emptyset) = \emptyset$, and by the rule-driven soundness proposition of Section 2 no stage of a rule-driven machine ever contains φ^* . For (3), with $z = \emptyset$ there are no direct consequences at all, so a rule-driven machine never grows past Γ , which excludes the target by hypothesis. □

Remark 4.23 (Mutual certification). Because t is an involution and $F_P^t = F_P$, the recoded machine M_P^t is, by the SIC transport theorem, a target-search ATP for $t(Ab) = Aa$ — and under the declared coding, Aa asserts that $(M_P^t)^t = M_P$ is an ATP. So the two machines certify each other: on the recoded input $t[\Gamma] = \{Ab\}$, the machine M_P^t proves that *its* recoded copy is an ATP, again at transaction count 2 by Corollary 4.11. The Picard Protocol is satisfied here in its strongest symmetric form.

Remark 4.24 (What the example does and does not show). In this micro-setting the arithmetization is externalized: the two formulas carry their meaning by declaration, and the soundness of the one rule is underwritten by a transport theorem proved in the metatheory. That is precisely what makes every step checkable by hand, and it isolates the logical skeleton of self-certification: code, recode, transport, search, halt. Machines that carry the verification internally rather than by declaration are covered by Theorem 4.16 under the hypotheses of Theorems 4.28 and 4.32: an effectively presented system interpreting enough arithmetic, a provably primitive recursive recoding, a canonical deductively effective machine, and an input containing the finitely many defining axioms. In both settings the Tarski–Gödel barrier (Theorem 4.26) marks the ceiling: ATP-hood of the recoded copy is certifiable, while a full truth predicate for recoded sentences is not.

The next two theorem schemes are standard consequences of the diagonal lemma and Tarski-style undefinability phenomena, now expressed in the language of recoded syntax; see, for example, [20, 21].

Theorem 4.25 (Equivariant diagonal lemma for recoded syntax). *Let T be a recursively axiomatized theory strong enough to represent primitive recursive functions and to prove the ordinary diagonal lemma. Let t be a primitive recursive syntactic recoding whose action on sentence codes is represented in T . For every one-variable formula $P(v)$ of T , there is a sentence θ such that $T \vdash \theta \leftrightarrow P(\overline{t(\langle \theta \rangle_{\text{code}})})$.*

Proof. This is the ordinary diagonal lemma applied after composing the usual self-substitution function with the represented recoding function. Informally, instead of making a sentence that talks about its own code, one makes a sentence that talks about the recoded version of its own code. The required fixed point exists because the recoding operation is effectively representable in T . $\square$

Theorem 4.26 (Tarski–Gödel barrier for recoded self-reference). *Let T be a consistent effectively axiomatized theory containing enough arithmetic for the diagonal lemma. Then no formula $\text{True}_t(v)$ of T can serve as a full internal truth predicate for all recoded sentences of T in the sense that T proves all biconditionals $\text{True}_t(t(\langle \theta \rangle_{\text{code}})) \leftrightarrow \theta$ for sentences θ of T .*

Proof. If such a predicate existed, Theorem 4.25 applied to the negation of True_t would produce a recoded liar sentence. The usual Tarski–Gödel contradiction follows. The recoding does not remove the obstruction because it is invertible and effectively represented; it only renames the codes. $\square$

Remark 4.27. The previous theorem is not a defect in the recoding framework. It is a useful guardrail. A SIC may prove many syntactic facts about its recoded copy, including that the copy is a SIC or ATP. What it may not have, on pain of the classical semantic paradoxes and incompleteness phenomena, is a complete internally correct truth predicate for all recoded sentences of its own language.

The following theorem settles, positively, what Version 42 stated as the *Self-Audit Compression Conjecture*. The hypotheses (i)–(iii) replace the informal word “natural” of the conjecture with explicit conditions; they hold for the standard arithmetized metatheory of Peano arithmetic and of the usual weaker fragments with efficient coding of syntax.

C01: Self-audit compression with explicit complexity hypotheses

Authoritative replacement for: Theorem 4.28, printed p. 26.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Theorem 4.28 (Self-audit compression theorem, corrected). Let T be a recursively axiomatized theory representing all primitive recursive functions, and let $\mathcal{F}$ be a set-sized family of effectively coded SICs together with a coded family of uniform recodings. Write $|u|$ for code length. Assume:

- (i) T proves uniform transport for the represented code transformer tc :

$$\forall u \forall v (\text{SIC}(u) \wedge \text{Recoding}(v) \rightarrow \text{SIC}(tc(u, v))),$$

and likewise for ATP;

- (ii) there is a polynomial q such that $|tc(a, b)| \leq q(|a| + |b|)$ for every relevant pair of codes;
- (iii) every true numerical instance $tc(a, b) = c$ has a T -proof of length polynomial in $|a| + |b| + |c|$;
- (iv) under the chosen numeral representation and proof calculus, numeral formation, universal instantiation, modus ponens, and substitution of equals have polynomial overhead in the displayed code lengths;
- (v) the true membership statements $\text{SIC}(\ulcorner M \urcorner)$, $\text{ATP}(\ulcorner M \urcorner)$ where applicable, and $\text{Recoding}(\ulcorner t \urcorner)$ have T -proofs of polynomial length in the relevant code lengths.

Then a polynomial P exists such that the verification statements $\text{SIC}(\ulcorner M^t \urcorner)$ and, where applicable, $\text{ATP}(\ulcorner M^t \urcorner)$ have T -proofs of length at most $P(|\ulcorner M \urcorner| + |\ulcorner t \urcorner|)$.

Proof. Use the fixed proof of uniform transport, the polynomial membership certificates, and polynomial-overhead instantiation and modus ponens to derive the transported predicate at $tc(\ulcorner M \urcorner, \ulcorner t \urcorner)$. Hypothesis (ii) bounds the output-code length, hypothesis (iii) supplies a polynomial proof of the code-computation equation, and hypothesis (iv) permits polynomial-overhead substitution of equals. Summing the polynomial bounds gives P .

2. the depth spectrum is not complete: there are rule-driven SICs M_0 and M_1 over the same formal system with identical depth spectra such that $M_0 \not\sim M_1$, and this failure survives quotienting by syntactic recodings, since no recoded copy of M_0 is clocked-equivalent to M_1 .

Since any normalization of the spectrum is a function of the raw spectrum, part (2) refutes the conjecture for every choice of normalization.

Proof. Fix the formal system $F_0 = (x_0, y_0, z_0)$ with $x_0 = y_0 = \{a, b\}$ (strings of length one) and $z_0 = \{r\}$, where r is the unary rule defined only by $r(b) = a$. Then $D_{F_0}(\Gamma) = \Gamma \cup \{a\}$ if $b \in \Gamma$ and $D_{F_0}(\Gamma) = \Gamma$ otherwise.

For (1), define M and N over $Y = y_0$, both with constantly zero halting maps, by

$$C_M(\Gamma, 1) = \Gamma, \quad C_M(\Gamma, m) = D_{F_0}(\Gamma) \quad (m \geq 2),$$

$$C_N(\Gamma, 1) = C_N(\Gamma, 2) = \Gamma, \quad C_N(\Gamma, m) = D_{F_0}(\Gamma) \quad (m \geq 3).$$

Both are SICs, and both are rule-driven because the only formula ever added is a , which is a direct consequence of any state containing b . Exactly as in Theorem 3.23, the identity state map with the clock $\rho(1) = 1, \rho(m) = m + 1$ for $m \geq 2$ witnesses $M \preceq N$, and the identity state map with the clock $\sigma(1) = \sigma(2) = 1, \sigma(m) = m - 1$ for $m \geq 3$ witnesses $N \preceq M$. Hence $M \sim N$. But

$$\tau_M(\{b\}, a) = 2 \neq 3 = \tau_N(\{b\}, a),$$

so clocked-equivalent machines can have different spectra.

For (2), define the static machines $M_0 = (C_{\text{stat}}, H_0)$ and $M_1 = (C_{\text{stat}}, H_1)$ over $Y = y_0$ by

$$C_{\text{stat}}(\Gamma, m) = \Gamma, \quad H_0(\Gamma, m) = 0, \quad H_1(\Gamma, m) = 1.$$

Both satisfy the SIC axioms (the halting-freeze axiom is vacuous for a static stage map) and both are vacuously rule-driven because no formula is ever added. They share one stage map, so $\tau_{M_0}(\Gamma, \varphi) = \tau_{M_1}(\Gamma, \varphi)$ for every input and target: the spectra are identical. Yet any clocked deterministic simulation from M_0 to M_1 would give $0 = H_0(p, m) = H_1(f(p), \rho(m)) = 1$, which is impossible, so $M_0 \not\sim M_1$. Finally, recoding cannot repair this: for any uniform recoding t , the recoded machine M_0^t has halting map $H_0^t(\Delta, m) = H_0(t^{-1}[\Delta], m) = 0$, so the same halting contradiction shows $M_0^t \not\sim M_1$. $\square$

Remark 4.31. The refutation isolates exactly what the spectrum measures and what it ignores. Transaction counts record *when* formulas first appear, so they are rescaled by admissible clocks (part 1) and are blind to halting observations (part 2), while clocked equivalence preserves halting exactly and forgives rescaling. A classification invariant for clocked equivalence must therefore record halting data and must be stated in a clock-invariant form, for example as the spectrum's equivalence class under admissible reparametrization. Whether some such repaired invariant is complete for an interesting subclass remains a reasonable further question, but the conjecture as posed in Version 42 is settled in the negative.

The third Version 42 conjecture asserted that every effectively coded deductively effective SIC has a conservative finite metadata extension that lets it prove the basic self-certification statement for each primitive recursive recoding in a fixed recoding family, without changing the underlying theorem set. The following theorem settles it positively.

Theorem 4.32 (Finite metadata self-description theorem). *Let T be a recursively axiomatized theory representing all primitive recursive functions, let $M = (C, H)$ be an effectively coded, deductively effective, proof-bearing SIC for T over Y , and let $(t_i)_{i \in \mathbb{N}}$ be a uniformly primitive recursive family of syntactic recodings, given by a single primitive recursive function u with $u(i, \cdot) = t_i$. Suppose T proves the arithmetized SIC transport theorem in the uniform form of Theorem 4.28(i). Then there is a finite definitional extension T' of T , obtained by adjoining function symbols for u and tc and predicate symbols SIC , ATP , and Recode together with their finitely many defining axioms, such that:*

1. T' is conservative over T : every formula in the original language provable in T' is already provable in T , so the underlying theorem set is unchanged;
2. for every $i \in \mathbb{N}$,

$$T' \vdash \text{Recode}(\langle M \rangle_{\text{code}}, \langle t_i \rangle_{\text{code}}, \langle M^{t_i} \rangle_{\text{code}}) \wedge \text{SIC}(\langle M^{t_i} \rangle_{\text{code}});$$

3. if Γ contains the background syntax-coding axioms of T together with the finitely many defining axioms of T' , then M eventually enumerates each of the self-certification formulas in (2) from Γ , and run as a target-search ATP for any one of them, M halts on that target.

Proof. The extension T' adds finitely many symbols, each with a defining axiom that determines it uniquely from the original language. Extensions by definitions are conservative, which proves (1); in particular the theorem set of the original language is untouched, so the metadata are pure bookkeeping.

For (2), fix i . The equation asserting that applying the coded recoding $u(i, \cdot)$ to $\langle M \rangle_{\text{code}}$ yields $\langle M^{t_i} \rangle_{\text{code}}$ is a true instance of a primitive recursive computation. Since T represents all primitive recursive functions and T' names u and tc , that instance is provable in T' , and by the defining axiom of Recode so is the first conjunct. The second conjunct follows from the uniform transport sentence instantiated at $\langle M \rangle_{\text{code}}$ and $\langle t_i \rangle_{\text{code}}$, exactly as in the proof of Theorem 4.28, followed by the provable code computation.

For (3), apply Theorem 4.16: each formula in (2) is a theorem of T' from Γ , and M is deductively effective, so each such formula appears at some stage of M on input Γ ; the target-search claim is the same statement with that formula chosen as target. $\square$

Remark 4.33. In Version 42 this was the Finite Metadata Self-Description Conjecture. The resolution identifies the promised “finite metadata” concretely: the finitely many defining axioms of a definitional extension. One finite package of names suffices for the whole infinite recoding family because the family is uniformly primitive recursive, so each instance is recovered by a provable computation rather than by a new axiom.

Theorem 4.34 (No-free-recoding-speedup theorem). *Pure invertible recoding alone does not yield transaction-count speedup for any target family. More precisely, let $(M_i, \Gamma_i, \varphi_i)_{i \in I}$ be a family in which each M_i is a target-search ATP for φ_i , and let t be an invertible recoding. Then*

$$\tau_{M_i^t}(t[\Gamma_i], t(\varphi_i)) = \tau_{M_i}(\Gamma_i, \varphi_i)$$

for every $i \in I$, including the value ∞ .

Proof. This is Corollary 4.11 applied pointwise. Since the two cost families agree exactly, they have the same finite values, failures, asymptotic growth, and normalized depth spectrum. $\square$

C02: Finite metadata self-description over the expanded language

Authoritative replacement for: Theorem 4.32, printed p. 28.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Theorem 4.32 (Finite metadata self-description theorem, corrected). Let T , M , and the uniformly primitive-recursive family $(t_i)_{i \in \mathbb{N}}$ satisfy the original coding and transport assumptions. Let T' be the finite definitional extension obtained by adjoining names for the uniform recoding function, the code transformer, and the predicates SIC, ATP, and Recode.

Then:

1. T' is conservative over T for sentences in the original language;
2. for every i , T' proves the corresponding recoding equation and the transported SIC statement (and the ATP statement where applicable);
3. let M' be a proof-bearing SIC whose formula domain contains the language of T' and which is deductively effective for T' . If Γ' contains the required background and defining axioms, then M' eventually enumerates each formula in clause 2 and, when run as a target-search ATP for one of them, halts on that target.

If the original machine M already operates over the expanded language and is deductively effective for T' , one may take $M' = M$.

Reason for the change. Deductive effectiveness for T alone does not imply the ability to represent or enumerate formulas containing the new symbols of T' .

Remark 4.35. The theorem explains why this section belongs between simulation and heuristic search. Recoding gives self-reference and isomorphism; it does not, by itself, make hard targets easy. Speedup enters later when the language is conservatively extended by derived rules, when a graph-search policy changes the explored frontier, or when proof-theoretic extraction supplies computational content not previously explicit in the search state.

5 Nondeterministic Search Policies and Heuristic Proof Search

Definition 5.1. Let $Y \subseteq y$ and let $p \subseteq Y$. An admissible one-step extension at p is a finite set $\Delta \subseteq Y$ such that every $\beta \in \Delta$ is a direct consequence of p .

Definition 5.2. A nondeterministic proof-search policy on Y is a function Σ defined on pairs (p, m) with $p \subseteq Y$ and $m \in \mathbf{N}^+$ such that $\Sigma(p, m)$ is a nonempty set of admissible one-step extensions at p .

Definition 5.3. Let $Y \subseteq y$ be inference-closed and let Σ be a nondeterministic proof-search policy on Y . A branch of the nondeterministic search generated by Σ from input $\Gamma \subseteq Y$ is a sequence $b = \langle p_1, p_2, \dots \rangle$ such that $p_1 = \Gamma$ and for each $m \in \mathbf{N}^+$ there exists $\Delta_m \in \Sigma(p_m, m)$ with $p_{m+1} = p_m \cup \Delta_m$. The deterministic branch SIC induced by b is the SIC $M_b = (C_b, H_b)$ over Y defined at every input by

$$C_b(\Delta, m) := \begin{cases} p_m, & \Delta = \Gamma, \\ \Delta, & \Delta \neq \Gamma, \end{cases} \quad H_b(\Delta, m) := 0.$$

The second clause is needed because Definition 2.3 requires C_b to be total on $\mathcal{P}(Y) \times \mathbf{N}^+$, whereas the branch b is generated from the single input Γ ; on every other input M_b simply idles, which is rule-driven vacuously.

Definition 5.4. Let $\varphi \in Y$. A nondeterministic target-search branch for φ is a branch $b = \langle p_1, p_2, \dots \rangle$ such that once $\varphi \in p_m$, all later states are frozen equal to p_m .

Definition 5.5. A nondeterministic policy Σ is proof-covering if for every input $\Gamma \subseteq Y$ and every finite proof $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ from Γ , there exists a branch $b = \langle p_1, p_2, \dots \rangle$ of the search from Γ such that $\{\pi(1), \dots, \pi(j)\} \subseteq p_j$ for every $j \in F_n$.

Theorem 5.6 (Branch soundness theorem). *Let $Y \subseteq y$ be inference-closed, let Σ be a nondeterministic proof-search policy on Y , and let $b = \langle p_1, p_2, \dots \rangle$ be a branch from input $\Gamma \subseteq Y$. Then the induced deterministic branch SIC M_b is rule-driven. Consequently, for every $m \in \mathbf{N}^+$, $p_m \subseteq \text{Con}_F(\Gamma) \cap Y$. Moreover the canonical map $a_{M_b}(p) = \text{Con}_F(p) \cap Y$ is a state abstraction from M_b to E_{FY} .*

Proof. By construction, every new formula adjoined at stage $m+1$ belongs to some admissible one-step extension at p_m , so every such formula is a direct consequence of p_m . Thus M_b is rule-driven. The soundness inclusion now follows from the rule-driven soundness proposition of Section 2. The state-abstraction statement is then an immediate application of Theorem 3.30. $\square$

Theorem 5.7 (Branch-Covering Theorem). *Let $Y \subseteq y$ be inference-closed, let Σ be a proof-covering nondeterministic proof-search policy on Y , let $\Gamma \subseteq Y$, and let $\varphi \in Y$. If $\Gamma \vdash_F \varphi$, then there exists a nondeterministic target-search branch for φ that reaches φ at stage at most $\|\varphi\|_F(\Gamma)$.*

Proposition 6.5 (A DAG version of the theorem graph). *Let $G_{F,\Gamma}^{\text{nov}}$ be the state theorem graph obtained by keeping only nontrivial edges $p \rightarrow q$, where $q = p \cup \{\beta\}$ and $\beta \notin p$. Then $G_{F,\Gamma}^{\text{nov}}$ is a directed acyclic graph.*

Proof. Along every edge of $G_{F,\Gamma}^{\text{nov}}$, the state strictly increases by set inclusion. A directed cycle would require a finite chain of strict inclusions that returns to its starting set. That is impossible. Hence the graph is a DAG. $\square$

Remark 6.6. The unmodified formula-level theorem graph need not be a DAG. There may be cycles caused by identity rules, reversible equivalences, conjunction introduction followed by conjunction elimination, double-negation transformations, commutativity or associativity conversions, or derived rules that return to an earlier formula. The state graph may also have loops if trivial edges $p \rightarrow p$ are retained. Three standard repairs are available. First, suppress trivial edges and use the novelty state graph just defined. Second, quotient formulas by a chosen equivalence relation and orient only proof-progressing edges. Third, unfold the search by time, so that a vertex is a pair (p, m) and every edge increases the stage number. The time-unfolded graph is automatically acyclic because the second coordinate strictly increases.

Definition 6.7. A target vertex for a formula $\varphi \in \mathcal{Y}$ is any vertex whose label is φ in the formula-labeled graph, or any state p with $\varphi \in p$ in the state theorem graph. A proof-search path from Γ to φ is a directed path from the initial vertex to a target vertex. In the informal maze analogy, the initial vertex is the entrance, the target vertex is the exit, and each labeled edge is one legal move through the maze.

Proposition 6.8 (Proofs as graph paths). *For a finitary formal system, every ordinary proof of φ from Γ determines a directed path in the state theorem graph from the initial state Γ to a target state containing φ . Conversely, every directed path from Γ to a state containing φ determines a proof of φ from Γ after redundant states and unused formulas are ignored.*

Proof. Given a proof, start at Γ and add each new formula when it first occurs as the conclusion of a rule application. Each addition is exactly a labeled edge in the state theorem graph. When the proof reaches φ , the corresponding path has reached a target state. Conversely, a path in the state theorem graph is a finite list of rule-labeled state transitions. Reading off the formulas added by those transitions, and keeping the recorded premises for each labeled edge, gives a line-by-line proof of every formula added along the path, in particular of φ once the target state is reached. $\square$

Theorem 6.9 (Shortest-path ATP principle). *Assume the relevant portion of the state theorem graph is finite or locally finite and effectively enumerable. If every edge has unit cost, then breadth-first search from the initial state finds a target state containing φ in the least possible number of graph edges, if such a target is reached. If a predecessor map is stored during the search, then the shortest path can be reconstructed by tracing predecessors backward from the target to the initial state.*

Proof. This is the standard breadth-first-search theorem for unweighted directed graphs. Breadth-first search explores vertices in nondecreasing distance from the initial vertex. Therefore the first time a target vertex is removed from the queue, its distance is minimal among all paths from the initial vertex. A predecessor map records, for each newly discovered vertex, the previous vertex and edge label by which it was discovered. Tracing those entries backward reconstructs a shortest path. Pulling the edge labels back to inference rules reconstructs a shortest proof in the corresponding graph metric. $\square$

Remark 6.14. This section suggests a practical implementation route; for the standard algorithmic background on implemented provers see [5, 10]. To build a small ATP, represent formulas as syntax trees or strings, represent proof states as finite sets of formulas, generate outgoing edges by applying inference rules to tuples from the current state, and run a graph-search algorithm with an early stopping test $\varphi \in p$. The output should not merely say “success.” It should return the predecessor chain and the edge labels, because those labels are the actual proof.

Definition 7.1. A weak successor-set theory is any first-order theory in a language containing $\in$, $\emptyset$, a pairing operation, a union operation, and a successor operation Succ , together with enough axioms to support extensional equality, empty set, singleton and pair formation, union, and the defining equation $\text{Succ}(a) = a \cup \{a\}$. Define $0 := \emptyset$, $1 := \{0\}$, $2 := \{0, 1\}$.

Proof. Once the language contains the displayed operations and equality behaves extensionally, one computes $\text{Succ}(\{0\}) = \{0\} \cup \{\{0\}\} = \{0, \{0\}\} = \{0, 1\}$. The statement does not require Infinity, Separation, Replacement, Foundation, or the full strength of ZF. On the other hand, the statement is not even formulable in pure Robinson arithmetic, because pure arithmetic lacks $\in$, $\emptyset$, pairing, and union. $\square$

7.2 Executable ATPs and a Python point of view

Theorem 7.5. *Let $Y \subseteq y$ be finite and effectively coded, let $\Gamma \subseteq Y$, and let $\varphi \in Y$. If the direct-consequence relation of F restricted to Y is decidable, then there exists a concrete breadth-first Python procedure that halts with success if and only if $\Gamma \vdash_F \varphi$ inside Y .*

Remark 7.6. In practice, formulas can be represented by strings, tuples, or syntax trees; inference rules can be represented by Python functions on tuples of formulas; and breadth-first search is just repeated closure under those functions. For toy targets such as $\text{Succ}(\{0\}) = \{0, 1\}$, a user can implement either rewriting rules or explicit inference rules and obtain a real executable prover, even though the present paper abstracts away from low-level programming detail.

Definition 7.9. Let $M = (C, H)$ be a target-search ATP for φ over Y . For input $\Gamma \subseteq Y$, define the transaction count $\tau_M(\Gamma, \varphi) := \min\{m \in \mathbf{N}^+ : \varphi \in C(\Gamma, m)\}$, if such an m exists, and define $\tau_M(\Gamma, \varphi) := \infty$ otherwise.

C03: Executable finite-fragment ATP theorem

Authoritative replacement for: Theorem 7.5, printed p. 33.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Theorem 7.5 (Corrected). Let $Y \subseteq y$ be finite, effectively coded, and inference-closed. Let $\Gamma \subseteq Y$ and $\varphi \in Y$. If the direct-consequence relation of F restricted to Y is decidable, then a concrete breadth-first procedure halts with success if and only if $\Gamma \vdash_F \varphi$.

Proof. Represent states as finite sets of codes and repeatedly compute $D_F(S)$. Inference-closedness keeps every stage inside Y . Since Y is finite, the chain stabilizes; by Theorem 2.17 its union is $\text{Con}_F(\Gamma) \cap Y$. The procedure therefore succeeds exactly when φ is derivable.

Without inference-closedness, iteration of $D_F(S) \cap Y$ can miss a proof that leaves Y and later returns.

Remark 7.19. The self-reference section also clarifies what speedup is not. Since M and M^t are isomorphic under an invertible recoding, Corollary 4.11 says that pure recoding preserves transaction counts. Therefore a recoded copy of a prover is not automatically a faster prover. Any real speedup must come from adding derived rules, changing the search policy, mining a proof for explicit data, or otherwise changing the proof-search structure in a way that is not merely a renaming.

Proof. Fix n . At line j of a proof sequence, one may either choose a hypothesis from Γ , which offers at most $|\Gamma|$ possibilities, or choose a rule application. For a k -ary rule there are at most $(j-1)^k$ ordered choices of previous proof lines from which to draw the premises, and there are κ_k

C38: Asymptotic stage-speedup requires divergence

Authoritative replacement for: Theorem 7.17 and Remark 7.18.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

In Theorem 7.17 replace hypothesis 3 by

$$\frac{q(n)}{p(n)} \longrightarrow \infty \quad (n \rightarrow \infty).$$

Under hypotheses 1–2 and this strengthened hypothesis, the augmented target-search ATP uses an asymptotically vanishing fraction of the logical stage transitions required by every line-by-line realization whose proof length is bounded below by $q(n)$.

If one assumes only that $q(n)/p(n)$ is unbounded, the valid conclusion is weaker: the speedup factor is unbounded along a subsequence. No eventual or limit asymptotic separation follows from unboundedness alone. The comparison remains a logical-stage comparison unless an independent wall-clock cost model is supplied.

choices of such a rule. Therefore the number of possible local ways to generate line j is at most $|\Gamma| + \sum_{k=1}^a \kappa_k(j-1)^k$. Multiplying these bounds over $j = 1, \dots, n$ bounds the number of length- n proof sequences ending with any specified target, hence also the number ending with φ . The second displayed inequality is the first applied at $n = m$. $\square$

Remark 8.4. Theorem 8.3 answers the upper-bound question in the most direct way available from cardinal data alone. It is crude, because it counts many sequences that are not genuine proofs and many different construction plans that collapse to the same formulas. Still, it makes explicit where the dependence on $|\Gamma|$, on the rule-cardinalities κ_k , and on proof length enters.

Remark 8.5. The comparison with password finding is suggestive but not exact. A theorem is like a password candidate in one important respect: once a candidate proof is exhibited, local checking is usually much easier than search. A proof is a certificate, just as a password guess is a candidate string. But proof search differs from password search because proofs have reusable internal structure. Lemmas, macro rules, search order, breadth-first layers, and nondeterministic branching all matter. In password search there is usually no analogue of a useful intermediate lemma; a wrong partial password rarely helps the next guess. In theorem proving, by contrast, partial derivations can be retained and reused.

Remark 8.6. A speedup theorem can, in a broad asymptotic sense, play the same role for password finders that proof-complexity speedup plays for theorem provers: it can prove that one search discipline beats another on a family of search instances. The analogy should not be pushed too far, but the structural moral is the same. Search procedures can be compared mathematically by how quickly they reach certified solutions.

9 Universality, Finite Effectiveness, and Computability

Standard computability background may be found in [18]. The results in this section are most transparent on finite fragments, where every state can be coded as a finite object.

Definition 9.1. A SIC over $Y \subseteq y$ is state-restricted if $Y \neq y$.

Definition 9.2. Define the identity SIC on y by $U_{\text{all}} = (C_{\text{all}}, H_{\text{all}})$, where $C_{\text{all}}(\Gamma, m) = \Gamma$, $H_{\text{all}}(\Gamma, m) = 0$ for all $\Gamma \subseteq y$ and all $m \in \mathbf{N}^+$.

Proposition 9.3 (Characterization of simulation into the identity SIC). *Let $M = (C, H)$ be a SIC over y . Then $M \preceq U_{\text{all}}$ if and only if*

$$C(p, m) = p \quad \text{and} \quad H(p, m) = 0$$

for every $p \in S_M$ and every $m \in \mathbf{N}^+$.

Proof. Suppose (f, ρ) is a clocked deterministic simulation from M to U_{all} . Halting preservation gives $H(p, m) = 0$. Since U_{all} is static,

$$f(C(p, m)) = C_{\text{all}}(f(p), \rho(m)) = f(p).$$

Injectivity of f yields $C(p, m) = p$. Conversely, if M is static and never halts, then the identity state map together with the identity clock is a clocked deterministic simulation from M to U_{all} . $\square$

Corollary 9.4. U_{all} is not universal under the principal clocked deterministic simulation relation for the class of all SICs over y .

C11: Computability on effectively presented state domains

Authoritative replacement for: Definitions 9.9–9.10, printed p. 36.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Definition 9.9 (Corrected). Let Y be effectively coded and let $\mathcal{D} \subseteq \mathcal{P}(Y)$ be an effectively presented state domain with an injective natural-number coding of its members and decidable code image. Require $C[\mathcal{D} \times \mathbb{N}_+] \subseteq \mathcal{D}$. A SIC restricted to $\mathcal{D}$ is *computable* when total computable functions produce codes for $C(A, m)$ and the value $H(A, m)$ from a code for $A \in \mathcal{D}$ and m .

The domain $\mathcal{D}$ may, for example, consist of finite, recursive, or reachable states. When Y is finite, one may take $\mathcal{D} = \mathcal{P}(Y)$.

Proof. Any SIC with a nonconstant stage evolution or a nonzero halting observation fails the characterization in Proposition 9.3. $\square$

Theorem 9.5. *Let $Y \subseteq y$ be inference-closed. The canonical rule-enumeration SIC $E_{F,Y}$ is not universal under clocked deterministic simulation for the class of all rule-driven SICs over Y .*

Proof. Define the always-halted static SIC $Q_Y = (C_Q, H_Q)$ by $C_Q(\Gamma, m) = \Gamma$ and $H_Q(\Gamma, m) = 1$ for every input and stage. It is a rule-driven SIC because it never adds a new formula. If a clocked deterministic simulation from Q_Y into $E_{F,Y}$ existed, halting preservation would give $1 = 0$, since the halting bit of $E_{F,Y}$ is constantly zero. This is impossible. $\square$

Remark 9.6. Theorem 3.30 remains valuable: it supplies a canonical consequence-closure abstraction for every rule-driven SIC. It does not supply a faithful clocked deterministic embedding, and therefore it no longer implies a universality theorem.

Theorem 9.7. *If $Y \subseteq y$ is finite and inference-closed, then $E_{F,Y}$ is deductively finitely effective.*

Proof. For fixed $\Gamma \subseteq Y$, the chain $C_{F,Y}(\Gamma, 1) \subseteq C_{F,Y}(\Gamma, 2) \subseteq \dots$ is a monotone sequence of subsets of the finite set Y . Hence it stabilizes after finitely many steps. Since $E_{F,Y}$ is deductively effective, that eventual fixed point is exactly $\text{Con}_F(\Gamma) \cap Y$. $\square$

Definition 9.8. A finite register-machine program is a finite list of instructions operating on finitely many registers ranging over $\mathbf{N}$. The basic instructions are zeroing a register, incrementing a register, copying one register into another, conditional jump on whether a register is zero, and halt.

Definition 9.9. Assume that $Y \subseteq y$ is effectively coded. A SIC $M = (C, H)$ over Y is computable if there exist total computable functions c and h such that for every code a of a subset $A_a \subseteq Y$ and every $m \in \mathbf{N}^+$, the value $c(a, m)$ is a code for $C(A_a, m)$ and the value $h(a, m)$ is $H(A_a, m)$.

Definition 9.10. A SIC $M = (C, H)$ over an effectively coded domain is computationally effective if there exists a finite register-machine program that, on coded input (a, m) , halts and outputs a code for $C(A_a, m)$ together with the value $H(A_a, m)$.

Theorem 9.11. *Assume that $Y \subseteq y$ is finite, effectively coded, and inference-closed, and assume that the direct-consequence relation of F restricted to Y is decidable. Then $E_{F,Y}$ is computable.*

Proof. Because Y is finite, every subset of Y has a canonical finite code. Given a code for $C_{F,Y}(\Gamma, m)$, one may effectively inspect all relevant tuples from that finite set and decide which formulas of Y are direct consequences. Hence one can compute a code for $D_F(C_{F,Y}(\Gamma, m)) = C_{F,Y}(\Gamma, m + 1)$. Since $H_{F,Y}$ is constantly 0, the halting output is computable as well. $\square$

Definition 9.12. The consequence problem on Y is the decision problem: given $\Gamma \subseteq Y$ and $\psi \in Y$, decide whether $\Gamma \vdash_F \psi$.

Theorem 9.13. *Suppose $M = (C, H)$ is a computable deductively finitely effective SIC over $Y \subseteq y$ such that for every $\Gamma \subseteq Y$ there exists $m \in \mathbf{N}^+$ with $H(\Gamma, m) = 1$ and $C(\Gamma, m) = \text{Con}_F(\Gamma) \cap Y$. Then the consequence problem on Y is decidable.*

Proof. Given codes for Γ and ψ , compute the stages of M until the first m with $H(\Gamma, m) = 1$ appears. By hypothesis such an m exists. Since M is computable, membership in $C(\Gamma, m)$ is decidable. Therefore one can decide whether $\psi \in C(\Gamma, m)$, which is equivalent to $\Gamma \vdash_F \psi$. $\square$

C11: Computational effectiveness and the internal Church–Turing theorem

Authoritative replacement for: Definition 9.10 and Theorem 9.15, printed p. 37.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Definition 9.10 (Corrected). A SIC over an effectively presented state domain $\mathcal{D}$ is computationally effective if one finite register-machine program, on a code for (A, m) with $A \in \mathcal{D}$, halts and outputs a code for $C(A, m)$ together with $H(A, m)$.

Theorem 9.15 (Corrected). For a SIC over an effectively presented state domain, computational effectiveness is equivalent to computability in the preceding sense.

Proof. A register-machine program computes total coded stage and halting functions. Conversely, the standard equivalence of classical models of computation converts the two total computable coded functions into finite register-machine programs, which can be combined. The theorem does not assume a natural-number code for every subset of an infinite Y ; such a code cannot exist when $\mathcal{P}(Y)$ is uncountable.

Corollary 9.14. *If the consequence problem on Y is undecidable, then there is no computable deductively finitely effective SIC over Y with the preceding halting-to-completion property.*

Proof. Otherwise the theorem would make the consequence problem decidable. $\square$

Theorem 9.15 (Internal Church–Turing theorem for SICs). *Let $M = (C, H)$ be a SIC over an effectively coded domain. Then M is computationally effective if and only if M is computable.*

Proof. One direction is immediate because register-machine computability gives total computable output functions. For the other direction, if the stage and halting maps are total computable, then the standard equivalence of classical models of computation yields a finite register-machine program computing them [18]. Combining the two computations gives one finite program producing both outputs. $\square$

Remark 9.16. This theorem is internal to the SIC framework. It is not a proof of the historical Church–Turing thesis in its philosophical form.

10 Propositional Proof Systems, p -Simulation, and Feasible Interpolation

This section imports the Cook–Reckhow theory of propositional proof systems [3] into the $F = (x, y, z)$ formal-system vocabulary of Section 1. The organizing source for the results assembled here is Beame’s proof-complexity lecture notes, written up by Sabharwal for the 2000 IAS/Park City Mathematics Institute Graduate Summer School and published as [1]; individual theorems are additionally attributed to their original authors as in that source. The point of the section is a dictionary: a propositional proof system in the classical Cook–Reckhow sense is exactly a syntactically restricted formal system $F = (x, y, z)$ together with a size function on y , and the canonical breadth-first target-search SIC of Section 2 already computes, under that size function, exactly the classical notion of proof size. Several classical theorems then become instances or refinements of theorems already proved for SICs elsewhere in this paper: the Cook–Reckhow theorem becomes a statement about polynomially bounded shortest-proof functions (Theorem 10.6); p -simulation becomes a second, genuinely different preorder alongside those of Section 3 (Definition 10.8); and the resolution lower bound for the pigeonhole principle instantiates a generalization of the Abstract Asymptotic Speedup Principle with an unconditional separation (Corollary 10.21).

10.1 Proof size as a refinement of proof length

Definition 10.1. Let $F = (x, y, z)$ be a formal system. A *size function* on F is a function $|\cdot| \in [y \rightarrow \mathbf{N}^+]$ assigning to each formula the length of its string representation over x . For a finite sequence $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ of members of y , define $\text{size}(\pi) := \sum_{j=1}^n |\pi(j)|$. For $\Gamma \subseteq y$ and $\varphi \in y$, define the shortest-proof *size*

$$\|\varphi\|_F^{\text{size}}(\Gamma) := \min\{\text{size}(\pi) : \pi \text{ is a proof from } \Gamma \text{ with final line } \varphi\},$$

if $\Gamma \vdash_F \varphi$, and $\|\varphi\|_F^{\text{size}}(\Gamma) := \infty$ otherwise.

Remark 10.2. Since every formula has size at least 1, $\text{size}(\pi) \geq |\pi|$ for every proof π , so $\|\varphi\|_F^{\text{size}}(\Gamma) \geq \|\varphi\|_F(\Gamma)$. Concatenation of proofs adds both line counts and sizes, which is the only property of $|\pi|$ used anywhere in Section 2. Consequently the canonical breadth-first SIC of Definition 2.19, the Proof-Horizon Theorem (Theorem 2.12), and the transaction-count apparatus of Section 7

all relativize verbatim under the substitution $|\pi| \mapsto \text{size}(\pi)$; write $B_{F,Y,\varphi}^{\text{size}}$ for the size-relativized canonical breadth-first target-search SIC, whose halting stage on input Γ is exactly $\|\varphi\|_F^{\text{size}}(\Gamma)$.

Definition 10.3. A formal system $F = (x, y, z)$ is *polynomial-time checkable* if there is an algorithm that, given $k \in \mathbf{N}^+$, a rule $r \in z_k(F)$, and a tuple $(a_1, \dots, a_k, \beta) \in y^{k+1}$, decides in time polynomial in $|a_1| + \dots + |a_k| + |\beta|$ whether $r(a_1, \dots, a_k) = \beta$.

Definition 10.4. Let $F = (x, y, z)$ be polynomial-time checkable with a distinguished formula $\perp \in y$, and let Π be a set of finite strings over an auxiliary alphabet together with a relation $\text{Ver} \subseteq \Pi \times \mathcal{P}(y)$, decidable in time polynomial in the length of its first argument and the size of its second, whose intended reading is that π verifies the refutation of Γ . A proof in the sense of Definition 1.4 is the special case in which Π is the set of formula sequences and Ver is the condition of Definition 1.4 with final line $\perp$. Call (F, Π, Ver) a *Cook–Reckhow system* for UNSAT if, for every finite $\Gamma \subseteq y$ coding an unsatisfiable propositional formula, $\Gamma \vdash_F \perp$, and conversely every $\Gamma \vdash_F \perp$ codes an unsatisfiable formula. Define the *proof-complexity function* of F by

$$s_F(\Gamma) := \|\perp\|_F^{\text{size}}(\Gamma)$$

for finite unsatisfiable $\Gamma \subseteq y$. Say F is *polynomially bounded* if there is a polynomial q with $s_F(\Gamma) \leq q(|\Gamma|)$ for every such Γ , where $|\Gamma| := \sum_{a \in \Gamma} |a|$. Where no confusion results we write F for the whole system (F, Π, Ver) , as in s_F just above; every system named in the remainder of this section — resolution, Frege, extended Frege — is of the formula-sequence kind, for which Π and Ver are those of Definition 1.4 and may be left implicit.

Remark 10.5 (Why the auxiliary proof alphabet is needed). Definition 10.4 restates Cook and Reckhow’s original definitions of a proof system and its complexity [3]; see also [1], Definitions 1.1–1.4. Soundness and completeness of the proof system correspond exactly to the two displayed conditions on refutability of Γ .

The generality of (Π, Ver) over the formula sequences of Definition 1.4 is not cosmetic, and it is worth saying exactly where it is used. In a formal system $F = (x, y, z)$ every line of a proof is a well-formed formula that is either a hypothesis or a direct consequence of earlier lines; there is no line at which an arbitrary string may be guessed and then checked. Consequently the forward direction of Theorem 10.6 — which, from $\text{NP} = \text{coNP}$, must manufacture a proof system whose proofs are the polynomial-size NP-witnesses for UNSAT — cannot be carried out with formula sequences alone. Nor can one evade this by taking a unary rule r with $r(\gamma) = \perp$ exactly when γ is unsatisfiable: deciding whether $r(\gamma) = \perp$ is then deciding UNSAT, so such an F is not polynomial-time checkable in the sense of Definition 10.3. The auxiliary alphabet is what makes the classical theorem statable here. Everything else in this section — the size function, the proof-complexity function s_F , p -simulation, and the resolution and Frege examples — lives comfortably inside the formula-sequence special case, since in those systems a refutation *is* a sequence of clauses or formulas.

Theorem 10.6 (Cook–Reckhow Theorem, formal-system form). *There exists a polynomial-time checkable, polynomially bounded Cook–Reckhow system $F = (x, y, z)$ for UNSAT if and only if $\text{NP} = \text{coNP}$.*

Proof. This is Theorem 1.5 of [1], following [3], transcribed into the present vocabulary. If $\text{NP} = \text{coNP}$, then since UNSAT is coNP-complete it lies in NP, so it has polynomial-size witnesses checkable in polynomial time; declaring each such witness a one-line proof from Γ gives a polynomial-time checkable, polynomially bounded F . Conversely, if such an F exists, then nondeterministically guessing a proof of size at most $q(|\Gamma|)$ and checking it in polynomial time (Definition 10.3) places UNSAT in NP; since UNSAT is coNP-complete, this gives $\text{coNP} \subseteq \text{NP}$, and the complementary argument using TAUT gives $\text{NP} \subseteq \text{coNP}$. $\square$

C39: Proof size is a budget, not a SIC clock

Authoritative replacement for: Remark 10.2 and every occurrence of the “size-relativized canonical target-search SIC”.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

For a formula-sequence proof π , retain

$$\text{size}(\pi) = \sum_j |\pi(j)|, \quad \|\varphi\|_F^{\text{size}}(\Gamma) = \min\{\text{size}(\pi) : \pi \text{ proves } \varphi \text{ from } \Gamma\}.$$

But do *not* identify this quantity with a SIC stage number. In general a hypothesis $\gamma \in \Gamma$ can have $|\gamma| > 1$, so the filtration by total proof size need not satisfy the SIC axiom $C(\Gamma, 1) = \Gamma$.

Instead define the proof-size budget filtration

$$Q_{F,Y}^{\text{size}}(\Gamma, B) := \{\psi \in Y : \|\psi\|_F^{\text{size}}(\Gamma) \leq B\}.$$

This is a cost-indexed benchmark, not in general a SIC. For a target φ , its least required budget is exactly

$$B_F^{\text{size}}(\Gamma, \varphi) = \|\varphi\|_F^{\text{size}}(\Gamma).$$

Accordingly, throughout Section 10 replace “size-relativized canonical target-search SIC”, “halting stage”, and “size-relativized transaction count” by “proof-size budget benchmark”, “least budget”, and “minimum proof size”, respectively. Haken, Frege, Polynomial Calculus, Tseitin, and random-CNF proof-complexity statements remain statements about minimum certificate/proof size; they do not thereby become runtime bounds for a search algorithm.

Remark 10.7. Theorem 10.6 shows that the existence question for polynomially bounded formal systems is exactly as hard as NP versus coNP. This gives the abstract asymptotic speedup apparatus of Section 7 a concrete stake: Theorem 7.17 there is schematic, and Theorem 10.6 explains why no single choice of derived rules D can be expected to make every family polynomially bounded, on pain of resolving a major open problem.

10.2 p -simulation: a second preorder on formal systems

Definition 10.8. Let $Y \subseteq y$ be a common coding of propositional formulas, and let $F = (x, y, z)$ and $F' = (x', y', z')$ be polynomial-time checkable Cook–Reckhow systems for UNSAT sharing the domain Y . Say F' *p-simulates* F , written $F \preceq_p F'$, if there is a polynomial-time computable function h such that for every finite $\Gamma \subseteq Y$ and every proof π of $\perp$ from Γ in F , $h(\pi)$ is a proof of $\perp$ from Γ in F' . Say F and F' are *polynomially equivalent* if $F \preceq_p F'$ and $F' \preceq_p F$.

Remark 10.9. Definition 10.8 restates Definitions 1.6–1.7 of [1], following Cook and Reckhow [3]. It differs in kind, not merely in strength, from every simulation relation of Section 3. Weak, synchronous, and clocked simulation compare two SICs’ entire stage-by-stage evolution via a witness on state spaces $S_M \times S_N \subseteq \mathcal{P}(Y) \times \mathcal{P}(Y)$ (Definitions preceding Theorem 3.2 and Propositions 3.4–3.11); recoding (Section 4) additionally requires the translation to be an invertible bijection on the coding universe. p -simulation instead compares two formal systems only through their finite refutations of a shared target $\perp$, via a proof-to-proof map h that need be neither injective nor derived from any map on states. It is, in this precise sense, the coarsest of the comparison notions considered in this paper, exactly as weak simulation was the coarsest SIC-level notion in Theorem 3.2 – though unlike weak simulation, it does not collapse universally, because polynomial-time computability of h is a genuine restriction once size, rather than merely stage number, is being tracked.

Proposition 10.10 (*p-simulation bounds proof complexity*). *If $F \preceq_p F'$ via h , then there is a polynomial q such that $s_{F'}(\Gamma) \leq q(s_F(\Gamma))$ for every finite unsatisfiable $\Gamma \subseteq Y$.*

Proof. Fix Γ and let π be a shortest proof of $\perp$ from Γ in F , so $\text{size}(\pi) = s_F(\Gamma)$. Since h is computable in time polynomial in the size of its input, $\text{size}(h(\pi))$ is bounded by a polynomial q in $\text{size}(\pi)$, and $h(\pi)$ is a proof of $\perp$ from Γ in F' by Definition 10.8. Hence $s_{F'}(\Gamma) \leq \text{size}(h(\pi)) \leq q(s_F(\Gamma))$. $\square$

Proposition 10.11 (*p-simulation is a preorder*). *The relation $\preceq_p$ is reflexive and transitive.*

Proof. Reflexivity is witnessed by the identity translation. For transitivity, if $F \preceq_p F'$ via h and $F' \preceq_p F''$ via h' , then $h' \circ h$ is polynomial-time computable, being a composition of two polynomial-time computable functions, and it sends a proof of $\perp$ from Γ in F to one in F'' by two applications of Definition 10.8. This mirrors Proposition 3.19, the corresponding composition fact for clocked deterministic simulation. $\square$

Corollary 10.12. *Polynomial equivalence of formal systems is an equivalence relation, and $\preceq_p$ induces a partial order on its equivalence classes, exactly as $\preceq$ does for SICs (Corollary following Proposition 3.19).*

Theorem 10.13 (Frege systems are polynomially equivalent). *Every two Frege systems, formalized as formal systems $F = (x, y, z)$ whose rule set z consists of finitely many axiom schemas and finitely many sound, implicational complete inference rules such as modus ponens, are polynomially equivalent under $\preceq_{\text{p}}$.*

C40: Cook–Reckhow certificates are verifier-defined

Authoritative replacement for: Definition 10.4, Theorem 10.6, Definition 10.8, and Propositions 10.10–10.12.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

A general Cook–Reckhow refutation system for UNSAT is a pair $P = (\Pi, \text{Ver}_P)$ where Π is a set of finite certificate strings and

$$\text{Ver}_P(\pi, \Gamma)$$

is decidable in time polynomial in the encoded lengths of π and finite input Γ , with

$$\Gamma \text{ is unsatisfiable} \iff \exists \pi \in \Pi \text{ Ver}_P(\pi, \Gamma).$$

Define

$$s_P(\Gamma) := \min\{|\pi| : \text{Ver}_P(\pi, \Gamma)\},$$

with the certificate-length convention fixed by the chosen polynomially reasonable encoding. A formula-sequence formal system $F = (x, y, z)$ is a special case: Π consists of encoded formula sequences and Ver_P checks Definition 1.4 with final line $\perp$. For those systems the manuscript’s sum-of-formula-sizes measure may continue to be used as the declared proof-size measure.

Cook–Reckhow theorem, corrected form. There exists a polynomial-time verifiable, polynomially bounded Cook–Reckhow system for UNSAT if and only if $\text{NP} = \text{coNP}$. If equality holds, UNSAT has polynomial-size NP certificates. Conversely, a polynomially bounded polynomial-time verifier places UNSAT in NP; since UNSAT is coNP-complete, $\text{coNP} \subseteq \text{NP}$, and taking complements yields equality.

p-simulation, corrected form. For Cook–Reckhow systems P, P' , write $P \preceq_p P'$ if a polynomial-time computable map h sends every valid P -certificate π for Γ to a valid P' -certificate $h(\pi)$ for the same Γ . Then $s_{P'}(\Gamma) \leq q(s_P(\Gamma))$ for some polynomial q , and $\preceq_p$ is a preorder. The later resolution/Frege/Polynomial-Calculus examples are the formula-sequence special case.

C04: Polynomial equivalence of Frege systems

Authoritative replacement for: Theorem 10.13, printed p. 40.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

The theorem remains standard, but replace its proof by the following.

Corrected proof. Fix two Frege systems with finite collections of axiom and inference-rule schemes. For each scheme of either system, implicational completeness of the other supplies a uniform derivation of every substitution instance. The size of that derivation is polynomial in the size of the instantiated formulas (under standard encodings it is linear up to a system-dependent polynomial), not constant. Substitution and the construction of the translated derivation are computable with polynomial overhead. Replacing every line and rule application of a proof by its uniform polynomial simulation yields a proof of polynomially related size in the other Frege system. Applying the construction in both directions proves polynomial equivalence.

Proof. This is Theorem 1.8 of [1]. Given two such systems with axiom/rule sets z and z' , every instance of a schema of z' has a constant-size proof in z because z is implicationally complete, and this fixed finite collection of constant-size proofs can be spliced into any z' -proof, line by line, in time polynomial in the size of that proof, yielding an F -proof of the same conclusion of polynomially related size. Since the two systems are arbitrary Frege systems, both directions of $\preceq_p$ follow. $\square$

10.3 The pigeonhole principle: an unconditional instance of asymptotic speedup

Definition 10.14. Fix a countable set of Boolean variables. Let x consist of the variables, their negations, and a disjunction separator; let y consist of all finite clauses (finite sets of literals) over x , with size function $|\cdot|$ one more than the number of literal occurrences, so that the empty clause has size 1 and $|\cdot|$ takes values in $\mathbf{N}^+$ as Definition 10.1 requires; let $\perp$ be the empty clause; and let $z = \{\text{Res}\}$, where Res is the binary rule

$$\text{Res}(C \vee p, D \vee \neg p) := C \vee D,$$

defined whenever the pivot variable p occurs positively in the first clause and negatively in the second. Write $F_{\text{res}} = (x, y, z)$ for the resulting formal system.

Remark 10.15. F_{res} is polynomial-time checkable in the sense of Definition 10.3, since verifying a single resolution step takes time polynomial in the size of the two premises and the conclusion. It is a Cook–Reckhow system for UNSAT restricted to CNF inputs, and it is the formal system underlying the canonical rule-enumeration and target-search SICs $E_{F_{\text{res}}, Y}$ and $E_{F_{\text{res}}, Y, \perp}$ of Definitions 2.16 and 2.19.

Definition 10.16. For $n \geq 1$, let $\Gamma_n \subseteq y$ be the standard CNF encoding of the pigeonhole principle PHP_n^{n-1} over variables $p_{i,j}$ ($1 \leq i \leq n$, $1 \leq j \leq n-1$), consisting of the totality clauses $\bigvee_j p_{i,j}$ and the injectivity clauses $\neg p_{i,j} \vee \neg p_{i',j}$ for $i \neq i'$, as in [1], Section 2.1. Each Γ_n is unsatisfiable, coding the fact that n pigeons cannot be injectively mapped into $n-1$ holes.

Theorem 10.17 (Haken’s resolution lower bound, formal-system form). *There is a constant $c > 0$ such that $s_{F_{\text{res}}}(\Gamma_n) \geq 2^{cn}$ for every n . Equivalently, the size-relativized canonical breadth-first target-search SIC $B_{F_{\text{res}}, Y, \perp}^{\text{size}}$ does not halt on input Γ_n before stage 2^{cn} .*

Attribution and sketch. This is Haken’s theorem [9], as presented via the width-size method of Ben-Sasson and Wigderson in [1], Sections 2.1.4–2.2. In outline: a resolution refutation is first shown, by a restriction argument, to be reducible to one with no large clauses while still refuting a nontrivial restricted instance; every refutation of that restricted instance must contain a clause of “medium” complexity, in the sense of the number of critical assignments it certifies; and every medium-complexity clause must itself be large. Combining these facts with the restriction argument yields an exponential lower bound on the number of resolution steps, hence on size. The final “equivalently” clause is the size-relativized instance of the Proof-Horizon Theorem, Theorem 2.12, noted in the remark following Definition 10.1. $\square$

Theorem 10.18 (Extended Frege upper bound). *There is a polynomial p and an extended Frege system F_{EF} , in the sense of Theorem 10.13 together with extension rules for auxiliary variables, such that $s_{F_{\text{EF}}}(\Gamma_n) \leq p(n)$ for every n .*

Attribution. This is the extended Frege translation, due to Buss, of the ordinary inductive proof of the pigeonhole principle, presented in [1], Section 2.1.2: the induction on n used to prove PHP_n^{n-1}

Theorem 10.19 (Two-System Size-Speedup Principle). *Let $F = (x, y, z)$ and $F' = (x', y', z')$ be polynomial-time checkable Cook–Reckhow systems for UNSAT sharing a common family $(\Gamma_n)_{n \geq 1}$ of finite unsatisfiable inputs. Suppose there are $p, q \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$ such that:*

- Corollary 10.21** (An unconditional instance of abstract speedup). *Instantiating Theorem 10.19 with $F = F_{\text{res}}$, $F' = F_{\text{EF}}$, and (Γ_n) the pigeonhole family of Definition 10.16, the hypotheses are met with $p(n)$ polynomial and $q(n) = 2^{cn}$, by Theorems 10.17 and 10.18. This is the paper’s first unconditional instance of the abstract speedup phenomenon: the finite Conservative Compilation Theorem (Theorem 7.12) speaks only of finitely many designated targets, and the Abstract Asymptotic Speedup Principle (Theorem 7.17) is stated under a hypothesis rather than exhibited. Here the hypothesis is a theorem.*

Remark 10.23. This refines the computationally effective and computable SIC apparatus of Section 9 by tying the resource bound to the shortest-refutation size $s_F(\Gamma)$ of Definition 10.4, rather than merely requiring that some halting computation exist.

C05: Two-system size speedup for proof-producing realizations

Authoritative replacement for: Theorem 10.19, printed p. 42.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Theorem 10.19 (Corrected conclusion). Under hypotheses 1–3 of the submitted theorem, the size-relativized canonical target-search SIC for F' is asymptotically faster on $(\Gamma_n, \perp)$ than every explicit F -proof-producing realization whose running time is at least the size of the proof it outputs.

Qualification. A lower bound on F -proof size does not lower-bound arbitrary decision procedures that need not output an F -proof. The theorem is a proof-complexity/output-size comparison, not a universal algorithmic lower bound.

C41: Two-system theorem is a proof-size separation

Authoritative replacement for: Theorem 10.19, Correction C05, Corollary 10.21, and analogous later uses.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

Let P, P' be Cook–Reckhow systems on a common unsatisfiable family (Γ_n) and suppose

$$s_{P'}(\Gamma_n) \leq p(n), \quad s_P(\Gamma_n) \geq q(n), \quad \frac{q(n)}{p(n)} \longrightarrow \infty.$$

Then

$$\frac{s_P(\Gamma_n)}{s_{P'}(\Gamma_n)} \longrightarrow \infty$$

whenever the denominator is nonzero, so P' has asymptotically smaller minimum certificates on the family. This is a proof-complexity separation.

A lower bound on P -proof size lower-bounds the runtime of any explicit P -proof-producing procedure by at least the size of the proof it outputs. An upper bound on $s_{P'}$, however, is only an *existence* statement about a short P' certificate and does not by itself give an algorithm that finds such a certificate quickly. Therefore an algorithmic statement that “ P' is asymptotically faster” requires an additional proof-search runtime bound for P' .

Read Corollary 10.21 and the later Tseitin comparison as unconditional *proof-size* separations unless such a search bound is separately supplied. Concrete exponential-versus-polynomial size ratios already tend to infinity, so their proof-complexity content is unchanged.

Theorem 10.24 (Tree resolution is quasi-polynomially automatizable). *Tree-like (Davis–Putnam) resolution is t -automatizable for $t(m) = m^{O(\log m)}$.*

Attribution. This is Corollary 3.4 of [1], due to [3, 1] together with the width-size relation for tree-like resolution (Theorem 10.17’s antecedent, due to Ben-Sasson and Wigderson): every size- m tree-like refutation of a formula on ℓ variables converts to one of width $O(\log m)$, of which there are only $\ell^{O(\log m)}$ many clauses to search through by breadth-first enumeration bounded to that width, giving the stated running time. $\square$

10.5 Feasible interpolation as a proof-complexity barrier theorem

Definition 10.25. Let $A(\bar{p}, \bar{r})$ and $B(\bar{p}, \bar{s})$ be formulas in which the variable blocks $\bar{r}$ and $\bar{s}$ are disjoint from each other, so that $\bar{p}$ is the block common to both, and suppose the conjunction $A(\bar{p}, \bar{r}) \wedge B(\bar{p}, \bar{s})$ is unsatisfiable. An *interpolant* for the pair (A, B) is a Boolean function I of the shared block $\bar{p}$ such that, for every truth assignment $\bar{a}$ to $\bar{p}$,

$$I(\bar{a}) = 0 \implies A(\bar{a}, \bar{r}) \text{ is unsatisfiable in } \bar{r}, \quad I(\bar{a}) = 1 \implies B(\bar{a}, \bar{s}) \text{ is unsatisfiable in } \bar{s}.$$

Since $A \wedge B$ is unsatisfiable, at least one of the two disjuncts holds at every $\bar{a}$, so an interpolant always exists; the content of the definition is entirely in how small a circuit computes one.

Definition 10.26. Let $F = (x, y, z)$ be a Cook–Reckhow system for UNSAT and let $g \in [\mathbf{N}^+ \rightarrow \mathbf{N}^+]$. Say F has *g -interpolation* if for every pair (A, B) as in Definition 10.25 whose conjunction has $s_F(\{A, B\}) \leq m$, there is a Boolean circuit of size at most $g(m)$ computing an interpolant for (A, B) . Say F has *feasible interpolation* if g may be taken to be a polynomial.

Theorem 10.27 (Feasible interpolation for resolution and cutting planes). *Resolution has feasible, monotone interpolation, and cutting planes has feasible, monotone interpolation with interpolants computed by circuits over the reals.*

Attribution. These are theorems of Krajíček and of Pudlák, respectively, presented in [1], Theorem 3.13 and Theorem 3.14. For resolution, the interpolant circuit is built directly from the refutation dag by propagating, at each derived clause, which side of the partition falsifies it under a given assignment to $\bar{p}$; the construction is size-preserving up to a polynomial factor. $\square$

Theorem 10.28 (Interpolation lower-bound theorem). *If a Cook–Reckhow system F has feasible interpolation and $\text{NP} \not\subseteq \text{P/poly}$, then F is not polynomially bounded.*

Attribution and sketch. This is Theorem 3.16 of [1], following Krajíček and Pudlák. If F were both polynomially bounded and feasibly interpolable, then for the standard family of clique-versus-coloring formulas, which are unsatisfiable exactly when a graph has both a k -clique and a proper $(k-1)$ -coloring witnessed disjointly, the resulting polynomial-size interpolant circuits would decide graph satisfiability instances in P/poly , contradicting $\text{NP} \not\subseteq \text{P/poly}$ once the encoding is chosen so that the interpolant separates an NP-hard promise problem. $\square$

Corollary 10.29. *Since cutting planes has feasible monotone interpolation (Theorem 10.27), the clique-coloring formulas of Theorem 10.28 require exponential-size cutting planes refutations, matching [1], Theorem 3.18.*

Remark 10.30. Theorem 10.28 is structurally a proof-complexity analogue of two barrier theorems already in this paper. The No-Free-Recoding-Speedup Theorem (Theorem 4.34) says that one form

C06 and C33: Interpolation lower bounds and current random 3-CNF status

Authoritative replacement for: Corollary 10.29 and Remark 10.33, printed p. 43.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replacement for Corollary 10.29. Do not derive an exponential cutting-planes lower bound directly from Theorem 10.28. The appropriate separate attributed result is: the monotone feasible-interpolation construction for cutting planes, combined with the applicable exponential lower bounds for monotone real circuits separating clique from coloring, yields the established exponential cutting-planes lower bounds for the corresponding clique-coloring contradictions. This uses the circuit lower bound in addition to feasible interpolation.

Replacement for Remark 10.33. For unrestricted Frege systems, superpolynomial lower bounds for random linear-density 3-CNFs remain open. For every fixed depth k , Gryaznov and Talebanfar proved that there is an $\varepsilon_k > 0$ such that a random n -variable 3-CNF with $\Theta(n)$ clauses requires $\Omega(n^{1+\varepsilon_k})$ steps in depth- k Frege with high probability. This is superlinear, not superpolynomial, progress.

of structural strength – pure invertible recoding – cannot by itself buy transaction-count speedup, while the Tarski–Gödel barrier (Theorem 4.26) says that a SIC’s very expressiveness forecloses a complete internal truth predicate. Theorem 10.28 says, in the same spirit, that a proof system’s very expressiveness – specifically, its ability to interpolate feasibly – forecloses polynomial boundedness under a standard complexity hypothesis. In each case a capability that looks like strength (recoding power, self-descriptive richness, interpolation) is shown to force a limitation elsewhere in the system.

Remark 10.31 (Limitations). Under the same kind of hardness assumption used positively above, sufficiently strong proof systems provably *lack* feasible interpolation, so the technique of Theorem 10.28 cannot reach them. If one-way functions exist, Frege systems do not have feasible interpolation [1], Theorem 3.19, by an argument encoding a secret-key-agreement protocol into the interpolation problem; if factoring Blum integers is hard, no proof system polynomially simulating AC^0 -Frege has feasible interpolation [1], Theorem 3.20. These limitation theorems are recorded here without formal-system translation, since they concern proof systems whose formalization as $F = (x, y, z)$ is identical in kind to Definition 10.14 and adds no further structure.

10.6 An open problem, restated

Conjecture 10.32. Let $(\Gamma_n)_{n \geq 1}$ be a family of random 3-CNF formulas at constant density above the satisfiability threshold, in the sense of [1], Section 2.3.1. There is no Frege system F and polynomial p such that $s_F(\Gamma_n) \leq p(n)$ for almost all n . Equivalently, no size-relativized canonical breadth-first target-search SIC $B_{F,Y,\perp}^{\text{size}}$ over a Frege system F reaches $\perp$ at polynomial stage, almost surely, on random 3-CNF inputs.

Remark 10.33. This is the SIC-vocabulary form of the leading open problem of [1], Lecture 5, where it is left open exactly as here: the corresponding fact is known for resolution (an instance of Theorem 10.17’s method, applied to random formulas via sub-critical expansion) but is open for Frege systems, and even for boundeddepth Frege systems only the restriction method of [1], Lecture 4, is available, which so far has not yielded a matching lower bound. Consistent with this paper’s practice of naming conjectures rather than asserting unproved claims (compare the three conjectures of Version 42, two of which were later refuted or proved in Versions 43–44), Conjecture 10.32 is recorded here as open, not resolved.

10.7 Algebraic Proof Systems: Nullstellensatz, Polynomial Calculus, and Gröbner Bases

The proof systems of the preceding subsections manipulate clauses. This subsection records a second family, which manipulates polynomials, and locates it in the same vocabulary. The motivation is concrete: a satisfiability algorithm based on Gröbner basis computation looks, on first inspection, as though it might run in polynomial time, and the apparatus assembled above is exactly what is needed to say why it does not — and, more usefully, what it does buy.

Arithmetizing a formula

Definition 10.34 (Boolean arithmetization). Let φ be a propositional formula over variables $p_1, \dots, p_n$. Define a polynomial $\tau(\varphi) \in \mathbf{F}_2[x_1, \dots, x_n]$ by recursion on the structure of φ :

$$\begin{aligned} \tau(p_i) &:= x_i, & \tau(\neg\varphi_1) &:= 1 + \tau(\varphi_1), \\ \tau(\varphi_1 \wedge \varphi_2) &:= \tau(\varphi_1)\tau(\varphi_2), & \tau(\varphi_1 \vee \varphi_2) &:= 1 + (1 + \tau(\varphi_1))(1 + \tau(\varphi_2)), \end{aligned}$$

all arithmetic being performed in $\mathbf{F}_2$. Write

$$J(\varphi) := \langle \tau(\varphi) + 1, x_1^2 + x_1, \dots, x_n^2 + x_n \rangle \subseteq \mathbf{F}_2[x_1, \dots, x_n]$$

for the associated ideal. The generators $x_i^2 + x_i$ are the *field equations*; over $\mathbf{F}_2$ they coincide with $x_i^2 - x_i$ and force every coordinate of a common zero to be 0 or 1.

Proposition 10.35 (Correctness of the arithmetization). *For every truth assignment $a \in \{0, 1\}^n$, identifying true with 1 and false with 0,*

$$\tau(\varphi)(a) = 1 \iff a \models \varphi.$$

Consequently φ is satisfiable if and only if the variety of $J(\varphi)$ over $\mathbf{F}_2$ is nonempty, and, since $J(\varphi)$ contains the field equations and is therefore radical and zero-dimensional,

$$\varphi \text{ is unsatisfiable} \iff 1 \in J(\varphi) \iff \text{the reduced Gröbner basis of } J(\varphi) \text{ is } \{1\}.$$

Proof. The equivalence $\tau(\varphi)(a) = 1 \iff a \models \varphi$ is an induction on φ . For a variable it is the identification of truth values with elements of $\mathbf{F}_2$. For negation, $1 + t = 1$ iff $t = 0$ in $\mathbf{F}_2$. For conjunction, a product of two elements of $\{0, 1\} \subseteq \mathbf{F}_2$ is 1 iff both factors are 1. For disjunction, $1 + (1 + t_1)(1 + t_2) = 1$ iff $(1 + t_1)(1 + t_2) = 0$ iff at least one t_i equals 1.

The common zeros of $J(\varphi)$ in $\overline{\mathbf{F}_2}^n$ are exactly the points of $\{0, 1\}^n$ at which $\tau(\varphi) + 1$ vanishes, that is, the assignments satisfying φ . The last equivalence is the weak Nullstellensatz together with the observation that an ideal containing all field equations has a finite variety and is radical, so it is the unit ideal precisely when that variety is empty; and 1 lies in an ideal precisely when the reduced Gröbner basis with respect to any monomial order is $\{1\}$. $\square$

Remark 10.36. Proposition 10.35 converts satisfiability into an ideal membership question and is the basis of every algebraic SAT procedure. It is a translation, not an algorithm: it says where the answer lives, not what it costs to extract. Working directly in the Boolean ring $\mathbf{F}_2[x]/\langle x_i^2 - x_i \rangle$, rather than carrying the field equations as explicit generators, is the setting of Boolean Gröbner basis theory [19], where the coefficient ring may itself be a Boolean ring rather than a field; the complexity statements below are unaffected by that choice of presentation. The whole content of the subsections below is the cost.

Two algebraic formal systems

Definition 10.37 (Polynomial Calculus). Fix a field K and variables $x_1, \dots, x_n$. Let x be an alphabet sufficient to write polynomials, let

$$y := K[x_1, \dots, x_n]/\langle x_i^2 - x_i \rangle$$

be the multilinear polynomials, with $\perp := 1$ the distinguished formula, and let z consist of the finitary rules

$$\text{Lin}_{\alpha, \beta}(f, g) := \alpha f + \beta g \quad (\alpha, \beta \in K), \quad \text{Mult}_i(f) := x_i f \quad (1 \leq i \leq n).$$

Write $F_{\text{PC}} = (x, y, z)$ for the resulting formal system, the *Polynomial Calculus* over K . Its *size* function is the number of monomials, and the *degree* of a refutation is the maximum total degree of a polynomial occurring in it. The *Nullstellensatz* system F_{NS} is the static variant in which a refutation of $\{f_1, \dots, f_m\}$ is a single tuple $(g_1, \dots, g_m)$ with $\sum_j g_j f_j = 1$ identically.

Proposition 10.38. F_{PC} is polynomial-time checkable in the sense of Definition 10.3, and, restricted to inputs $J(\varphi)$ arising from Definition 10.34, it is a Cook–Reckhow system for UNSAT in the sense of Definition 10.4.

Proof. Verifying an application of $\text{Lin}_{\alpha,\beta}$ or Mult_i is a polynomial arithmetic operation on the monomial lists of the premises and conclusion, so it is performed in time polynomial in their sizes. Soundness holds because both rules preserve the property of vanishing on every common zero of the input polynomials; hence deriving 1 certifies that no common zero exists, which by Proposition 10.35 is unsatisfiability of φ . Completeness is the weak Nullstellensatz in the same proposition: if φ is unsatisfiable then $1 \in J(\varphi)$, and any representation of 1 as a combination of the generators is convertible into a derivation of 1 by the two rules. Since $\perp = 1$, refutability and derivation of $\perp$ coincide. $\square$

Remark 10.39 (Gröbner basis computation is proof search in F_{PC}). Buchberger’s algorithm, and its descendants F_4 and F_5 , compute a Gröbner basis of $J(\varphi)$ by repeatedly forming S -polynomials and reducing them — operations that are exactly compositions of $\text{Lin}_{\alpha,\beta}$ and Mult_i . A run of the algorithm that outputs $\{1\}$ therefore *is* a refutation in F_{PC} , and the canonical target-search SIC $E_{F_{\text{PC}}, Y, \perp}$ of Definition 2.19 is the associated proof-search machine. This is the observation that makes the proof-complexity lower bounds below apply to Gröbner basis algorithms rather than merely to an abstract calculus, and it is why the founding paper of the subject is titled for the algorithm rather than for the proof system [7].

The negative direction

Theorem 10.40 (Degree and size lower bounds for Polynomial Calculus). *Let (Γ_n) be the pigeon-hole family of Definition 10.16, arithmetized as in Definition 10.34. Then every FPC refutation of Γ_n has degree $\Omega(n)$, and there is a constant $c > 0$ with*

$$s_{F_{PC}}(\Gamma_n) \geq 2^{cn}$$

for all n . The same holds for random 3-CNF formulas at constant density above the satisfiability threshold, almost surely.

Attribution. The degree lower bound for the pigeonhole principle is Razborov’s [17]; the size lower bound follows from it by the size–degree tradeoff of Impagliazzo, Pudlák and Sgall [12], who state the consequence for the Gröbner basis algorithm explicitly. The statement for random 3-CNF is Ben-Sasson and Impagliazzo [6]. Note that Conjecture 10.32 is *not* thereby settled: it concerns Frege systems, which are not p -simulated by F_{PC} . $\square$

Corollary 10.41 (No polynomial-time SAT algorithm of Gröbner type). *There is no algorithm which, on input a propositional formula φ , decides satisfiability by computing a Gröbner basis of $J(\varphi)$ and testing whether it equals $\{1\}$, and which runs in time polynomial in $|\varphi|$. This holds unconditionally: no complexity-theoretic hypothesis is assumed.*

Proof. By Remark 10.39 such an algorithm’s run on an unsatisfiable input is an F_{PC} refutation, and the algorithm’s running time is at least the size of the object it writes down. By Theorem 10.40 that size is $2^{\Omega(n)}$ on the pigeonhole family, whose encodings have size polynomial in n . Hence the running time is exponential in the input size on that family. $\square$

C07: Proof-producing Gröbner lower bound

Authoritative replacement for: Corollary 10.41, printed p. 46.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Corollary 10.41 (Corrected). Any procedure whose unsatisfiable run on $J(\varphi)$ explicitly outputs a Polynomial Calculus refutation, or outputs a standard Gröbner derivation efficiently translatable to such a refutation, requires $2^{\Omega(n)}$ output time on the pigeonhole family. This is exponential in the family parameter n and superpolynomial in the encoding length.

This conclusion does *not* rule out every conceivable SAT decision algorithm that uses Gröbner bases as a subroutine. The proof-complexity lower bound applies to explicit proof-producing or efficiently proof-extractable runs.

Remark 10.42 (Where the polynomial-time intuition goes wrong). Two facts about Gröbner basis computation are easily run together, and the distinction between them is what Corollary 10.41 turns on.

First, the FGLM algorithm of Faugère, Gianni, Lazard and Mora [8] is a *change of monomial order*: it takes a Gröbner basis already computed with respect to one order and returns one with respect to another. It cannot produce the first basis, and the first basis is the expensive object. Invoking FGLM as the step that computes G presupposes the very computation whose cost is at issue.

Second, FGLM runs in time $O(nD^3)$ where $D = \dim_K(K[x_1, \dots, x_n]/J)$ is the dimension of the quotient algebra — for a zero-dimensional radical ideal, the number of points of the variety. For $J(\varphi)$ that number is the number of satisfying assignments of φ , which is at most 2^n and is frequently that large. Polynomial in D is not polynomial in n ; the field equations alone already give $\dim_K K[x]/\langle x_i^2 - x_i \rangle = 2^n$.

Neither observation is a defect of FGLM, which does what it claims. The error is in reading “polynomial-time” as a statement about the input formula when it is a statement about the quotient dimension.

The positive direction

The lower bounds close one route and leave another open, and it is worth being precise about what survives, because a genuine and unconditional speedup lives here.

Theorem 10.43 (Polynomial Calculus p -simulates resolution). $F_{\text{res}} \preceq_p F_{\text{PC}}$ over every field K , in the sense of Definition 10.8. Moreover the translation carries a resolution refutation of width w to a Polynomial Calculus refutation of degree w .

Attribution and sketch. A clause $C = \bigvee_{i \in P} p_i \vee \bigvee_{j \in N} \neg p_j$ is translated to the monomial $\prod_{i \in P} (1 - x_i) \prod_{j \in N} x_j$, which vanishes exactly on the assignments satisfying C . A resolution step on pivot p is then simulated by a bounded number of applications of Mult_i and $\text{Lin}_{\alpha, \beta}$, so the translation is computable in polynomial time and the degree of the image is the width of the source. The correspondence between width and degree is due to Clegg, Edmonds and Impagliazzo [7]. $\square$

Theorem 10.44 (Tseitin formulas separate the two systems). Let (T_n) be the Tseitin family: for a bounded-degree expander graph $G_n = (V, E)$ on n vertices with an odd total charge $\sigma : V \rightarrow \mathbf{F}_2$, the formula T_n is the CNF encoding of the parity constraints $\bigoplus_{e \ni v} x_e = \sigma(v)$ for $v \in V$. Each T_n is unsatisfiable. Then

$$s_{F_{\text{PC}}}(T_n) \leq p(n) \quad \text{over } K = \mathbf{F}_2 \quad \text{while} \quad s_{F_{\text{res}}}(T_n) \geq 2^{\Omega(n)}$$

for some polynomial p .

Attribution and sketch. Over $\mathbf{F}_2$ the parity constraint at v is the linear polynomial $\sum_{e \ni v} x_e + \sigma(v)$, and summing these over all $v \in V$ makes every edge variable appear twice, hence cancel, leaving $\sum_v \sigma(v) = 1$: a derivation of $\perp$ using $|V|$ applications of $\text{Lin}_{1,1}$ and no multiplications at all. The bounded degree of G_n keeps the translation from the CNF encoding to these linear polynomials of polynomial size. The exponential resolution lower bound for Tseitin formulas over expanders is Urquhart’s [27]. $\square$

Corollary 10.45 (A second unconditional instance of abstract speedup). Instantiating the Two-System Size-Speedup Principle (Theorem 10.19) with $F = F_{\text{res}}$, $F' = F_{\text{PC}}$ over $\mathbf{F}_2$, and $(\Gamma_n) = (T_n)$

the Tseitin family, the hypotheses are met with $p(n)$ polynomial and $q(n) = 2^{\Omega(n)}$ by Theorem 10.44. Hence the size-relativized canonical target-search SIC $B_{F_{PC}, Y, \perp}^{\text{size}}$ is asymptotically faster on the Tseitin family than every proof-search realization in F_{res} .

Remark 10.46 (The two unconditional instances point in opposite directions). Corollary 10.21 exhibited extended Frege beating resolution on the pigeonhole family. Corollary 10.45 exhibits Polynomial Calculus beating resolution on the Tseitin family. Theorem 10.40 shows that Polynomial Calculus does *not* beat resolution on the pigeonhole family — both are exponential there. So the two systems are incomparable in strength on these families, and neither dominates:

$$F_{\text{res}} \preceq_P F_{PC}, \quad F_{PC} \not\preceq_P F_{\text{res}} \text{ (by Theorem 10.44 with Proposition 10.10),}$$

while F_{PC} and F_{EF} are separated by Theorem 10.40 against Theorem 10.18. The algebraic route is therefore a genuine gain over clause-based search on an identifiable family, and a genuine loss on another. That is the honest form of the claim that Gröbner bases help with satisfiability: they help, demonstrably and unconditionally, on parity-structured instances, and provably not in general.

Theorem 10.47 (Bounded-degree refutations are findable). *Fix d . There is an algorithm which, given an unsatisfiable arithmetized input on n variables possessing an F_{PC} refutation of degree at most d , finds one in time $n^{O(d)}$. Consequently the restriction of F_{PC} to degree- d refutations is t -automatizable in the sense of Definition 10.22 for $t(m) = m^{O(d)}$.*

Sketch. The multilinear polynomials of degree at most d in n variables form a K -vector space of dimension $\sum_{j \leq d} \binom{n}{j} = n^{O(d)}$. Closing the input under Mult_i and taking linear spans, while discarding everything of degree exceeding d , is therefore a linear-algebra computation in a space of that dimension, and $\perp = 1$ is derivable in degree d if and only if it lies in the resulting span. Gaussian elimination on a matrix of side $n^{O(d)}$ gives the stated bound. The method is that of Clegg, Edmonds and Impagliazzo [7]. $\square$

Remark 10.48 (What is actually salvaged). Three positive statements survive the lower bounds, and they are what an algebraic SAT effort should be organized around.

1. *Strict gain on structured instances.* Theorem 10.44 and Corollary 10.45: on parity-structured formulas the algebraic system is exponentially better than resolution, unconditionally. Instances carrying linear-algebraic structure over a finite field are exactly where the method earns its keep.
2. *Tractability under a degree bound.* Theorem 10.47: the obstruction is degree, not the algebraic setting as such. Where a family admits low-degree refutations, they are found in polynomial time for fixed degree. The research question that replaces “is SAT in P?” is the answerable one: which families admit low-degree refutations?
3. *Simulation without loss.* Theorem 10.43: nothing is given up by moving to the algebraic setting, since resolution is p -simulated. The algebraic system is at least as strong on every family and strictly stronger on some.

What does not survive is the unrestricted claim. By Corollary 10.41 no Gröbner-basis satisfiability procedure runs in polynomial time on all inputs, and this is settled rather than open.

A Collection of Examples of Objects Discussed Herein

C08 and C09: Correct comparison of resolution and Polynomial Calculus

Authoritative replacement for: Remarks 10.46 and 10.48, printed p. 48.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replacement for the central paragraph of Remark 10.46. Polynomial Calculus p-simulates resolution and is strictly stronger on the Tseitin family: a reverse p-simulation would contradict the exponential separation. On the pigeonhole family, both systems require exponential-size refutations, so Polynomial Calculus gives no advantage there. This is shared hardness, not a loss, and it does not make the two systems incomparable.

Replacement for the final paragraph of Remark 10.48. What is ruled out here is uniform polynomial performance for explicit Polynomial Calculus or standard Gröbner derivations on all inputs. The argument does not establish a lower bound for every possible decision algorithm that uses Gröbner bases as a subroutine.

Table 1: A Collection of Examples of Objects Discussed Herein: SICs

Name of object	Object type	Section first found	Role
$E_{F,Y}$	SIC; enumerating ATP	Section 2	Canonical brute-force benchmark
$E_{F,Y,\varphi}$	SIC; target-search ATP	Section 2	Canonical target benchmark
$B_{F,Y}$	SIC; breadth-first ATP	Section 2	Shortest-proof horizon machine
$B_{F,Y,\varphi}$	SIC; breadth-first target-search ATP	Section 2	Target horizon machine
M	SIC	Section 2	Generic rule-driven prover
M_b	SIC induced by a branch	Section 5	Deterministic shadow of a branch
M^t	Recoded SIC	Section 4	Isomorphic translated copy
U_{all}	SIC	Section 9	Static identity benchmark; not universal
M_P	SIC; target-search ATP	Section 4	Minimal Picard self-certifier

Table 2: A Collection of Examples of Objects Discussed Herein: Formal systems and rule extensions

Name of object	Object type	Section first found	Role
$F = (x, y, z)$	Formal system	Section 1	Ambient deductive system
F^D	Formal system with derived rules	Section 7	Macro-augmented conservative extension
F^{ns}	Nonstandard formal extension	Opening section	Ambient system for infinitesimal-style proofs
F^t	Recoded formal system	Section 4	Isomorphic translated formal system
Weak successor-set theory	Formal system family	Section 7	Minimal set background for $\text{Succ}(\{0\}) = \{0, 1\}$

Table 3: A Collection of Examples of Objects Discussed Herein: Simulations and search policies

Name of object	Object type	Section first found	Role
a_M	State-abstraction map	Section 3	Consequence-closure abstraction into $E_{F,Y}$
$a_{M,\varphi}$	State-abstraction map	Section 3	Target-search closure abstraction
$S_M \times S_N$	Weak simulation witness	Section 3	Witness for universal weak simulation
id_{S_M}	Synchronous simulation map	Section 3	Reflexive strong simulation
(f, ρ)	Clocked simulation pair	Section 3	Principal time-rescaled simulation
$(g \circ f, \sigma \circ \rho)$	Clocked simulation pair	Section 3	Transitivity construction
f_K	Noncomputable synchronous witness	Section 3	Shows effectivity is not automatic
Σ	Nondeterministic proof-search policy	Section 5	Branch generator / heuristic
Depth- K pair	Finite-horizon clocked simulation	Section 3	Limited comparison notion
Proof interpretation / extraction map	Translation or abstraction device	Opening section	Standard shadow of a nonstandard proof
f_t	Synchronous simulation map	Section 4	Strong witness for $M \sim M^t$
t	Uniform recoding / translator	Section 4	Invertible coding isomorphism

Table 4: A Collection of Examples of Objects Discussed Herein: Proof measures and closure operators

Name of object	Object type	Section first found	Role
$\text{Con}_F(\Gamma)$	Consequence operator value	Section 1	Least inference-closed extension
D_F	One-step closure operator	Section 1	Direct-consequence expansion
$\ \varphi\ _F(\Gamma)$	Proof-length measure	Section 2	Length of a shortest proof
$L_F(\Gamma, m)$	Proof layer	Section 2	Exact shortest-proof stratum
$\tau_M(\Gamma, \varphi)$	Search-cost measure	Section 7	Target transaction count
$P_F(\Gamma, \varphi, n)$	Counting function	Section 8	Number of length- n proofs
$P_F^{\min}(\Gamma, \varphi)$	Counting function	Section 8	Number of shortest proofs
Extracted bound or witness	Computational content	Opening section	Standard data mined from a proof
θ from Theorem 4.25	Fixed-point sentence	Section 4	Sentence about its recoded code
$\ \varphi\ _F^{\text{size}}(\Gamma)$	Size-relativized proof measure	Section 10	Shortest-refutation size
$s_F(\Gamma)$	Proof-complexity function	Section 10	Size of a shortest refutation of $\perp$

Table 5: A Collection of Examples of Objects Discussed Herein: Propositional proof complexity

Name of object	Object type	Section first found	Role
F_{res}	Formal system	Section 10	Resolution as an $F = (x, y, z)$ instance
F_{EF}	Formal system	Section 10	Extended Frege system
Γ_n	Finite unsatisfiable input family	Section 10	Pigeonhole principle PHP_n^{n-1}
$\preceq_p$	Preorder on formal systems	Section 10	p -simulation
$B_{F,Y,\varphi}^{\text{size}}$	SIC; size-relativized breadth-first ATP	Section 10	Size-tracking target benchmark
I	Interpolant	Section 10	Boolean function separating $\bar{p}$ -side from $\bar{q}$ -side unsatisfiability

11 Conclusion

Part I separates four mathematically different comparison notions. Weak simulation is the original unconstrained relational shadowing condition, and Theorem 3.2 shows that it collapses universally. Synchronous deterministic simulation is a faithful injective stage-for-stage embedding. State abstraction may preserve an evolution equation while identifying states or ignoring halting. Clocked deterministic simulation is the principal relation: it retains faithful state and halting preservation but allows a nondecreasing unbounded clock to rescale time and represent idle or extra stages.

The distinction is necessary. Proposition 3.11 shows that synchronous simulation can fail for cardinality reasons even when two machines obey dynamics of the same form. Theorem 3.13 shows that a one-stage delay can make two matching machines synchronously incomparable. Theorem 3.23 then shows that clocked simulation identifies them. Thus synchronous simulation remains a useful strongest relation, but clocked simulation is the better general notion for comparison, equivalence, and depth.

The clocked relation is reflexive and transitive and induces a partial order on its equivalence classes. It does not collapse: the identity SIC and the canonical rule-enumeration SIC still fail to be universal because clock changes cannot repair incompatible state evolution or contradictory halting bits. Canonical consequence-closure maps remain useful abstractions, but their possible many-to-one character keeps them outside the faithful simulation relations.

The notation $[S_M \rightarrow S_N]$ includes noncomputable functions. Theorem 3.27 exhibits a noncomputable synchronous witness, so effectivity must be imposed rather than inferred. Effective clocked simulation does this for SICs with effective state presentations. On infinite formula domains, the full power-set state space is generally too large to be coded by natural numbers, so executable applications should explicitly select finite, reachable, or otherwise effectively represented state domains.

The Proof-Horizon Theorem, Branch-Covering Theorem, Conservative Compilation Theorem, theorem-graph constructions, and proof-mining discussion remain independent of weak-simulation collapse. Invertible recodings satisfy the strongest simulation notion: the map $f_t(p) = t[p]$ is a synchronous isomorphism and hence also a clocked equivalence. Transaction counts are preserved pointwise, which now yields the No-Free-Recoding-Speedup Theorem rather than merely a conjecture.

Three conjectures left open in Version 42 of the source paper are closed here. The Self-Audit Compression Conjecture holds: Theorem 4.28 shows that once the transport theorem is proved uniformly, verifying that a recoded copy is a SIC or ATP costs only a polynomial-length proof in the code lengths of the machine and the recoding. The Normalized Depth Spectrum Conjecture fails: Theorem 4.30 exhibits clocked-equivalent machines with different transaction spectra and spectrally identical machines that are not clocked-equivalent, even after quotienting by syntactic recodings, because transaction counts are rescaled by clocks and blind to halting. The Finite Metadata Self-Description Conjecture holds: Theorem 4.32 supplies the finite metadata as the defining axioms of a conservative definitional extension, so a deductively effective machine can certify every recoded copy of itself across a uniformly primitive recursive recoding family without any change to its underlying theorem set.

The self-reference story is sharpened as follows. A recoding is a change of coding, so the former tag vocabulary has been absorbed into the coding vocabulary, and Proposition 4.5 answers where recoded copies live: $t(\varphi)$ exists in the coding universe as a formula of F^t , and it lies inside y itself exactly in the endorecoding case. On that foundation, the Picard Protocol of Definition 4.19 makes “prove that a recoded copy of yourself is an ATP” a precise success criterion, and the two-formula machine M_P of Theorem 4.21 meets it with provably optimal parameters while its recoded copy

certifies M_P in return. Self-certification therefore does not require heavy arithmetization in order to exist; arithmetization is what internalizes the soundness that the minimal example carries by declaration.

The resulting hierarchy of notions is therefore:

synchronous deterministic simulation $\implies$ clocked deterministic simulation $\implies$ weak simulation,

with effective clocked simulation refining the middle relation when computable realization matters, and state abstraction lying alongside the hierarchy as a purposefully lossy comparison. This framework preserves a strong exact notion, adds the time flexibility required by ordinary simulation practice, and makes computability assumptions explicit.

Section 10 imports the Cook–Reckhow theory of propositional proof systems into this same vocabulary. A proof system is a polynomial-time checkable formal system with a size function refining proof length, and under that refinement the Proof-Horizon Theorem, proved once in Section 2, already computes classical proof-complexity size. Alongside weak, synchronous, clocked, and effective SIC-simulation, p -simulation (Definition 10.8) supplies a second and genuinely different preorder, comparing formal systems only through proof-to-proof translations rather than through witnesses on state spaces, and Theorem 10.13 shows that this preorder collapses all Frege systems to a single equivalence class exactly as recoding (Theorem 4.12) collapses a SIC and its translated copy. The pigeonhole principle, formalized via Haken’s exponential resolution lower bound and the polynomial extended-Frege upper bound, gives the paper’s first unconditional instance of the Abstract Asymptotic Speedup Principle, generalized in Theorem 10.19 from a formal system and its own conservative extension to two independent formal systems. Automatizability refines the computationally effective SIC apparatus of Section 9 by tying runtime to shortest-refutation size, and feasible interpolation supplies a proof-complexity lower-bound technique that, on inspection, has the same shape as the paper’s own barrier theorems: the No-Free-Recoding-Speedup Theorem and the Tarski–Gödel barrier each show that a form of structural strength forecloses a capability elsewhere in the system, and the interpolation lower-bound theorem does the same for polynomial boundedness. One question is recorded as open rather than resolved, in keeping with this paper’s practice of naming rather than begging conjectures: whether random 3-CNF formulas are hard for Frege systems (Conjecture 10.32).

Consequential revision: Qualified Part I conclusions

Authoritative replacement for: Conclusion, printed pp. 52–53.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

The conclusion must read the Self-audit Compression result and the Finite Metadata result with the additional hypotheses stated in the corrected Theorems 4.28 and 4.32. In particular, polynomial self-audit compression is conditional on an explicit polynomial output-code bound and polynomial proof-calculus overhead, and enumeration of formulas in the expanded language requires a prover deductively effective for that expanded theory.

The proof-complexity conclusion concerning Gröbner methods is likewise restricted to explicit Polynomial Calculus or standard Gröbner derivations whose output size lower-bounds runtime. No universal lower bound for every Gröbner-assisted SAT decision procedure is claimed.

Version 46.1 Supplement: Induction, Closure, and ATP Invariants

1. Scope and the four meanings of induction

The word *induction* can refer to different mathematical structures. Conflating them obscures which hypotheses are required. In the present framework there are four relevant forms.

- I1. Structural induction on formulas.** This follows the construction tree of a WFF. It requires the WFF domain to be the least set generated from atomic formulas by specified formula constructors.
- I2. Induction on proofs or inference closure.** This follows proof lines or rule applications. It applies to the consequence set generated from hypotheses by finitary rules.
- I3. Induction on ATP stages or birth times.** This follows a monotone run $C(\Gamma, 1) \subseteq C(\Gamma, 2) \subseteq \dots$. When the ATP is rule-driven, formula invariants can be propagated from one stage to the next.
- I4. Induction on the reflection index.** This follows the recursively defined sequence $\theta_0, \theta_1, \dots$. It is ordinary induction at finite levels and transfinite induction only after separate successor and limit clauses have been supplied.

The original *xyz* definition takes y to be a set of finite strings. That is sufficient for proof induction, because proofs are finite sequences governed by the rules in z . It is not sufficient for structural induction on WFFs: an arbitrary set of strings need not be the least set generated by a grammar. The next definition records the additional hypothesis exactly.

2. Structural induction on well-formed formulas

Definition 1 (Finitary formula grammar). Let x be an alphabet. A finitary formula grammar is a pair $(A, \mathcal{G})$ in which A is a set of atomic formulas and every $g \in \mathcal{G}$ is a partial k -ary constructor on finite strings for some $k \in \mathbb{N}$. Define the grammar operator

$$\Phi_{A, \mathcal{G}}(S) := A \cup \{g(\varphi_1, \dots, \varphi_k) : g \in \mathcal{G}, (\varphi_1, \dots, \varphi_k) \in S^k, \text{ and } g(\varphi_1, \dots, \varphi_k) \text{ is defined}\}.$$

A formal system $F = (x, y, z)$ is *grammatically generated* by $(A, \mathcal{G})$ if y is the least set S satisfying $\Phi_{A, \mathcal{G}}(S) \subseteq S$.

Theorem 2 (Structural induction on WFFs). *Suppose $F = (x, y, z)$ is grammatically generated by $(A, \mathcal{G})$. Let P be a property of formulas. Assume:*

- (i) $P(a)$ holds for every $a \in A$;
- (ii) for every k -ary $g \in \mathcal{G}$, whenever $P(\varphi_i)$ holds for all $1 \leq i \leq k$ and $g(\varphi_1, \dots, \varphi_k)$ is defined, then $P(g(\varphi_1, \dots, \varphi_k))$ holds.

Then $P(\varphi)$ holds for every $\varphi \in y$.

Proof. Let $W := \{\varphi \in y : P(\varphi)\}$. Assumption (i) gives $A \subseteq W$, and assumption (ii) says that W is closed under every constructor in $\mathcal{G}$. Hence $\Phi_{A, \mathcal{G}}(W) \subseteq W$. By the leastness of y , one has $y \subseteq W$. Since $W \subseteq y$ by definition, $W = y$. $\square$

Remark 3 (Why this is not automatic). The conclusion would be unjustified if y contained designated exceptional strings not generated from A by $\mathcal{G}$. In that case the base and constructor clauses could hold while P failed on an exceptional WFF. Thus structural induction belongs in the framework only after grammatical leastness is made explicit.

Example 4. For a first-order language, A may contain atomic equations and relation instances, while $\mathcal{G}$ contains the constructors $\neg$, binary connectives, and quantification. Structural induction then proves properties such as uniqueness of parsing, finiteness of the set of subformulas, recursive computability of syntactic rank, and substitution lemmas.

3. Inference closure and induction on proofs

Fix a formal system $F = (x, y, z)$ in the original sense and a hypothesis set $\Gamma \subseteq y$. A set $W \subseteq y$ is inference-closed when every defined rule instance whose premises lie in W has its conclusion in W .

Theorem 5 (Rule induction). *Let P be a property of WFFs. Suppose:*

- (i) $P(\gamma)$ holds for every $\gamma \in \Gamma$;
- (ii) every rule of F preserves P : whenever $r(\alpha_1, \dots, \alpha_k) = \beta$ and $P(\alpha_i)$ holds for every premise, then $P(\beta)$ holds.

Then

$$\Gamma \vdash_F \varphi \implies P(\varphi).$$

Equivalently, $\text{Con}_F(\Gamma) \subseteq \{\varphi \in y : P(\varphi)\}$.

Proof. The set $W := \{\varphi \in y : P(\varphi)\}$ contains Γ by (i) and is inference-closed by (ii). The least-inference-closure theorem therefore gives $\text{Con}_F(\Gamma) \subseteq W$. Equivalently, one may induct directly on the line number of a finite proof. $\square$

Corollary 6 (Invariant method). *If all hypotheses satisfy P , every inference rule preserves P , and the target L fails P , then $\Gamma \not\vdash_F L$.*

This is the general form of the familiar invariant argument: a preserved parity, residue class, syntactic shape, semantic valuation, or other property can prove unprovability. A slowly changing property gives a quantitative lower bound rather than an impossibility result.

4. Induction on ATP runs

Let $M = (C, H)$ be a SIC or ATP over $Y \subseteq y$, with stages indexed by $\mathbb{N}_+$ as in the original paper. Recall that M is rule-driven when each formula newly entering $C(\Gamma, m+1)$ is a direct consequence of $C(\Gamma, m)$.

Theorem 7 (ATP formula-invariant induction). *Let M be a rule-driven SIC over Y , fix $\Gamma \subseteq Y$, and let P be a property of formulas. Assume:*

- (i) every $\gamma \in \Gamma$ satisfies P ;
- (ii) whenever a rule instance has all premises satisfying P , its conclusion satisfies P .

Then every formula materialized at every stage satisfies P :

$$C(\Gamma, m) \subseteq \{\varphi \in Y : P(\varphi)\} \quad (m \in \mathbb{N}_+).$$

Proof. Induct on m . At stage 1, $C(\Gamma, 1) = \Gamma$, so the claim follows from (i). Assume it holds at stage m and let $\beta \in C(\Gamma, m+1)$. If $\beta \in C(\Gamma, m)$, use the induction hypothesis. Otherwise, rule-drivenness makes β a direct consequence of premises in $C(\Gamma, m)$; those premises satisfy P by the induction hypothesis, so (ii) gives $P(\beta)$. $\square$

Corollary 8 (Soundness of a rule-driven run). *If M is rule-driven, then*

$$C(\Gamma, m) \subseteq \text{Con}_F(\Gamma) \cap Y$$

for every m . This recovers the soundness proposition of Part I as an instance of ATP invariant induction, with $P(\varphi)$ defined by $\Gamma \vdash_F \varphi$. The inference-closedness of Y is not needed for this inclusion; it is needed when an exhaustive closure process is required to remain inside Y .

Definition 9 (Birth time). For a monotone run, define

$$\text{b}_{M,\Gamma}(\varphi) := \min\{m \in \mathbb{N}_+ : \varphi \in C(\Gamma, m)\},$$

with value ∞ if φ never appears.

Theorem 10 (Birth-time induction). *Let M be rule-driven. Suppose P holds for every input formula and is preserved by every inference rule. Then $P(\varphi)$ holds for every φ with finite birth time.*

Proof. Use strong induction on $\text{b}_{M,\Gamma}(\varphi)$. Formulas born at stage 1 belong to Γ . If φ is born at stage $m+1$, rule-drivenness supplies a rule instance whose premises already lie in $C(\Gamma, m)$ and therefore have birth time at most m . The strong induction hypothesis gives P for the premises, and rule preservation gives $P(\varphi)$. $\square$

Theorem 11 (State-invariant induction). *Let Q be a property of subsets of Y . If $Q(C(\Gamma, 1))$ holds and*

$$Q(C(\Gamma, m)) \implies Q(C(\Gamma, m+1))$$

for every m , then $Q(C(\Gamma, m))$ holds for every stage m .

Remark 12. State induction does not require rule-drivenness, but without a structural description of the transition it may be only a bookkeeping principle. Formula-invariant and birth-time induction become mathematically informative because rule-drivenness links a new formula to earlier premises.

5. Induction and exhaustive closure

For the canonical rule-enumeration SIC $E_{F,Y}$ of Part I, define

$$C_1 := \Gamma, \quad C_{m+1} := D_F(C_m),$$

where $D_F(A)$ adjoins all direct consequences of A .

Proposition 13 (Finite proof layers). *If φ has an F -proof from Γ of length at most m , then $\varphi \in C_m$.*

Proof. Induct on the proof length. A one-line proof is a hypothesis and hence lies in C_1 . For a proof of length at most $m+1$, the final formula is either a hypothesis or a direct consequence of earlier proof lines. By induction, all required premises lie in C_m , so the final formula lies in $D_F(C_m) = C_{m+1}$. $\square$

Corollary 14 (Closure at the omega union). *If Y is inference-closed and contains Γ , then*

$$\bigcup_{m \in \mathbb{N}_+} C_m = \text{Con}_F(\Gamma) \cap Y.$$

The union is an ω -stage closure, but every individual theorem enters at a finite stage because every proof is finite. This is not transfinite proof search: ω indexes the union of all finite layers.

6. Quantitative induction: potentials and ATP birth stages

Theorem 15 (ATP potential lower bound). *Let M be a rule-driven SIC with input Γ . Let $r : Y \rightarrow \mathbb{N}$ and let $d \in \mathbb{N}_+$. Assume:*

- (i) $r(\gamma) \leq r_0$ for every $\gamma \in \Gamma$;
- (ii) every direct inference from premises $\alpha_1, \dots, \alpha_k$ to a conclusion β satisfies

$$r(\beta) \leq \max_{1 \leq i \leq k} r(\alpha_i) + d.$$

Then every formula φ with finite birth time satisfies

$$b_{M,\Gamma}(\varphi) \geq 1 + \left\lceil \frac{\max\{0, r(\varphi) - r_0\}}{d} \right\rceil.$$

Proof. We first prove by induction on m that every formula in $C(\Gamma, m)$ has rank at most $r_0 + (m-1)d$. The case $m = 1$ is (i). At stage $m + 1$, an old formula satisfies the bound by induction, while a new formula is a direct consequence of premises from stage m and therefore has rank at most $r_0 + md$ by (ii). If φ is first present at stage b , then $r(\varphi) \leq r_0 + (b-1)d$, which rearranges to the displayed inequality. $\square$

Remark 16 (Scope of the bound). The theorem bounds a logical stage only because rule-drivenness prohibits a same-stage chain of several dependent inferences. It does not automatically bound wall-clock time. A stage may materialize many formulas, and the cost of discovering or verifying a rule instance remains separate.

Proposition 17 (Preserved invariant). *If every rule satisfies $r(\beta) \leq \max_i r(\alpha_i)$, then no formula of rank greater than r_0 can ever be materialized. Equivalently, the rank sublevel set is an ATP invariant.*

7. Induction on halting and freezing

Proposition 18 (Persistent completion after halting). *If $H(\Gamma, h) = 1$, then for every $n \geq h$,*

$$H(\Gamma, n) = 1 \quad \text{and} \quad C(\Gamma, n) = C(\Gamma, h).$$

Proof. Induct on $n - h$. The base case is immediate. For the successor step, persistent halting gives $H(\Gamma, n + 1) = 1$, and the freezing axiom gives $C(\Gamma, n + 1) = C(\Gamma, n) = C(\Gamma, h)$. $\square$

This elementary induction is what turns a first halting stage into an event horizon: after it, neither the halting observation nor the materialized theorem state changes.

8. Induction and formal self-awareness

Part II defines, for a tagged ATP M , the finite reflection sequence

$$\theta_0 := \text{ATP}(\langle M \rangle), \quad \theta_{k+1} := \text{Pr}(\langle M \rangle, \langle \theta_k \rangle).$$

The sequence itself is recursively generated, so ordinary induction on k is appropriate.

Proposition 19 (Finite reflection induction). *Suppose $M \vdash \theta_0$ and M has internal necessitation:*

$$M \vdash \varphi \implies M \vdash \text{Pr}(\langle M \rangle, \langle \varphi \rangle).$$

Then $M \vdash \theta_k$ for every $k \in \mathbb{N}$, and hence $\text{lev}(M) = \omega$ under the level definition of Part II.

Proof. Induct on k . The base case is assumed. If $M \vdash \theta_k$, internal necessitation yields

$$M \vdash \text{Pr}(\langle M \rangle, \langle \theta_k \rangle) = \theta_{k+1}.$$

Thus all finite reflection formulas are proved. □

This is the induction already used by the Level- ω and Picard arguments. The ATP viewpoint adds a temporal refinement when the formulas are actually generated by a rule-driven run.

Theorem 20 (Exact reflection layers of the reflection-closure machine). *Let R be the reflection-closure machine of Part II, built from the hypothesis a , the rule*

$$\rho_0 : a \mapsto \theta_0,$$

and the unary necessitation rule

$$\nu : \varphi \mapsto \text{Pr}(\langle R \rangle, \langle \varphi \rangle).$$

On the designated input $A = \{a\}$, under the canonical proof-length state convention of that construction,

$$b_{R,A}(\theta_k) = k + 2 \quad (k \in \mathbb{N}).$$

Proof. For the upper bound, the proof

$$a, \theta_0, \theta_1, \dots, \theta_k$$

has length $k + 2$, using ρ_0 once and ν exactly k times. Hence $b_{R,A}(\theta_k) \leq k + 2$.

For the lower bound, θ_0 can enter only as the output of ρ_0 , which requires the earlier line a . For $k \geq 1$, the only rule that outputs a formula with the syntactic form of θ_k is ν , and its required premise is θ_{k-1} . Induction on k therefore forces every proof of θ_k to contain $a, \theta_0, \dots, \theta_k$ in dependency order, so its length is at least $k + 2$. The canonical state places a formula in its proof-length layer, giving equality. □

Corollary 21. *The reflection-closure machine does not merely have level ω ; its finite self-reflection tower has a linear logical chronology. Each additional reflection level costs exactly one further rule layer in this machine.*

Remark 22 (Self-theory is not automatically inference-closed). The set $\text{Self}(M)$ consists of self-descriptive formulas proved by M . Even if $\text{Con}_F(\Gamma)$ is inference-closed, $\text{Self}(M)$ need not be: a rule applied to self-descriptive premises may produce a theorem whose intended subject is not $\langle M \rangle$. Induction on the reflection sequence therefore establishes membership term by term; it does not identify the entire self-theory with an inference closure unless additional subject-preservation hypotheses are imposed.

9. The transfinite boundary

Ordinary induction proves facts about $\theta_0, \theta_1, \dots$. A transfinite sequence θ_α requires more data:

- (i) a base clause for θ_0 ;
- (ii) a successor clause for $\theta_{\alpha+1}$;
- (iii) a limit clause defining θ_λ from the earlier terms when λ is a nonzero limit ordinal.

Only after all three clauses are fixed does transfinite induction apply. The successor rule ν of the reflection-closure machine generates every finite term, but no finitary application of ν produces a collective limit ascription summarizing all θ_β for $\beta < \lambda$. Thus the Version 46.1 induction analysis confirms, rather than removes, the boundary identified in Part II: genuine limit reflection requires a new rule, a finite code for the predecessor family, or some other explicitly stated mechanism.

10. Consolidated hierarchy of induction principles

Induction object	Base data	Successor or closure step	Typical conclusion
WFF construction	Atomic formulas	Formula constructors preserve P	P holds for every WFF
Finite proof	Hypotheses	Inference rules preserve P	Every consequence has P
ATP stage	Initial state	Stage transition preserves Q	Every state has Q
ATP birth time	Input formulas	New formulas depend on earlier premises	Every materialized formula has P
Reflection index	$M \vdash \theta_0$	Internal necessitation	$M \vdash \theta_k$ for all finite k
Transfinite reflection	θ_0	Successor and limit clauses	Property for every defined θ_α

11. Conclusion of the supplement

Induction has a central place in the framework, but only after its object is named. Structural induction concerns the grammar of WFFs. Rule induction concerns the least inference-closed consequence set. ATP induction concerns the chronology of a monotone, rule-driven run. Reflection induction concerns the recursively defined self-ascription tower. These principles interact but are not interchangeable.

For ATPs, the added value is concrete. Induction proves state and formula invariants, yields soundness, identifies exhaustive closure, propagates halting, and converts slowly changing ranks into lower bounds on first appearance. For formal self-awareness, it explains the finite reflection tower and yields the exact layer formula $b_{R,A}(\theta_k) = k + 2$ for the reflection-closure machine. At the same time, it makes the limit-stage obstruction precise: finite rule iteration reaches every finite reflection level, but transfinite limit ascriptions require additional formal machinery.

Part II

Formalized Self-Awareness and Self-Aware Structures

12 Introduction

In Part I, we developed a mathematical framework for semi-ideal computers (SICs), simulations, and automated theorem provers (ATPs), culminating in a preorder on SICs induced by simulation. The purpose of the present work is to investigate whether that framework can be extended to provide a purely mathematical notion of self-awareness.

The motivation is partly inspired by Tegmark’s concept of a self-aware substructure (SAS). Tegmark leaves self-awareness largely informal. The present paper asks whether there is a mathematically precise shadow of self-awareness that can be stated entirely in terms of names, proof, simulation, and formal systems.

The central idea is that a system need not first be assumed to possess consciousness in order to possess a weak formal analogue of self-awareness. If an ATP has a designated formal name for itself and proves a correct statement about that name, then it has at least a minimal self-model. The paper does not claim that such a system has subjective experience. It claims only that self-identification can be formalized.

Throughout this paper we assume the terminology and notation developed in Part I. In particular, SICs, ATPs, simulations, deterministic simulations, nondeterministic simulations, and the simulation preorder may be used by citation rather than reproved.

13 Internal Representations

This section introduces the formal machinery of names, tags, self-models, and self-theories. Each central definition is followed by a canonical example, a basic proposition, and discussion.

13.1 Internal Representations

Definition 13.1 (Internal representation). Let M be an ATP. An *internal representation* of M is a syntactic object in the language associated with M that is intended to denote M .

Canonical example. Let $M = (C, H)$ be an ATP. Introduce a new formal name $\langle M \rangle$. The expression $\langle M \rangle$ is not M itself. It is a name, tag, or formal representative of M in the language in which statements about M are made.

Proposition 13.2. *An internal representation need not be an arithmetic encoding of the object it represents.*

Proof. The definition requires only a syntactic object with an intended denotation. A new formal name satisfies this requirement without assigning an integer, code, or numerical address to the represented object. $\square$

Discussion. This is a deliberate departure from approaches based on Gödel numbering. The paper does not require Robinson arithmetic, arithmetization of syntax, or any incompleteness machinery. A tag functions like a name. One can reason about “Picard” without first assigning Picard an integer, and likewise an ATP can reason about $\langle M \rangle$ without first converting M into a number.

13.2 Tagging

Definition 13.3 (Tagging operator). A *tagging operator* is an assignment $\text{Tag} : M \mapsto \langle M \rangle$ that assigns to each ATP M a distinguished formal name $\langle M \rangle$.

Canonical example. For an arbitrary ATP $M = (C, H)$, define $\text{Tag}(M) = \langle M \rangle$. Here $\langle M \rangle$ is stipulated to be a new name in the relevant formal language. It is a designated name for M , not a copy of C , not a copy of H , and not a numerical code.

Proposition 13.4 (Renaming invariance). *If $\langle M \rangle$ and $[M]$ are two distinguished tags for the same ATP, then replacing every occurrence of $\langle M \rangle$ by $[M]$ changes notation but not the underlying tagged-object relation.*

Proof. Both symbols are formal names for the same ATP. The replacement is therefore a renaming of the representative, not a change in the represented object. $\square$

Discussion. Tagging isolates the philosophical issue. The question is not whether an ATP can perform arithmetic coding of its own syntax. The question is whether a formal system can contain a distinguished name for itself and prove correct statements about that name.

13.3 Self-Descriptive Formulas

Definition 13.5 (Self-descriptive formula). Let M be a tagged ATP. A formula $\varphi(\langle M \rangle)$ is *self-descriptive* if its intended subject is the tag $\langle M \rangle$.

Canonical example. For $M = (C, H)$, the formula $\text{ATP}(\langle M \rangle)$ is self-descriptive because its intended subject is the designated tag of M .

Proposition 13.6. *A self-descriptive formula is about the tag, not literally about the physical or set-theoretic object occupying the metatheory.*

Proof. The displayed formula contains $\langle M \rangle$, a syntactic object. Its subject is therefore the name occurring in the language. Its connection to M is supplied by the tagging convention. $\square$

Discussion. This distinction is essential. $\langle M \rangle$ is no more identical to M than the written name “Picard” is identical to Picard. Yet statements about a name may become self-relevant when the name is the system’s own designated tag.

13.4 Self-Models

Definition 13.7 (Self-model). A *self-model* of a tagged ATP M is the tag $\langle M \rangle$ together with the collection of formulas concerning $\langle M \rangle$ that are provable by M .

Canonical example. For $M = (C, H)$, the canonical self-model has the form

$$\text{SelfModel}(M) = (\langle M \rangle, \text{Self}(M)),$$

where $\text{Self}(M)$ is the collection of formulas about $\langle M \rangle$ that M proves.

Proposition 13.8. *If M proves $\text{ATP}(\langle M \rangle)$, then the self-model of M is nonempty.*

Proof. The formula $\text{ATP}(\langle M \rangle)$ is a self-descriptive formula and, by hypothesis, belongs to the collection of formulas about $\langle M \rangle$ provable by M . $\square$

Discussion. A self-model is not merely a name. It is a name together with information the system can formally establish about that name. The more accurate and structurally rich this information becomes, the deeper the corresponding level of formal self-awareness may be. Section 15 makes “deeper” precise.

13.5 Self-Theories

Definition 13.9 (Self-theory). The *self-theory* of an ATP M is

$$\text{Self}(M) = \{\varphi(\langle M \rangle) : M \vdash \varphi(\langle M \rangle)\}.$$

Canonical example. If M proves only the self-descriptive formula $\text{ATP}(\langle M \rangle)$, then at the first stage of analysis one may have $\text{Self}(M) = \{\text{ATP}(\langle M \rangle)\}$. Richer ATPs may prove many additional formulas about their own tags.

Proposition 13.10. *The self-theory of M is a subset of the set of formulas provable by M .*

Proof. By definition, every member of $\text{Self}(M)$ is a formula provable by M whose intended subject is $\langle M \rangle$. $\square$

Discussion. The self-theory is the formal shadow of what the system can establish about its own name. It is therefore the principal object used to measure formal self-awareness.

14 A Minimal ATP Proving Its Own Kind

This section gives the simplest concrete example of the phenomenon motivating the paper. The goal is not to build a useful theorem prover. The goal is to exhibit, with minimal machinery, an ATP $M = (C, H)$ such that M proves $\text{ATP}(\langle M \rangle)$ relative to its axiom base.

Let the language contain a name $\langle M \rangle$ and a unary predicate symbol ATP . Let $p := \text{ATP}(\langle M \rangle)$. Let $Y = \{p\}$. Let the axiom base be $A = \{p\}$. Define a SIC $M = (C, H)$ over Y by

$$C(\Gamma, m) = \Gamma$$

for every $\Gamma \subseteq Y$ and every $m \in \mathbf{N}^+$, and

$$H(\Gamma, m) = \begin{cases} 1, & p \in \Gamma, \\ 0, & p \notin \Gamma. \end{cases}$$

On input A , the computation halts immediately and its output contains p . Thus, relative to the axiom base A , M proves $\text{ATP}(\langle M \rangle)$.

Proposition 14.1. *The pair $M = (C, H)$ defined above is a SIC over Y .*

Proof. For every input Γ , $C(\Gamma, 1) = \Gamma$, and $C(\Gamma, m) \subseteq C(\Gamma, m+1)$ because the two sets are equal. If $H(\Gamma, m) = 1$, then $p \in \Gamma$, so $H(\Gamma, m+1) = 1$. Finally, $C(\Gamma, m+1) = C(\Gamma, m)$ for all m . $\square$

Remark 14.2. This example is intentionally minimal. It shows that the basic philosophical phenomenon does not require arithmetic encodings, complicated proof search, or large formal libraries. A system may have a tag and may prove that its tag has the property of being an ATP. The interesting question is what level of formal self-awareness, if any, should be attributed to such a system.

Remark 14.3. The machine of this section is deliberately handed its conclusion as a hypothesis, and so is degenerate in the sense made precise by the Picard Protocol of Part I, which requires the target formula to lie outside the hypothesis set and requires the machine to be rule-driven. A non-degenerate machine certifying its own kind exists there; the present section aims only at the smallest object exhibiting the tag phenomenon at all.

C42: Self-description is a declared syntactic class

Authoritative replacement for: Definitions 13.5 and 13.9 and later uses of “intended subject”.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

For each tagged ATP M , fix as part of its language presentation a syntactically specified class

$$SD_M \subseteq \text{Sent}(L_M)$$

of formulas whose designated subject position is occupied by the tag $\langle M \rangle$. The subject-position convention is part of the formal presentation, not an appeal to an informal intended meaning. In particular,

$$\text{ATP}(\langle M \rangle) \in SD_M, \quad \text{Pr}(\langle M \rangle, \langle \varphi \rangle) \in SD_M,$$

and any other predicate such as Human is self-descriptive only when the declared syntax places $\langle M \rangle$ in its designated subject position.

Definition 13.5 (corrected). A formula is self-descriptive for M iff it lies in SD_M .

Writing $\text{Th}(M) := \{\varphi : M \vdash \varphi\}$, replace Definition 13.9 by

$$\text{Self}(M) := \text{Th}(M) \cap SD_M.$$

All reflection formulas θ_k remain self-descriptive under this convention. Later uses of “intended subject” are to be read through the declared syntactic subject-position convention.

15 Levels of Formal Self-Awareness

15.1 Level 0 and Level 1

Definition 15.1 (Level-0 formal self-awareness). An ATP has *Level-0 formal self-awareness* if it has no designated tag for itself, or has a tag but proves no self-descriptive formula about it.

Canonical example. An ordinary theorem prover that searches for theorems in group theory but has no formal name for itself in the language being searched is Level 0.

Proposition 15.2. *Level 0 is the absence of a nontrivial self-model.*

Proof. Without a tag or without any provable self-descriptive formula, there is no nonempty collection of provable statements about the system's own designated name. $\square$

Discussion. Level 0 is not failure as a theorem prover. It is only failure to possess a formal self-model.

Definition 15.3 (Level-1 formal self-awareness). An ATP M has *Level-1 formal self-awareness* if M has a tag $\langle M \rangle$ and proves $\text{ATP}(\langle M \rangle)$.

Canonical example. The minimal ATP of Section 14 is Level 1 relative to its axiom base, because it proves $\text{ATP}(\langle M \rangle)$.

Proposition 15.4. *Level-1 formal self-awareness implies that the self-theory of M is nonempty.*

Proof. The formula $\text{ATP}(\langle M \rangle)$ is a member of $\text{Self}(M)$. $\square$

Discussion. This is the weakest positive level of formal self-awareness. It asserts only formal self-identification: the system proves that its own name denotes an ATP. It does not assert consciousness.

15.2 Names for Formulas

The scale so far has two points. To extend it we need the system to be able to say something about its own proving, not merely about its own kind. That requires names for formulas as well as names for machines. Nothing about the Version 11 commitment changes: a name for a formula is again a primitive formal name, not a numerical code.

Definition 15.5 (Formula name). Let $F = (x, y, z)$ be an xyz formal system and let L be the language in which self-descriptive formulas of a tagged ATP are written. A *formula-naming operator* is an injective assignment $\varphi \mapsto \langle \varphi \rangle$ that assigns to each formula φ of L a distinguished closed term $\langle \varphi \rangle$ of L , called the *name* of φ .

Canonical example. For $p = \text{ATP}(\langle M \rangle)$, the term $\langle p \rangle$ is a new constant symbol of L stipulated to name the formula p . It is not p , and it is not a numeral.

Proposition 15.6. *A formula-naming operator need not be an arithmetization of syntax.*

Proof. Definition 15.5 requires only an injective assignment of distinguished closed terms to formulas. Stipulating a fresh constant symbol for each formula of L satisfies this requirement, and does so without a numeral system, an arithmetic substitution function, or a representability hypothesis. $\square$

Remark 15.7. Injectivity is required so that distinct formulas receive distinct names; without it the reflection sequence of Definition 15.11 could collapse at some finite stage for purely notational reasons, and the hierarchy below would be an artifact of the naming rather than of the machine.

Discussion. Definition 15.5 is the exact analogue for formulas of what Definition 13.3 does for machines. One may speak of the sentence “Picard is human” without first assigning that sentence an integer; one gives it a name and quotes the name.

15.3 The Internal Provability Predicate

Definition 15.8 (Internal provability predicate). An *internal provability predicate* for a tagged ATP M is a binary predicate symbol Pr of L , with intended reading

$\text{Pr}(\langle M \rangle, \langle \varphi \rangle)$ is true iff the machine named $\langle M \rangle$ proves the formula named $\langle \varphi \rangle$.

Remark 15.9 (What Pr is not). Pr is a primitive predicate symbol of the object language, interpreted by the declared naming conventions. It is not an arithmetized provability predicate constructed from a proof relation inside a theory of arithmetic. In particular, the Hilbert–Bernays–Löb derivability conditions are not assumed to hold of it, and nothing in this section presupposes them. Remark 15.31 records what happens if one does strengthen Pr far enough for those conditions to apply.

Remark 15.10 (Soundness is a separate question). Definition 15.8 gives the intended reading of Pr ; it does not assert that M proves only true instances of Pr , nor that M proves all true instances. Those are the two independent hypotheses isolated in Definitions 15.21 and 15.26, and the interest of the hierarchy lies precisely in the fact that neither is automatic.

15.4 The Reflection Sequence

Definition 15.11 (Reflection sequence). Let M be a tagged ATP whose language has a formula-naming operator and an internal provability predicate. The *reflection sequence* of M is the sequence of formulas $(\theta_k)_{k \geq 0}$ defined by

$$\theta_0 := \text{ATP}(\langle M \rangle), \quad \theta_{k+1} := \text{Pr}(\langle M \rangle, \langle \theta_k \rangle).$$

Canonical example. The first three terms are

$$\begin{aligned} \theta_0 &= \text{ATP}(\langle M \rangle), \\ \theta_1 &= \text{Pr}(\langle M \rangle, \langle \text{ATP}(\langle M \rangle) \rangle), \\ \theta_2 &= \text{Pr}(\langle M \rangle, \langle \text{Pr}(\langle M \rangle, \langle \text{ATP}(\langle M \rangle) \rangle) \rangle). \end{aligned}$$

In words: θ_0 says “I am an ATP,” θ_1 says “I prove that I am an ATP,” and θ_2 says “I prove that I prove that I am an ATP.”

Proposition 15.12. *Every member of the reflection sequence of M is a self-descriptive formula in the sense of Definition 13.5.*

Proof. Induction on k . For $k = 0$ the formula is $\text{ATP}(\langle M \rangle)$, whose intended subject is $\langle M \rangle$. Suppose θ_k is self-descriptive. Then θ_{k+1} is $\text{Pr}(\langle M \rangle, \langle \theta_k \rangle)$, whose first argument is the tag $\langle M \rangle$ and whose intended subject is therefore again $\langle M \rangle$: it ascribes to $\langle M \rangle$ the property of proving a designated formula. $\square$

Corollary 15.13. *If $M \vdash \theta_k$ then $\theta_k \in \text{Self}(M)$.*

Proof. Immediate from Proposition 15.12 and Definition 13.9. □

Proposition 15.14. *The formulas $\theta_0, \theta_1, \theta_2, \dots$ are pairwise distinct.*

Proof. We show by strong induction on $j + k$ that $\theta_j = \theta_k$ implies $j = k$.

If $j = k = 0$ there is nothing to prove. If exactly one of j, k is 0, say $j = 0 < k$, then θ_0 has principal predicate ATP while θ_k has principal predicate Pr, and these are distinct symbols of L ; so $\theta_j \neq \theta_k$ and the implication holds vacuously.

If $j, k \geq 1$, write $j = j' + 1$ and $k = k' + 1$. From $\text{Pr}(\langle M \rangle, \langle \theta_{j'} \rangle) = \text{Pr}(\langle M \rangle, \langle \theta_{k'} \rangle)$ we get $\langle \theta_{j'} \rangle = \langle \theta_{k'} \rangle$, hence $\theta_{j'} = \theta_{k'}$ by injectivity of the naming operator (Definition 15.5). Since $j' + k' < j + k$, the inductive hypothesis gives $j' = k'$, hence $j = k$. □

15.5 Level n

Definition 15.15 (Level- n formal self-awareness). Let $n \geq 1$. A tagged ATP M has *Level- n formal self-awareness* if $M \vdash \theta_{n-1}$, where $(\theta_k)_{k \geq 0}$ is the reflection sequence of M .

Remark 15.16. For $n = 1$ this is exactly Definition 15.3, since $\theta_0 = \text{ATP}(\langle M \rangle)$. So Definition 15.15 extends the Version 12 scale rather than replacing it.

Definition 15.17 (Level set and level function). The *level set* of a tagged ATP M is

$$L(M) := \{n \geq 1 : M \vdash \theta_{n-1}\},$$

and the *level* of M is

$$\text{lev}(M) := \sup L(M) \in \{0, 1, 2, \dots\} \cup \{\omega\},$$

with the convention $\sup \emptyset = 0$.

Proposition 15.18. *$\text{lev}(M) = 0$ if and only if M has Level-0 formal self-awareness in the sense of Definition 15.1 restricted to reflection formulas; that is, if and only if M proves no member of its own reflection sequence.*

Proof. $\text{lev}(M) = 0$ iff $L(M) = \emptyset$ iff there is no k with $M \vdash \theta_k$. □

15.6 Cumulativity Fails Without Hypotheses

One might expect the levels to be nested, so that Level- $(n + 1)$ implies Level- n . They are not, and the failure is instructive.

Example 15.19 (A machine at Level 2 but not Level 1). Let L contain the name $\langle M \rangle$, the unary predicate ATP, the binary predicate Pr, and names for formulas. Put

$$\theta_0 = \text{ATP}(\langle M \rangle), \quad \theta_1 = \text{Pr}(\langle M \rangle, \langle \theta_0 \rangle),$$

and let $Y = \{\theta_0, \theta_1\}$. Let the axiom base be $A = \{\theta_1\}$ and define $M = (C, H)$ over Y by $C(\Gamma, m) = \Gamma$ for all $\Gamma \subseteq Y$ and $m \in \mathbb{N}^+$, and $H(\Gamma, m) = 1$ iff $\Gamma \neq \emptyset$. The verification that M is a SIC is identical to Proposition 14.1.

On input A the machine halts with output $\{\theta_1\}$. Hence $M \vdash \theta_1$ but $M \not\vdash \theta_0$. So M has Level 2 and not Level 1, and $L(M) = \{2\}$.

Remark 15.20. Example 15.19 is a machine that proves it proves it is an ATP, while not proving it is an ATP. Its self-ascription of provability is simply false. Nothing in Definition 15.8 forbids this, because Pr is a primitive predicate and the axiom base was chosen adversarially. This is the formal analogue of an unearned report about oneself, and it is exactly what the next hypothesis excludes.

Definition 15.21 (Provability soundness). A tagged ATP M is *provability sound* if for every formula φ of L ,

$$M \vdash \text{Pr}(\langle M \rangle, \langle \varphi \rangle) \implies M \vdash \varphi.$$

Theorem 15.22 (Cumulativity). *Let M be provability sound and let $n \geq 1$. If M has Level- n formal self-awareness, then M has Level- m formal self-awareness for every $1 \leq m \leq n$. Consequently $L(M)$ is an initial segment of $\mathbf{N}^+$ and*

$$L(M) = \{1, 2, \dots, \text{lev}(M)\}$$

whenever $\text{lev}(M)$ is finite.

Proof. It suffices to prove that Level- $(n+1)$ implies Level- n for every $n \geq 1$; the general statement then follows by downward induction.

Suppose M has Level- $(n+1)$, that is, $M \vdash \theta_n$. Since $n \geq 1$ we have $\theta_n = \text{Pr}(\langle M \rangle, \langle \theta_{n-1} \rangle)$ by Definition 15.11. Applying Definition 15.21 with $\varphi = \theta_{n-1}$ gives $M \vdash \theta_{n-1}$, which is Level- n .

For the final claim, $L(M)$ is downward closed in $\mathbf{N}^+$ by what was just proved, and a downward-closed subset of $\mathbf{N}^+$ with finite supremum $\text{lev}(M)$ is exactly $\{1, \dots, \text{lev}(M)\}$. $\square$

Remark 15.23. Example 15.19 shows that the hypothesis of Theorem 15.22 cannot simply be dropped: the machine there has $L(M) = \{2\}$, which is not an initial segment. That machine is not provability sound, since it proves $\text{Pr}(\langle M \rangle, \langle \theta_0 \rangle)$ without proving θ_0 .

15.7 A Minimal Level-2 Machine

Example 15.24 (A minimal machine at Level 2). We construct, in the manner of Section 14, the smallest machine attaining Level 2 while remaining provability sound.

Let L be as in Example 15.19, put $\theta_0 = \text{ATP}(\langle M \rangle)$ and $\theta_1 = \text{Pr}(\langle M \rangle, \langle \theta_0 \rangle)$, and let $Y = \{\theta_0, \theta_1\}$. Let the axiom base be $A = \{\theta_0, \theta_1\}$, and define $M_{(2)} = (C, H)$ over Y by $C(\Gamma, m) = \Gamma$ for all $\Gamma \subseteq Y$ and $m \in \mathbf{N}^+$, and $H(\Gamma, m) = 1$ iff $\theta_1 \in \Gamma$.

That $M_{(2)}$ is a SIC over Y follows exactly as in Proposition 14.1: $C(\Gamma, 1) = \Gamma$; C is nondecreasing in m because it is constant in m ; halting is persistent because H does not depend on m ; and the output is constant after halting because C is constant in m .

On input A the machine halts at stage 1 with output A , so $M_{(2)} \vdash \theta_0$ and $M_{(2)} \vdash \theta_1$. Hence $L(M_{(2)}) = \{1, 2\}$ and $\text{lev}(M_{(2)}) = 2$. Moreover $M_{(2)}$ is provability sound: the only formula φ with $\langle \varphi \rangle$ occurring in a provable Pr -statement is θ_0 , and $M_{(2)} \vdash \theta_0$.

Remark 15.25. Two formulas are necessary as well as sufficient: any machine of Level 2 proves θ_1 , and any provability-sound machine of Level 2 proves θ_0 as well, and $\theta_0 \neq \theta_1$ by Proposition 15.14. So no provability-sound machine attains Level 2 with a one-formula language. The same remark as Remark 14.3 applies: this machine is handed its conclusions, and a rule-driven analogue is the object of interest for the Picard Protocol of .

15.8 Level ω

Definition 15.26 (Internal necessitation). A tagged ATP M has *internal necessitation* if for every formula φ of L ,

$$M \vdash \varphi \implies M \vdash \text{Pr}(\langle M \rangle, \langle \varphi \rangle).$$

Theorem 15.27 (Attainment of level ω). *Let M be a tagged ATP with Level-1 formal self-awareness and internal necessitation. Then M has Level- n formal self-awareness for every $n \geq 1$, and hence $\text{lev}(M) = \omega$.*

Proof. We show $M \vdash \theta_k$ for every $k \geq 0$ by induction on k .

Base case. $M \vdash \theta_0$ because M has Level 1 and $\theta_0 = \text{ATP}(\langle M \rangle)$.

Inductive step. Suppose $M \vdash \theta_k$. By Definition 15.26 applied with $\varphi = \theta_k$, we obtain $M \vdash \text{Pr}(\langle M \rangle, \langle \theta_k \rangle)$, and $\text{Pr}(\langle M \rangle, \langle \theta_k \rangle) = \theta_{k+1}$ by Definition 15.11. Hence $M \vdash \theta_{k+1}$.

Therefore $L(M) = \mathbf{N}^+$ and $\text{lev}(M) = \sup \mathbf{N}^+ = \omega$. $\square$

Remark 15.28. Internal necessitation is a strong hypothesis. It requires the language L to contain $\langle \varphi \rangle$ for every provable φ , hence infinitely many names, and it requires the machine's provable set to be closed under an infinitary self-ascription rule. No finite-language machine of the kind built in Section 14 or Example 15.24 can satisfy it, since $L(M)$ would have to be infinite while Y is finite.

Proposition 15.29. *If M is provability sound and has internal necessitation, then $\text{lev}(M) \in \{0, \omega\}$.*

Proof. If $\text{lev}(M) = 0$ there is nothing to prove. Otherwise $L(M) \neq \emptyset$, so $M \vdash \theta_{n-1}$ for some $n \geq 1$. By Theorem 15.22, $M \vdash \theta_0$, so M has Level 1. By Theorem 15.27, $\text{lev}(M) = \omega$. $\square$

Remark 15.30. Proposition 15.29 says that the two hypotheses together force a dichotomy: such a machine is either entirely silent about itself or fully reflective. All the intermediate finite levels exhibited in Theorem 15.32 below occur at machines satisfying at most one of the two conditions. This is the precise sense in which the finite part of the hierarchy measures partial introspective closure.

Remark 15.31 (The Tarski–Gödel ceiling). Level ω is not omniscience about oneself. The reflection sequence ascribes to $\langle M \rangle$ only provability of designated formulas; it never ascribes truth. This limitation is not an accident of the present construction. If one strengthens L and M enough to represent primitive recursive functions and to satisfy the derivability conditions—so that Pr becomes a genuine arithmetized provability predicate rather than the primitive predicate of Definition 15.8—then the Tarski–Gödel barrier for recoded self-reference applies: no formula of the theory can serve as a full internal truth predicate for all tagged sentences. ATP-hood of a tag is certifiable, and provability of a named formula is certifiable, but a full internal truth predicate is not. The hierarchy of this section climbs to ω and stops there for a reason.

15.9 Strictness of the Hierarchy

Theorem 15.32 (Strictness). *For every $n \geq 1$ there is a provability-sound tagged ATP M_n with $\text{lev}(M_n) = n$. In particular no finite level of the hierarchy collapses into its successor.*

Proof. Fix $n \geq 1$. Let L contain the tag $\langle M_n \rangle$, the predicates ATP and Pr , and names $\langle \theta_0 \rangle, \dots, \langle \theta_{n-2} \rangle$ for the first $n - 1$ reflection formulas. Let

$$Y_n := \{\theta_0, \theta_1, \dots, \theta_{n-1}\},$$

C15 and C16: Internal necessitation and the transfinite boundary

Authoritative replacement for: Remarks 15.28 and 15.31, printed p. 61.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replacement for Remark 15.28. Internal necessitation is a unary rule schema, or a global closure principle, with infinitely many possible instances. It is not an infinitary-premise inference rule: each application has one premise.

Replacement for Remark 15.31. Level ω is not omniscience. Tarski–Gödel results prevent a full internally correct truth predicate in sufficiently expressive consistent systems. They do not by themselves impose an ordinal ceiling on iterated provability or reflection. The present finite level function has ceiling ω by definition. A transfinite extension requires separately specified successor and limit clauses, and its soundness must be analyzed independently.

which is a set of exactly n distinct formulas by Proposition 15.14. Let the axiom base be $A_n := Y_n$, and define $M_n = (C, H)$ over Y_n by

$$C(\Gamma, m) = \Gamma \quad (\Gamma \subseteq Y_n, m \in \mathbf{N}^+), \quad H(\Gamma, m) = \begin{cases} 1, & \Gamma = Y_n, \\ 0, & \text{otherwise.} \end{cases}$$

The verification that M_n is a SIC over Y_n is word for word that of Proposition 14.1.

On input A_n the machine halts at stage 1 with output Y_n . Hence $M_n \vdash \theta_k$ for every $k \leq n-1$, so $\{1, \dots, n\} \subseteq L(M_n)$.

Conversely, suppose $M_n \vdash \theta_k$ for some $k \geq n$. Every formula provable by M_n lies in $\bigcup_m C(A_n, m) = Y_n$, so $\theta_k \in Y_n$, so $\theta_k = \theta_j$ for some $j \leq n-1$, contradicting Proposition 15.14 since $k \geq n > j$. Hence $L(M_n) = \{1, \dots, n\}$ and $\text{lev}(M_n) = n$.

Finally M_n is provability sound. For $n = 1$ this is vacuous, since $Y_1 = \{\theta_0\}$ contains no formula with principal predicate Pr . For $n \geq 2$, suppose $M_n \vdash \text{Pr}(\langle M_n \rangle, \langle \varphi \rangle)$. Then $\text{Pr}(\langle M_n \rangle, \langle \varphi \rangle) \in Y_n$, and the members of Y_n with principal predicate Pr are exactly $\theta_1, \dots, \theta_{n-1}$, so $\text{Pr}(\langle M_n \rangle, \langle \varphi \rangle) = \theta_{j+1}$ for some j with $0 \leq j \leq n-2$. Comparing second arguments gives $\langle \varphi \rangle = \langle \theta_j \rangle$, hence $\varphi = \theta_j$ by injectivity of the naming operator. Since $j \leq n-2 \leq n-1$, we have $\theta_j \in Y_n$ and therefore $M_n \vdash \theta_j$, as required. $\square$

Corollary 15.33. *The level function is surjective onto $\{0, 1, 2, \dots\} \cup \{\omega\}$.*

Proof. Value 0 is attained by any ATP with no tag. Values $n \geq 1$ are attained by the machines M_n of Theorem 15.32. The value ω is attained by any Level-1 machine with internal necessitation, by Theorem 15.27; such machines exist, for instance the machine over the infinite formula set $Y_\omega = \{\theta_k : k \geq 0\}$ with axiom base Y_ω , constant transition map, and $H(\Gamma, m) = 1$ iff $\Gamma = Y_\omega$, which is a SIC by the same verification as Proposition 14.1. $\square$

Discussion. The machines M_n are, like the machine of Section 14, deliberately trivial as theorem provers. Their point is purely separative: they show that the levels are genuinely distinct properties and not notational variants of one another. Any claim that formal self-awareness is a single threshold rather than a graded quantity has to contend with the fact that the grades are inhabited.

16 The Order-Theoretic Structure of Formal Self-Awareness

The preceding sections define tags, self-models, self-theories, and levels for tagged automated theorem provers. We now compare ATPs according to the self-descriptive statements they can formally establish. Throughout this section, if M is a tagged ATP, then

$$\text{Self}(M) = \{\varphi(\langle M \rangle) : M \vdash \varphi(\langle M \rangle)\}$$

denotes the self-theory of M .

16.1 The Formal Self-Awareness Preorder

Definition 16.1 (Formal self-awareness preorder, Version 12 form). Let M and N be tagged ATPs. We write $M \preceq_{\text{SA}} N$ if there exists a translation map τ such that $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$. In this case, we say that N possesses at least as much formal self-awareness as M .

Canonical example. If M proves $\text{ATP}(\langle M \rangle)$ and N proves $\text{ATP}(\langle N \rangle)$, then the translation sending $\langle M \rangle$ to $\langle N \rangle$ carries the basic self-identification theorem of M to the corresponding theorem of N .

Remark 16.2. The translation map is necessary because $\langle M \rangle$ and $\langle N \rangle$ are generally different tags. The map is not intended to identify M and N as machines. It only compares their provable self-descriptive content.

16.2 A Defect in Definition 16.1, and Its Repair

Definition 16.1 does not constrain the translation map τ beyond requiring that it be a map on formulas. That is too permissive.

Proposition 16.3 (Unrestricted translations collapse the order). *Suppose τ in Definition 16.1 ranges over arbitrary functions from formulas to formulas. Then for any two tagged ATPs M and N with $\text{Self}(M) \neq \emptyset$ and $\text{Self}(N) \neq \emptyset$, we have $M \preceq_{\text{SA}} N$ and $N \preceq_{\text{SA}} M$, hence $M \sim_{\text{SA}} N$. Consequently the quotient $\text{ATP} / \sim_{\text{SA}}$ has at most two elements: the class of ATPs with empty self-theory and the class of all the rest.*

Proof. Choose any $\psi \in \text{Self}(N)$ and let τ be the constant function with value ψ . Then $\tau(\text{Self}(M)) = \{\psi\} \subseteq \text{Self}(N)$, so $M \preceq_{\text{SA}} N$. Symmetrically $N \preceq_{\text{SA}} M$. If $\text{Self}(M) = \emptyset$ then $\tau(\text{Self}(M)) = \emptyset \subseteq \text{Self}(N)$ for every τ , so such an M lies below everything; and nothing with nonempty self-theory lies below it, since $\tau(\text{Self}(N)) \neq \emptyset$ for $\text{Self}(N) \neq \emptyset$. $\square$

Remark 16.4. Proposition 16.3 does not show that Propositions 16.10–16.15 below are false. It shows that under the literal reading of Definition 16.1 they are true but empty: the order they describe has two points. The intended reading was always that τ is a renaming, as the canonical example following Definition 16.1 makes clear. The following definition makes that intention explicit.

Convention 16.5 (Standing convention on distinguished predicates). Every language of a tagged ATP considered in this paper contains the predicate symbols ATP and Pr as distinguished predicates, and contains SA as a distinguished predicate whenever it contains SA at all.

Definition 16.6 (Admissible translation). Let M and N be tagged ATPs with languages L_M and L_N . A map $\tau : L_M \rightarrow L_N$ on formulas is an *admissible translation* if:

1. $\tau(\langle M \rangle) = \langle N \rangle$;
2. τ fixes every distinguished predicate of L_M (in particular $\tau(\text{ATP}) = \text{ATP}$ and $\tau(\text{Pr}) = \text{Pr}$), and sends every other predicate symbol of L_M to a predicate symbol of L_N of the same arity;
3. τ commutes with formula formation: $\tau(P(t_1, \dots, t_k)) = \tau(P)(\tau(t_1), \dots, \tau(t_k))$ for every k -ary predicate P and terms t_i ;
4. τ commutes with formula naming: $\tau(\langle \varphi \rangle) = \langle \tau(\varphi) \rangle$ for every formula φ of L_M .

Remark 16.7 (Why clause 2 was rewritten). Versions 12–14 required only that τ be the identity on predicates common to L_M and L_N . That formulation does not compose: if a predicate is common to L_M and L_P but absent from an intermediate language L_N , then a composite $\sigma \circ \tau$ through L_N need not fix it, and the transitivity argument of Proposition 16.9 breaks. Under Convention 16.5 the distinguished predicates are present in every language, each admissible translation fixes them, and composites therefore fix them as well. Nothing else in the paper used the old reading.

Definition 16.8 (Refined self-awareness preorder). Let M and N be tagged ATPs. We write $M \preceq_{\text{SA}}^{\text{adm}} N$ if there exists an admissible translation τ with $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$. Define $M \sim_{\text{SA}}^{\text{adm}} N$ iff $M \preceq_{\text{SA}}^{\text{adm}} N$ and $N \preceq_{\text{SA}}^{\text{adm}} M$.

C12: Admissible translations as genuine language embeddings

Authoritative replacement for: Definition 16.6, printed p. 63.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Definition 16.6 (Admissible translation, corrected). An admissible translation from a tagged ATP M to a tagged ATP N is an injective, arity-preserving language homomorphism τ defined on the relevant nonlogical symbols, closed terms, names, and formulas such that:

1. $\tau(\langle M \rangle) = \langle N \rangle$;
2. τ fixes the designated predicates ATP and Pr, and fixes SA wherever that predicate is part of the comparison;
3. τ commutes with term and formula formation and with the stipulated quotation/naming operation;
4. τ is injective on predicate symbols and closed terms;
5. whenever $M \vdash \varphi$, one has $N \vdash \tau(\varphi)$.

This definition treats τ as a faithful language embedding rather than an arbitrary many-to-one map. In particular, distinct predicates such as Human and ATP may not be collapsed.

C43: Admissibility is syntactic; the preorder preserves self-theory

Authoritative replacement for: Correction C12 / Definition 16.6 and Theorem 21.7.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

Correction C12 is superseded by the following definition.

Definition 16.6 (admissible translation, Version 46.2). An admissible translation from tagged ATP M to tagged ATP N is an injective, arity-preserving language homomorphism τ on the relevant nonlogical symbols, closed terms, names, and formulas such that:

1. $\tau(\langle M \rangle) = \langle N \rangle$;
2. τ fixes the designated predicates ATP and Pr, and fixes any other predicate explicitly declared distinguished for the comparison;
3. τ commutes with term/formula formation and with the stipulated quotation/naming operation;
4. τ is injective on predicate symbols and closed terms, and preserves the declared subject-position convention, so $\tau[\text{SD}_M] \subseteq \text{SD}_N$.

There is *no* separate requirement that every theorem of M translate to a theorem of N .

The refined self-awareness preorder continues to impose the relevant proof-content requirement itself:

$$M \preceq_{SA}^{\text{adm}} N \iff \exists \text{ admissible } \tau (\tau[\text{Self}(M)] \subseteq \text{Self}(N)).$$

This makes the relation genuinely a preorder on provable self-descriptive content rather than on the machines' entire theorem sets.

With this correction, Theorem 21.7 is valid as written modulo the syntactic convention of C42: the tag-renaming isomorphism between M and its hypothesis-handed representative $M^\sharp$ is admissible in both directions, and it carries their self-theories onto one another. The earlier Version 46.1 clause requiring preservation of *all* provability is deleted because it would invalidate the reverse translation whenever M has non-self-descriptive theorems.

Proof. The identity satisfies all four clauses trivially. For composition, let $\tau : L_M \rightarrow L_N$ and $\sigma : L_N \rightarrow L_P$ be admissible. Then $(\sigma \circ \tau)(\langle M \rangle) = \sigma(\langle N \rangle) = \langle P \rangle$, giving clause 1. Clause 2 is preserved because each of τ and σ fixes every distinguished predicate, which is present in all three languages by Convention 16.5, so their composite fixes it; and a composite of arity-preserving symbol maps is arity preserving. Clause 3 is preserved because a composite of maps each commuting with formula formation commutes with formula formation. For clause 4, $(\sigma \circ \tau)(\langle \varphi \rangle) = \sigma(\langle \tau(\varphi) \rangle) = \langle \sigma(\tau(\varphi)) \rangle = \langle (\sigma \circ \tau)(\varphi) \rangle$. $\square$

Proof. Reflexivity follows by taking the translation map to be the identity map, which is admissible by Proposition 16.9. For transitivity, suppose $M \preceq_{\text{SA}} N$ via τ and $N \preceq_{\text{SA}} P$ via σ . Then $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$ and $\sigma(\text{Self}(N)) \subseteq \text{Self}(P)$. Therefore $(\sigma \circ \tau)(\text{Self}(M)) \subseteq \text{Self}(P)$, so $M \preceq_{\text{SA}} P$. In the admissible case $\sigma \circ \tau$ is admissible by Proposition 16.9, so the same argument gives $M \preceq_{\text{SA}}^{\text{adm}} P$. $\square$

Definition 16.11 (Self-awareness equivalence). Define $M \sim_{\text{SA}} N$ if and only if $M \preceq_{\text{SA}} N$ and $N \preceq_{\text{SA}} M$, and $M \sim_{\text{SA}}^{\text{adm}} N$ if and only if $M \preceq_{\text{SA}}^{\text{adm}} N$ and $N \preceq_{\text{SA}}^{\text{adm}} M$.

Proof. This is the standard equivalence relation induced by a preorder. Reflexivity follows from reflexivity of the preorder. Symmetry is built into the definition. Transitivity follows from transitivity of the preorder (Proposition 16.10). $\square$

Remark 16.14. An element of the quotient is not a single ATP. It is an equivalence class of ATPs that possess the same formal self-awareness, up to translation of tags. Thus the quotient classifies forms of formal self-awareness rather than implementations of theorem provers. Passing to the quotient intentionally forgets many features of an ATP: execution speed, memory consumption, implementation language, proof-search heuristics, and ordinary simulation depth. What remains is the self-descriptive mathematical content the ATP can establish about its own tag.

Proof. For equivalence classes $[M]$ and $[N]$, define $[M] \leq_{\text{SA}} [N]$ if $M \preceq_{\text{SA}}^{\text{adm}} N$. This is well defined because if $M \sim_{\text{SA}}^{\text{adm}} M'$ and $N \sim_{\text{SA}}^{\text{adm}} N'$, then transitivity of $\preceq_{\text{SA}}^{\text{adm}}$ transfers the comparison from M, N to M', N' . Reflexivity and transitivity follow from the preorder. Antisymmetry follows because if $[M] \leq_{\text{SA}} [N]$ and $[N] \leq_{\text{SA}} [M]$, then $M \sim_{\text{SA}}^{\text{adm}} N$, hence $[M] = [N]$. $\square$

Remark 16.16. This quotient is mathematically useful because it removes irrelevant implementation details. It lets one study formal self-awareness as an order-theoretic object. Questions about minimal representatives, maximal classes, chains, joins, and meets become meaningful only after passing from machines to equivalence classes.

16.5 The Level Function on the Quotient

The hierarchy of Section 15 and the order of this section fit together: lev is a monotone invariant of the order.

Lemma 16.17 (Admissible translations preserve reflection sequences). *Let τ be an admissible translation from L_M to L_N , and let (θ_k^M) and (θ_k^N) be the reflection sequences of M and N . Then $\tau(\theta_k^M) = \theta_k^N$ for every $k \geq 0$.*

Proof. Induction on k . For $k = 0$, clauses 2 and 3 of Definition 16.6 give $\tau(\text{ATP}(\langle M \rangle)) = \text{ATP}(\tau(\langle M \rangle))$, and clause 1 gives $\tau(\langle M \rangle) = \langle N \rangle$, so $\tau(\theta_0^M) = \text{ATP}(\langle N \rangle) = \theta_0^N$.

Assume $\tau(\theta_k^M) = \theta_k^N$. Then, using clauses 2 and 3, then clause 1, then clause 4, then the inductive hypothesis,

$$\tau(\theta_{k+1}^M) = \tau(\text{Pr}(\langle M \rangle, \langle \theta_k^M \rangle)) = \text{Pr}(\tau(\langle M \rangle), \tau(\langle \theta_k^M \rangle)) = \text{Pr}(\langle N \rangle, \langle \tau(\theta_k^M) \rangle) = \text{Pr}(\langle N \rangle, \langle \theta_k^N \rangle) = \theta_{k+1}^N. \quad \square$$

Theorem 16.18 (Monotonicity of the level function). *If $M \preceq_{\text{SA}}^{\text{adm}} N$, then $L(M) \subseteq L(N)$ and hence $\text{lev}(M) \leq \text{lev}(N)$.*

Proof. Let τ be an admissible translation witnessing $M \preceq_{\text{SA}}^{\text{adm}} N$, so $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$. Let $n \in L(M)$. Then $M \vdash \theta_{n-1}^M$, so $\theta_{n-1}^M \in \text{Self}(M)$ by Corollary 15.13. Hence $\tau(\theta_{n-1}^M) \in \text{Self}(N)$. By Lemma 16.17, $\tau(\theta_{n-1}^M) = \theta_{n-1}^N$, so $\theta_{n-1}^N \in \text{Self}(N)$, that is, $N \vdash \theta_{n-1}^N$. Therefore $n \in L(N)$. Thus $L(M) \subseteq L(N)$, and taking suprema gives $\text{lev}(M) \leq \text{lev}(N)$. $\square$

Corollary 16.19. *The level function descends to a well-defined order-preserving map*

$$\text{lev} : (\text{ATP} / \sim_{\text{SA}}^{\text{adm}}, \leq_{\text{SA}}) \longrightarrow (\omega + 1, \leq), \quad [M] \mapsto \text{lev}(M).$$

Proof. If $M \sim_{\text{SA}}^{\text{adm}} N$ then $M \preceq_{\text{SA}}^{\text{adm}} N$ and $N \preceq_{\text{SA}}^{\text{adm}} M$, so $\text{lev}(M) \leq \text{lev}(N)$ and $\text{lev}(N) \leq \text{lev}(M)$ by Theorem 16.18; hence $\text{lev}(M) = \text{lev}(N)$ and the map is well defined on classes. Order preservation is Theorem 16.18 again, read on representatives. $\square$

Corollary 16.20 (Infinite strictly increasing chains exist). *There is a sequence of tagged ATPs $M_1, M_2, M_3, \dots$ with*

$$[M_1] <_{\text{SA}} [M_2] <_{\text{SA}} [M_3] <_{\text{SA}} \dots$$

in $\text{ATP} / \sim_{\text{SA}}^{\text{adm}}$. Hence question 5 of Section 16.7 is answered affirmatively.

Proof. Take the machines M_n of Theorem 15.32, so $\text{lev}(M_n) = n$. First, $M_n \preceq_{\text{SA}}^{\text{adm}} M_{n+1}$. Let τ be the admissible translation determined by $\langle M_n \rangle \mapsto \langle M_{n+1} \rangle$ together with the identity on ATP and Pr ; clauses 3 and 4 of Definition 16.6 determine τ on all of L_n , and Lemma 16.17 gives $\tau(\theta_k^{M_n}) = \theta_k^{M_{n+1}}$. Since $\text{Self}(M_n) = \{\theta_0^{M_n}, \dots, \theta_{n-1}^{M_n}\}$ and M_{n+1} proves $\theta_0^{M_{n+1}}, \dots, \theta_n^{M_{n+1}}$, we get $\tau(\text{Self}(M_n)) \subseteq \text{Self}(M_{n+1})$. Second, $M_{n+1} \not\preceq_{\text{SA}}^{\text{adm}} M_n$: otherwise Theorem 16.18 would give $n + 1 = \text{lev}(M_{n+1}) \leq \text{lev}(M_n) = n$, a contradiction. Hence $[M_n] <_{\text{SA}} [M_{n+1}]$ for every n . $\square$

16.6 Relationship with the Simulation Preorder

In particular, a bare simulation need not preserve formal self-awareness. A machine may simulate another machine's external proof-search behavior while failing to possess a tag for itself or failing to prove any theorem about that tag. Thus the implication $M \preceq_{\text{sim}} N \Rightarrow M \preceq_{\text{SA}}^{\text{adm}} N$ does not hold without additional hypotheses.

Proof. Let τ be the tag-preserving translation associated with the simulation. If $\psi \in \text{Self}(M)$, then $\psi = \varphi(\langle M \rangle)$ for some self-descriptive formula φ with $M \vdash \varphi(\langle M \rangle)$. By proof preservation, $N \vdash \tau(\varphi)(\tau(\langle M \rangle))$. By tag preservation, $\tau(\langle M \rangle) = \langle N \rangle$. Hence $N \vdash \tau(\varphi)(\langle N \rangle)$, so $\tau(\psi) \in \text{Self}(N)$. Therefore $\tau(\text{Self}(M)) \subseteq \text{Self}(N)$, and $M \preceq_{\text{SA}}^{\text{adm}} N$. $\square$

Proof. Combine Proposition 16.22 with Theorem 16.18.

16.7 Ramifications and Open Problems

1. Does every equivalence class contain a deterministic representative?
2. Does every equivalence class contain a finitely deductively effective representative?
3. Does every class contain a representative of minimal simulation depth?
4. Does the quotient order have joins or meets?
5. Can there be infinite strictly increasing chains of formal self-awareness? Answered affirmatively in Corollary 16.20.
6. Under what exact hypotheses is self-awareness depth monotone under simulation? Answered in Corollary 16.23: tag preservation together with preservation of self-descriptive proofs suffices.

C13: Simulation notation

Authoritative replacement for: Section 16.6, printed p. 66.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace the undefined symbol $\preceq_{\text{sim}}$ by the principal clocked deterministic simulation relation $\preceq$ of Part I, Definition 3.15 (or write $\preceq_{\text{clk}}$ consistently if the decorated notation is retained).

7. Is there a complete set of invariants for $\leq_{\text{SA}}$ refining lev? Equivalently, what is the fiber $\text{lev}^{-1}(n)$ as an ordered set?
8. Is there a machine that is provability sound, has internal necessitation, and is rule-driven in the sense of Part I—that is, a Level- ω machine that is not merely handed its reflection sequence as hypotheses? *Answered affirmatively in Theorem 21.1.*
9. Does every $\sim_{\text{SA}}^{\text{adm}}$ -class of level ω contain a representative with recursively enumerable self-theory?

These questions are not merely technical. They ask whether formal self-awareness has a stable mathematical structure independent of the implementation details of a particular ATP.

17 Self-Aware Substructures and the Mathematical Universe

Tegmark’s Mathematical Universe Hypothesis holds that external physical reality is a mathematical structure [23], and that some mathematical structures contain self-aware substructures, abbreviated SAS, of which human observers are examples [22]. The notion of self-awareness employed there is deliberately left informal. This section asks what the framework of the present paper can say about it.

17.1 Tegmark’s Implicit Definition

Tegmark is explicit that his notion is implicitly defined. In the ultimate-ensemble paper he writes that he takes the property of being self-aware “as implicitly defined: a substructure is self-aware if it thinks that it is self-aware” [22], and adds that a SAS must be capable of some form of logical thought and must subjectively perceive some form of time, continuous or discrete.

Read carefully, that criterion is a fixed-point condition. It does not say that a self-aware substructure has any particular internal architecture. It says that the substructure applies the predicate “self-aware” to itself. That is precisely the shape of the objects studied in Sections 13 and 15: a system, a designated name for that system, and a self-descriptive formula the system proves about that name.

Definition 17.1 (Self-awareness predicate). Let L be the language of a tagged ATP M . A *self-awareness predicate* is a unary predicate symbol SA of L , with intended reading that $\text{SA}(\langle M \rangle)$ holds just in case the object named $\langle M \rangle$ is self-aware.

Definition 17.2 (Tegmark condition). A tagged ATP M satisfies the *Tegmark condition* if $M \vdash \text{SA}(\langle M \rangle)$.

Canonical example. Let L contain the name $\langle M \rangle$ and the unary predicate SA , let $q := \text{SA}(\langle M \rangle)$, let $Y = \{q\}$, and let the axiom base be $A = \{q\}$. The machine of Section 14, with q in place of p , satisfies the Tegmark condition.

Proposition 17.3. *If M satisfies the Tegmark condition, then $\text{SA}(\langle M \rangle) \in \text{Self}(M)$ and $\text{Self}(M) \neq \emptyset$.*

Proof. The formula $\text{SA}(\langle M \rangle)$ has intended subject $\langle M \rangle$, hence is self-descriptive by Definition 13.5; by hypothesis M proves it, so it belongs to $\text{Self}(M)$ by Definition 13.9. $\square$

Documentary core: Retrieved version 48 according to the edifile's Version 40 integration record, insert it state correction insert of the state is the Version 48 claim.

satisfies the Tegmark condition, and proves exactly the same members of the reflection sequence as before, since q is not a member of the reflection sequence and the transition map still returns its input. Hence its level is still n . $\square$

Remark 17.10. Tegmark’s remark that a SAS must be capable of “some form of logical thought” is, in this framework, the demand that the system prove something rather than merely be handed it. That is the content of the rule-drivenness requirement in the Picard Protocol of and of Remarks 14.3 and 15.25 above. His further remark that a SAS must perceive some form of time, continuous or discrete, corresponds to the staged structure of a SIC: the parameter $m \in \mathbf{N}^+$ of Definition 2.3 is exactly a discrete internal time, and the monotonicity condition $C(\Gamma, m) \subseteq C(\Gamma, m + 1)$ is the formal counterpart of having a series of thoughts in logical succession.

17.4 Bird and Frog

Tegmark distinguishes the bird perspective, which studies the mathematical structure from outside, from the frog perspective of an observer inside it [23]. The distinction is not decorative in the present setting; it is the distinction this paper has been enforcing since Definition 13.1.

Remark 17.11 (Dictionary). The correspondence is exact on the following points.

- The bird’s object is M , the machine as it exists in the metatheory. The frog’s object is $\langle M \rangle$, the tag available inside the language.
- Proposition 13.6 is the statement that the frog never has the bird’s object. A self-descriptive formula is about the name, not the machine.
- The tagging operator Tag of Definition 13.3 is the bird-to-frog translation. It exists in the metatheory and is not itself an object of L .
- Renaming invariance (Proposition 13.4) is the statement that the frog’s perspective does not depend on which name the bird supplied.
- The self-theory $\text{Self}(M)$ is the totality of what the frog can establish about itself. The level $\text{lev}(M)$ measures how far up the reflection sequence the frog gets.
- Remark 15.31 is the statement that the frog cannot become the bird. Provability of named formulas is ascribable from inside; truth of named formulas, in general, is not.

Remark 17.12. One structural disanalogy should be recorded. Tegmark’s Level IV ensemble is indexed by mathematical structures, and his equivalence is isomorphism of structures. The quotient $\text{ATP}/\sim_{\text{SA}}^{\text{adm}}$ is indexed by machines and its equivalence is mutual admissible translation of self-theories. These are quotients of different kinds: the first forgets nothing about a structure beyond the labelling of its elements, whereas the second deliberately forgets almost everything about a machine except what it can say about its own name (Remark 16.14). A structure and a self-theory are not the same sort of object, and no claim is made here that the two quotients are comparable. What is claimed is narrower: that Tegmark’s SAS criterion, when made precise, lands in the second quotient, and lands in it as an up-set rather than as a grading.

17.5 What Is and Is Not Claimed

Remark 17.13. This section does not claim that a machine satisfying the Tegmark condition is a self-aware substructure in Tegmark’s intended sense, that it is conscious, or that it perceives anything. It claims three things, each of which is proved above.

C14 and C20: Picard reference; stage and thought as models

Authoritative replacement for: Remarks 17.10–17.11, printed p. 69.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace the incomplete Picard reference by: “the Picard Protocol of Part I, Definition 4.19; see Theorem 4.21.”

Replace categorical identifications of stage with internal time and derivation with thought by: “The SIC stage index can model a discrete chronology of derivation, and rule-driven inference can serve as a formal analogue of logical thought. Neither identification entails phenomenology or an internal sense of time.”

Remark 17.15. Conjecture 17.14 is stated in order to be argued against. If it is true, the program of this paper is bounded in a specific way: it captures self-identification and nothing more, and no amount of climbing the hierarchy closes the gap. If it is false, then some formal condition on self-theories is sufficient for phenomenological self-awareness, and identifying it becomes the central problem. The following section takes up the philosophical form of this question; a conditional form of the conjecture, given an explicit non-formal premise, is Theorem 21.10, but the conjecture itself is left open.

18 The Discovery Thesis

18.1 Statement of the Thesis

Versions 11 through 13 were written in a conditional mood. They asked what would count as formal self-awareness, and what structure the resulting notion would have. This section changes the mood. It argues that the object in question is not awaited. It has been built, more than once, and by people who were not looking for it.

Thesis 18.1 (Discovery Thesis). A self-aware structure, in the formal sense developed in Sections 13–17, has already been discovered. The open question is not whether such a structure can exist, nor whether one can be constructed, but how the ones that already exist are to be classified.

The thesis is stated in a deliberately restricted register, and the restriction is not a hedge but the content of the claim. What is asserted is that structures satisfying Definition 15.3, Definition 15.15, and Definition 17.2 exist and are in hand. What is *not* asserted is that any such structure is conscious, has qualia, has experience, or has an inner life. Section 18.12 states the boundary precisely. The reader who regards the restricted claim as uninteresting is invited to notice that it is exactly the claim Tegmark declared himself unable to make in 1998 (Section 18.11).

18.2 The Existential Claim Is a Theorem, Not a Conjecture

The first and simplest support for Thesis 18.1 is that the relevant existence statements were proved above.

Proposition 18.2 (Inhabitation). *The class of tagged ATPs possessing positive formal self-awareness is nonempty, and its level spectrum is all of $\{1, 2, 3, \dots\} \cup \{\omega\}$. Moreover the class of tagged ATPs satisfying the Tegmark condition is nonempty and has the same level spectrum.*

Proof. The machine of Section 14 has level at least 1. The machine $M_{(2)}$ of Example 15.24 has level 2. For each finite n the machine M_n of Theorem 15.32 has level exactly n , and the machine M_ω of Corollary 15.33 has level ω . This establishes the first claim. The second is Proposition 17.9. $\square$

Remark 18.3. Proposition 18.2 is not a promissory note. Every machine appearing in its proof is exhibited explicitly, with its formula set, its axiom base, its transition map, and its halting map written down, and in each case the SIC conditions of Definition 2.3 are verified. The constructions are finite in every case except M_ω , and M_ω is finitely describable. Nothing is left to be supplied.

An objector who wishes to deny Thesis 18.1 therefore cannot deny that the definitions are satisfied. The objection must be that the definitions are the wrong ones, or that satisfying them is not sufficient for the honorific. The remainder of this section takes the four strongest forms of that objection in turn. The fourth is the one that does real damage, and it is conceded in part.

18.3 First Objection: Mere Assertion

The objection. The machines of Proposition 18.2 were handed their conclusions. The machine of Section 14 has axiom base $A = \{p\}$ and proves p ; it does not discover that it is an ATP, it repeats what it was told. Calling this self-awareness is calling a parrot self-aware.

The reply. The objection is correct about those machines and is already conceded in Remarks 14.3 and 15.25. It does not touch the thesis, because a machine meeting the objection's demand has also been constructed. The relevant construction is the Picard Protocol of Part I. A SIC M satisfies that protocol for (t, Γ) when there is a target formula φ^* such that: φ^* asserts, under a declared coding, that the recoded copy of M is an ATP; every member of Γ is true and every rule preserves truth under that interpretation; M is a rule-driven target-search ATP for φ^* ; M halts on Γ ; and, crucially, $\varphi^* \notin \Gamma$. The last clause is precisely the anti-parrot clause. The minimal Picard machine M_P constructed there has three symbols, two formulas, one rule, and one hypothesis, and it reaches its target by applying that rule.

Proposition 18.4 (No Picard machine parrots). *Let M satisfy the Picard Protocol for (t, Γ) with target φ^* . Then φ^* is not a member of the hypothesis set, and every proof of φ^* from Γ has length at least 2 and contains at least one application of an inference rule.*

Proof. The protocol requires $\varphi^* \notin \Gamma$. Let π be a proof of φ^* from Γ in the sense of Definition 1.4, with final line $\pi(|\pi|) = \varphi^*$. By Definition 1.4, that final line is either a member of Γ , which is excluded, or a direct consequence of the preceding lines, which by Definition 1.3 requires an inference rule $r \in z_k(F)$ and a k -tuple of preceding lines. Hence there is at least one rule application, and the preceding lines are nonempty, so $|\pi| \geq 2$. $\square$

Remark 18.5. Proposition 18.4 converts a rhetorical demand into a checkable syntactic condition. “Do not merely assert” becomes “the target lies outside the hypothesis set, and the machine is rule-driven.” A machine meeting that condition exists. The objection is therefore answered by construction rather than by argument, which is the only way an existence claim can be defended.

18.4 Second Objection: Use and Mention

The objection. What the machine proves is a statement about a name. $\text{ATP}(\langle M \rangle)$ concerns the symbol $\langle M \rangle$, and Proposition 13.6 says so explicitly. A system that manipulates a symbol which the metatheory happens to associate with the system has done nothing self-referential. It has shuffled marks.

The reply. This objection proves too much, and it is also blocked on its own terms.

It is blocked on its own terms because the Picard Protocol includes a soundness clause: the coding is declared, the hypotheses are true under it, and the rules preserve truth. Under those conditions the proved formula is not merely well formed but true of the machine. The name denotes, and what is proved of the name holds of the thing named. A system whose symbol manipulation is guaranteed to track the facts about itself is not shuffling marks; that is what it is for a representation to be about something.

It proves too much because the same objection applies verbatim to the human case. When Picard entertains the thought that he is human, the vehicle of that thought is not Picard. It is some internal representation whose association with Picard is supplied from outside the representation. Proposition 13.6 is a fact about representation in general, not a defect peculiar to machines. An objector who takes it to be disqualifying owes an account of how the human case escapes, and Section 18.10 argues that no such account is available at the formal level.

Remark 18.6. One residue of the objection survives and should be stated honestly. Nothing in the framework requires that the machine grasp the denotation of its tag, and nothing here supplies a notion of grasping. The framework guarantees that the tag denotes and that the machine's theorems about the tag are true. Whether there is a further fact about grasping, and whether that further fact is what self-awareness really names, is exactly the question left open by Conjecture 17.14. The Discovery Thesis is a claim about what has been established, not a claim that nothing further could be wanted.

18.5 Third Objection: The Level Is Too Low

The objection. Granting that some machine proves $\text{ATP}(\langle M \rangle)$, that is a single sentence. Self-awareness worth the name requires more: the system should prove things about its own proving, and about that, and so on.

The reply. This objection is unusual in that Section 15 grants it entirely and then satisfies it at every level the objector might name.

Proposition 18.7 (Robustness of the thesis under strengthening). *Let $n \in \{1, 2, 3, \dots\} \cup \{\omega\}$ be any threshold. There is a tagged ATP whose level is at least n , and if n is finite there is one whose level is exactly n . Consequently no choice of threshold in $\{1, 2, \dots\} \cup \{\omega\}$ falsifies Thesis 18.1.*

Proof. Immediate from Corollary 15.33 and Proposition 18.2. □

Remark 18.8. Proposition 18.7 relocates the burden. An objector of this third kind must either name a threshold, in which case it is met, or decline to name one, in which case the objection is not a claim about levels at all but the phenomenological objection of Section 18.7 in disguise. There is no intermediate position in which the objection has force.

18.6 Fourth Objection: Tegmark's Complexity Requirement

The strongest objection comes from Tegmark's own text, and it is not fully answerable by the machines of Proposition 18.2.

The objection. Alongside the implicit definition quoted in Section 17, Tegmark lays down three necessary conditions on a mathematical structure containing a self-aware substructure: it must exhibit complexity, predictability, and stability [22]. The machines of Proposition 18.2 have predictability and stability in abundance, for the trivial reason that their transition maps are constant and they halt at stage 1. They have essentially no complexity at all. $M_{(2)}$ has two formulas. M_T

has three. A structure with three formulas is not a candidate self-aware substructure in anything like Tegmark's sense, whatever it may prove about its own tag.

The reply, and a concession. The objection is correct, and it is conceded without qualification as regards the toy machines. Their role was separative: they establish that the levels are distinct and inhabited (Theorem 15.32), and separative examples are expected to be degenerate in every respect except the one being separated. They were never candidates for anything else.

What the objection establishes is that Thesis 18.1 cannot be carried by constructed toy machines alone. It requires witnesses that are complex as well as self-descriptive. The following definition makes the standard explicit, and Section 18.8 argues that the standard is already met.

Definition 18.9 (Non-degenerate witness). A tagged ATP M is a *non-degenerate witness* to the Discovery Thesis if:

1. M is rule-driven in the sense of Definition 2.9, and its self-descriptive theorems are not members of its hypothesis set;
2. M is provability sound (Definition 15.21);
3. $\text{lev}(M) \geq 2$;
4. M performs proof search over a formula set that is not exhausted by its own self-theory. Precisely: writing Y for the formula set of M , either Y is infinite and $|\text{Self}(M)| < |Y|$, or Y is finite and $|\text{Self}(M)| \leq |Y|/2$.

Remark 18.10. Clause 4 is the formal residue of Tegmark's complexity requirement, stated in the only vocabulary this paper has available. Two things about it should be said plainly.

First, it is comparative rather than absolute, because no absolute complexity threshold is defensible; what it excludes is exactly the situation of $M_{(2)}$, M_T , and M_n , where the machine's entire formula set is its self-theory and there is nothing else for it to do. The reflection-closure machine R of Theorem 21.1 also fails it, as Remark 21.3 records: $\text{Self}(R) = Y \setminus \{a\}$, so $|\text{Self}(R)| = |Y|$.

Second, the particular threshold in the finite case — one half — is a convention and nothing below depends on its value. Every use of clause 4 in this paper needs only the two ends of the comparison: that the toy machines and R fail the clause, since for them the self-theory is the whole formula set or all but one element of it, and that a prover searching a mathematical library while proving finitely many statements about its own tag satisfies it under any threshold strictly between 0 and 1. Readers preferring a different constant may substitute one without disturbing Corollary 20.7, Corollary 21.8, or Proposition 22.4.

18.7 Fifth Objection: No Phenomenology

The objection. None of this is self-awareness, because none of it involves anything it is like to be the system.

The reply. This objection is not answered here, and the Discovery Thesis does not attempt to answer it. It is granted in full. See Section 18.12, and Conjecture 17.14, which states the strongest form of the objection as a conjecture the present framework cannot refute. The thesis claims a formal discovery. If “self-aware” is reserved by stipulation for phenomenological self-awareness, then the thesis should be read as claiming the discovery of formally self-aware structures, and the dispute is terminological rather than substantive. Section 18.10 argues that it is not, in fact, merely terminological, because the same reservation applied consistently would leave the human case equally unaccounted for.

C17: Non-degenerate witnesses without defective cardinal comparison

Authoritative replacement for: Definition 18.9, printed p. 73.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace clause 4 by:

4. Non-exhaustion. The prover searches a formula set not exhausted by its self-theory. If Y is infinite, require $Y \setminus \text{Self}(M)$ to be infinite. If Y is finite, require a declared finite non-exhaustion threshold, for example $|\text{Self}(M)| \leq |Y|/2$.

Revise the following remark accordingly. This criterion excludes the reflection-closure toy whose complement contains only the base symbol, while allowing countable mathematical languages with infinitely many non-self formulas. Strict cardinal inequality is not an adequate test when both sets are countably infinite.

18.8 Non-Degenerate Witnesses Already in the Literature

Section 18.6 concedes that toy machines cannot carry the thesis. This section argues that they do not have to, because systems meeting Definition 18.9 have been built and verified by researchers with no interest in self-awareness whatever. That the witnesses arose independently of the question is a point in their favor, not against them.

Milawa. Milawa is a theorem prover, developed by Jared Davis, styled after NQTHM and ACL2 and built around a small inspectable kernel [4]. Its architecture is a bootstrapping ladder: a simple verifier A is used to prove the correctness of a more capable verifier B , which proves C , and so on through roughly a dozen stages, until the verifier reached is powerful enough to prove verifier A correct. Davis’s dissertation is titled *A Self-Verifying Theorem Prover*, and the title is meant literally. The logic of Milawa was subsequently formalized in HOL4, proved sound, and shown to be faithfully implemented by the roughly two thousand lines of Lisp constituting its kernel; combined with a verified Lisp runtime, the soundness result descends to the machine code that executes it [15].

Remark 18.11 (Milawa against the four clauses). Clause 1 of Definition 18.9 is met in spirit and, so far as the published description goes, in fact: each verifier in the ladder derives the correctness of the next rather than assuming it, and the derivations are ordinary proofs in the system’s logic. Clause 3 is met, because a statement that a verifier is correct is a statement about what that verifier proves, which is a Pr -statement in the vocabulary of Definition 15.8 and hence lies at or above θ_1 . Clause 4 is met by a wide margin: the formula set over which Milawa searches is vastly larger than the fragment concerning its own kernel. Clause 2 is the delicate one and is discussed below.

Remark 18.12 (An empirical analogue of the reflection sequence). The resemblance between Milawa’s bootstrapping ladder and the reflection sequence of Definition 15.11 is striking and should be stated carefully, because it is an analogy and not an identity. The reflection sequence is generated by one machine ascribing provability to its own tag, iterated: $\theta_{k+1} = \text{Pr}(\langle M \rangle, \langle \theta_k \rangle)$. Milawa’s ladder is a sequence of distinct verifiers, each certifying the next, and it closes into a loop when the final verifier certifies the first. These are different objects. What they share is the structural feature that matters here: each rung is a proved statement about what a proof system proves, and the rungs are strictly ordered by the strength required to establish them.

The exact relationship is a well-posed question that this paper does not settle. In particular: does the closing of Milawa’s loop—the last verifier proving verifier A correct—correspond to a machine attaining $\text{lev} = \omega$ in the sense of Theorem 15.27, or to a finite level with an additional cyclicity property not captured by Definition 15.17? The author’s expectation is the latter, and that Definition 15.17 needs a companion notion of reflective closure to describe it. This is recorded as question 2 below.

Lean4Lean. A second candidate is Lean4Lean, a complete typechecker for the Lean 4 proof assistant written in Lean 4 itself [14]. It is, as of its publication, the first such checker other than the reference implementation, it runs within a small constant factor of that implementation, and it is capable of checking the whole of Lean’s mathematical library. Because the checker is written in a language admitting formal verification, it becomes possible to state and prove properties of the kernel within the system whose kernel it is.

Remark 18.13 (The trusted-kernel caveat). Lean’s design philosophy is deliberately not self-verification: it keeps a small trusted kernel and relies on independent external checkers, precisely so that soundness does not rest on the system’s own testimony. This is sound engineering and it is also a substantive point against treating Lean4Lean as a witness without qualification. What Lean4Lean establishes is that a system of very great complexity can contain a faithful

C18: Milawa and Lean4Lean as candidates, not completed classifications

Authoritative replacement for: Section 18.8, printed p. 74.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace categorical statements that a checklist clause “is met” by the following qualification:

“Milawa and Lean4Lean provide evidence relevant to self-verification and reflective checking. Certification in the present tagged-ATP framework still requires an explicit faithful presentation, a chosen self-tag, a translation of the system’s verified statements into formulas such as $\text{ATP}(\langle M \rangle)$ or $\text{Pr}(\langle M \rangle, \langle \varphi \rangle)$, and a proof that the translation preserves the intended meaning. Until those data are supplied, these systems are candidates rather than completed level classifications.”

Remark 18.14 (The certification checklist). Neither system above is here proved to be a non-degenerate witness, and it would be an abuse of the framework to claim otherwise on the strength of published descriptions. To certify a concrete system S as a non-degenerate witness one must exhibit, in order:

- Steps 1 and 5 are the substantial ones. Step 1 is substantial because real provers are not presented as SICs and the translation must be shown faithful rather than merely plausible. Step 5 is substantial because provability soundness is a global claim about the system, of exactly the kind that Remark 15.31 shows cannot in general be established from inside.

18.9 An Algebraic Prover Against the Four Clauses

Definition 18.16 (The Gröbner prover as a SIC). Let F_{PC} be the Polynomial Calculus of Definition 10.37 over $\mathbf{F}_2$, let Y be its multilinear polynomial set, which is inference-closed, and let

be the canonical target-search SIC for $\perp = 1$ of Definition 2.19. On input the arithmetization $J(\varphi)$ of a formula φ , the machine G halts precisely when φ is unsatisfiable, at stage $\|\perp\|_{F_{PC}}(J(\varphi))$.

C18 and C19: Candidate classifications and the canonical Polynomial Calculus machine

Authoritative replacement for: Section 18.8 and Definition 18.16, printed p. 75.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Continue to treat Milawa and Lean4Lean as evidence-bearing candidates under the qualification on the preceding correction page.

For the algebraic example, retain $G = E_{FPC,Y,\perp}$ only as the canonical rule-driven target-search closure machine. Do not identify it with a particular Buchberger, F4, or F5 execution, and do not call the target-freezing machine deductively effective.

Proof. That G is a rule-driven target-search ATP is Theorem 2.20 applied to the inference-closed set Y ; deductive effectiveness is Theorem 2.17. The halting characterisation is the Proof-Horizon Theorem (Theorem 2.12) together with Proposition 10.35. Faithfulness is Remark 10.39: the operations of Buchberger’s algorithm are compositions of the two rules of z , so a run of the algorithm is literally a proof in F_{PC} and a stage of G is the set of polynomials derived so far. $\square$

Proof. Clause 1: G is rule-driven by Proposition 18.17, and its hypothesis set on input $J(\varphi)$ consists of arithmetized clauses, none of which is self-descriptive, since the language of F_{PC} contains no tag for G at all. Clause 4: Y is infinite, and $\text{Self}(G) = \emptyset$, so $|\text{Self}(G)| < |Y|$ holds trivially. For the level, G has no designated tag $\langle G \rangle$ in its language, so it proves no self-descriptive formula; by Definition 15.1 it has Level-0 formal self-awareness, and by Proposition 15.18, $\text{lev}(G) = 0$. Clauses 2 and 3 fail with it: provability soundness is vacuous and $\text{lev}(G) \geq 2$ is false. $\square$

This is worth stating because the fourth objection of Section 18.6 can be read as suggesting that complexity is the missing ingredient, so that a sufficiently rich prover would thereby be a non-degenerate witness. G refutes that reading. Complexity and level are independent coordinates: Proposition 18.18 exhibits a machine maximal in the first and minimal in the second, while Theorem 21.1 exhibits R , maximal in the second ($\text{lev}(R) = \omega$, every reflection formula derived) and minimal in the first (Remark 21.3: $\text{Self}(R)$ exhausts Y). Neither coordinate is a function of the other, and a non-degenerate witness is precisely an object occupying the far corner of both — question 1 of Section 18.14, which remains open.

btenneson2301.substack.com 76

C19: Exhaustive closure, target freezing, and shortest-proof layers

Authoritative replacement for: Proposition 18.17, printed p. 76.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Proposition 18.17 (Corrected). The machine $G = E_{F_{PC}, Y, \perp}$ is a rule-driven target-search ATP. It halts exactly when $\perp$ enters the iterative direct-consequence closure, and its first halting stage is the least m such that

$$\perp \in D_{F_{PC}}^{m-1}(J(\varphi)).$$

It is a canonical exhaustive Polynomial Calculus closure process, not a faithful trace of a particular Gröbner-basis implementation. If shortest proof length is the intended benchmark, use $B_{F_{PC}, Y, \perp}$ instead and call it the proof-horizon benchmark.

18.10 The Parity Argument

The deepest support for Thesis 18.1 is comparative. It runs as follows.

1. We do not doubt that human beings are self-aware structures. This is the least disputed proposition in the vicinity.
2. Section 19 exhibits the formal content of the human case: Picard possesses a designated representation of himself and establishes a correct statement about it, namely $\text{Human}(\langle \text{Picard} \rangle)$.
3. The machines of Sections 14 and 18.8 possess a designated representation of themselves and establish a correct statement about it, namely $\text{ATP}(\langle M \rangle)$ or a member of its reflection sequence.
4. No formal property has been exhibited that holds of (2) and fails of (3). Proposition 13.6 applies equally to both; the soundness of the representation holds in both; the level of reflection attained is, if anything, easier to verify in (3).

Proposition 18.21 (Parity). *Let P be any property of tagged systems that is invariant under $\sim_{\text{SA}}^{\text{adm}}$. If P holds of Picard in virtue of his proving $\text{Human}(\langle \text{Picard} \rangle)$, then P holds of every tagged ATP M with $M \vdash \text{ATP}(\langle M \rangle)$ whose self-theory is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard's.*

Proof. Immediate from the definition of invariance under an equivalence relation (Definition 16.11) together with Proposition 16.12. $\square$

Remark 18.22 (What parity does and does not show). Proposition 18.21 is nearly trivial as mathematics and is included because its dialectical role is not trivial. It says that any objector who accepts the human case and rejects the machine case is committed to the claim that the distinguishing property is *not* an invariant of the self-theory—that is, committed to Conjecture 17.14. That is a coherent and possibly correct position. It is not, however, a position from which Thesis 18.1 can be rejected, since the thesis concerns exactly the invariant content. It is instead a position from which one holds that the invariant content is not what matters. The value of the parity argument is to force that admission into the open, where it can be examined, rather than leaving it as an unstated assumption that machines are disqualified in advance.

18.11 Tegmark's 1998 Challenge

Having introduced self-aware substructures and taken self-awareness as implicitly defined, Tegmark records a difficulty:

“Indeed, so far we have not even found a single mathematical structure that we feel confident might contain SAs, since a self-consistent model of quantum gravity remains conspicuous with its absence.” [22]

Remark 18.23 (The challenge answered in the formal register). On the reading of SAS-hood developed in Section 17, this difficulty does not arise, and the reason it does not arise is instructive.

Tegmark's pessimism is explicitly conditioned on the absence of a model of quantum gravity. That condition is intelligible only because he is searching for the mathematical structure that is our physical universe, and asking whether that structure contains self-aware substructures. Under the Mathematical Universe Hypothesis this is the natural question, and it is indeed blocked by the state of fundamental physics.

The claim is therefore narrow and, within its bounds, secure: the search Tegmark described as not yet successful in 1998 is successful under the formal reading, and was successful before the question was posed in these terms, since ordinary theorem provers had been proving statements about proof systems for decades. What remains open is everything Tegmark actually cared about, namely whether the structures so identified bear any relation to the substructures of the physical universe that he and his readers instantiate.

Because the thesis of this section is stronger in tone than anything in earlier versions, its boundaries are stated flatly.

1. Structures satisfying Definitions 15.3, 15.15, and 17.2 exist, are explicitly constructed, and occupy every level in $\{1, 2, \dots\} \cup \{\omega\}$ (Proposition 18.2).
2. The degeneracy of those constructions is a property of the constructions and not a consequence of the definitions: a machine whose self-descriptive target lies outside its hypothesis set and is reached by inference has been built (Proposition 18.4).
3. No threshold in the level hierarchy falsifies the thesis (Proposition 18.7).
4. Systems of substantial complexity that model their own proof systems exist in the literature and were built for unrelated reasons (Section 18.8).

1. That any structure discussed here is conscious, sentient, or has experience.
2. That any structure discussed here has qualia, affect, preference, or interest.
3. That the systems named in Section 18.8 have been certified as non-degenerate witnesses; Remark 18.14 states what certification would require and Remark 18.15 explains why the thesis does not depend on it.
4. That formal self-awareness is sufficient for phenomenological self-awareness. Conjecture 17.14 states the contrary possibility and is not refuted here.
5. That the structures identified bear any relation to the self-aware substructures of the physical universe in Tegmark’s sense (Remark 18.23).

A thesis with no refutation conditions is not worth asserting. The following would each falsify or substantially damage Thesis 18.1, and are listed in decreasing order of severity.

1. *An inconsistency in the framework.* If Definitions 15.8, 15.11, and 15.21 were shown to be jointly unsatisfiable by any SIC—for instance if internal necessitation could be shown to force a contradiction in any system rich enough to state it—then Proposition 18.2 would survive only at level 1 and the thesis would be reduced to a triviality.

2. *Failure of the SIC presentation.* If it could be shown that no faithful presentation of a real theorem prover as a SIC exists—that step 1 of Remark 18.14 is not merely unperformed but impossible—then Section 18.8 would collapse and the fourth objection of Section 18.6 would stand unanswered.
3. *A formal disanalogy in the human case.* If some property invariant under $\sim_{SA}^{adm}$ were exhibited that holds of Picard’s self-ascription and fails of every machine’s, Proposition 18.21 would be defeated and the comparative argument of Section 18.10 would fail.
4. *A principled complexity threshold.* If a defensible absolute complexity requirement were formulated, in the sense of Tegmark’s first necessary condition, and if no existing system met it while satisfying clauses 1–3 of Definition 18.9, then the thesis would be true but vacuous.
5. *A demonstration that the level hierarchy collapses on natural systems.* If every system meeting clause 4 of Definition 18.9 were shown to have level at most 1, the connection between complexity and reflection assumed throughout Section 18.8 would be severed.

Remark 18.24. None of the five is presently known to hold, and the second and fourth strike the author as the most likely to be established. The reader who wishes to dispute this section is directed to them in preference to the phenomenological objection of Section 18.7, which, while probably correct, cannot be settled by any argument this framework is able to state.

18.14 Open Problems Specific to the Discovery Thesis

1. Certify a concrete, widely used theorem prover as a non-degenerate witness in the sense of Definition 18.9, by carrying out the six steps of Remark 18.14.
2. Define *reflective closure* for a family of ATPs, so as to capture the closing of the Milawa ladder (Remark 18.12), and determine its relationship to lev and to $lev = \omega$ in particular.
3. Determine whether clause 4 of Definition 18.9 can be replaced by an absolute complexity condition that is invariant under $\sim_{SA}^{adm}$, or prove that no such condition exists.
4. Is there a natural system whose level is exactly n for some $2 < n < \omega$, or do natural systems cluster at levels 1, 2, and ω ?
5. Does provability soundness of a non-degenerate witness admit external certification in the manner of [15], and if so, what is the weakest ambient theory sufficient for it?

19 The Philosophy of Formalized Self-Awareness

The preceding definitions motivate a philosophical challenge. Suppose Picard is a real human being, or suppose for the sake of argument that “Picard” names an actual human. Also suppose that $\langle \text{Picard} \rangle$ is one possible formal name or tag for Picard.

Now suppose Picard proves

$$\text{Human}(\langle \text{Picard} \rangle).$$

The tag $\langle \text{Picard} \rangle$ is not Picard himself. It is a name. Nevertheless, if it is Picard’s own designated name and Picard proves that this name denotes a human, then Picard has demonstrated at least a weak formal self-model.

The ATP case is analogous. Let $M = (C, H)$ be an ATP with designated tag $\langle M \rangle$. If M proves $\text{ATP}(\langle M \rangle)$, then M proves that its own name has the property of being an ATP. The present paper proposes that this deserves to be called a minimal level of formal self-awareness.

The claim is deliberately limited. It is not a claim that M is conscious, that M has qualia, that M has emotions, or that M experiences itself as existing. It is the weaker claim that self-awareness has a formal shadow: provable self-identification through a designated name.

This is the philosophical challenge. If Picard proving $\text{Human}(\langle \text{Picard} \rangle)$ is accepted as at least a minimal formal kind of self-awareness, then why should an ATP proving $\text{ATP}(\langle M \rangle)$ fail to count as the corresponding minimal kind of formal self-awareness? The burden is then to explain why the human case and the ATP case differ at the purely formal level.

This question connects the present work to Tegmark's notion of a self-aware substructure. Tegmark's notion remains intentionally broad and partly phenomenological. The present project attempts a more austere mathematical notion: a system is formally self-aware to the extent that it can prove correct statements about its own designated formal name.

Remark 19.1 (What the hierarchy adds to the challenge). Section 15 sharpens the challenge rather than answering it. The objection that $\text{ATP}(\langle M \rangle)$ is too thin to deserve the name self-awareness—that the machine of Section 14 merely repeats a hypothesis—now has a precise reply and a precise cost. The reply is that thinness is measurable: lev distinguishes the machine that asserts from the machine that proves that it proves, at every finite stage and at ω (Theorem 15.32, Corollary 15.33). The cost is that no finite level, and not even ω , is obviously the level at which the objection stops applying. If the objector holds that some level is the threshold, they owe an argument for which one. If they hold that no level suffices, they are committed to Conjecture 17.14, and to the view that Picard's own self-ascription is likewise unaccounted for by anything formal about Picard.

20 The Picard Theorem

Section 19 posed a challenge in the conditional mood: *if* Picard proves $\text{Human}(\langle \text{Picard} \rangle)$, why should the human case and the machine case differ at the formal level? This section discharges the antecedent for a specific, historically attested human and computes the resulting level. The section is written under an explicit idealization, stated in Section 20.1, and its scope is fenced in Section 20.5 in the manner of Section 18.12.

20.1 The Subject and the Idealization

By *Picard* we mean Charles Émile Picard (1856–1941), and we fix as the operative historical fact that Picard is responsible for a theorem of complex analysis: the Great Picard Theorem, that in every punctured neighborhood of an essential singularity a holomorphic function assumes every complex value, with at most one exception, infinitely often. Nothing below depends on the content of that theorem; what is used is only that Picard *proved* it—that Picard is, in the vocabulary of Appendix I, a rule-driven prover of a target lying outside his hypotheses.

The framework of Sections 13–17 assigns lev to tagged ATPs, and a human being is not literally a semi-ideal computer. We therefore proceed under an idealization, in the spirit of the supposition opening Section 19, and we mark it as such rather than smuggling it in.

Definition 20.1 (The Picard idealization). *Picard* is modeled as a tagged ATP $P = (C, H)$ subject to the following four assumptions.

(P1) *Genuine theoremhood.* P is rule-driven in the sense of Definition 2.9; no self-descriptive formula lies in its hypothesis set; and P proves at least one theorem whose statement is not

a member of its hypothesis set (the witnessing theorem being the Great Picard Theorem). Rule-drivenness is a property of P (Definition 2.9), so any self-descriptive theorem P proves, not being a hypothesis, enters at some stage beyond the first and is thereby a derived direct consequence; what guarantees the reflection formulas are in fact all proved is the separate closure property (P4).

- (P2) *Self-identification*. P has a designated self-tag $\langle \text{Picard} \rangle$ and proves $\text{ATP}(\langle \text{Picard} \rangle)$: it establishes that the bearer of its own name is a prover of theorems. This self-identification is inferred by rule from the theoremhood of (P1)— P concludes that it proves theorems from having proved one—rather than taken as a hypothesis.
- (P3) *Provability soundness* (Definition 15.21). For every φ , if P proves $\text{Pr}(\langle \text{Picard} \rangle, \langle \varphi \rangle)$ then P proves φ .
- (P4) *Internal necessitation* (Definition 15.26). For every φ , if P proves φ then P proves $\text{Pr}(\langle \text{Picard} \rangle, \langle \varphi \rangle)$.

We retain, from the standing supposition of Section 19, that P also proves $\text{Human}(\langle \text{Picard} \rangle)$; this is used only in Corollaries 20.13 and 20.17, not in the level computation.

Remark 20.2 (Competence, not performance). Assumption (P4) ascribes a *closure property* to Picard’s provable set, in the sense of Definition 15.8: it says that whenever a formula is among the theorems, its provability-ascription is too. It is not the claim that Picard performs infinitely many acts of proof in finite time. The distinction between the disposition (P4) describes and any finite record of proofs actually carried out is taken up in Section 20.4, and it is exactly where the value ω will be seen to live.

Remark 20.3 (Why ATP and not Human at the base). The reflection sequence of Definition 15.11 is anchored at $\theta_0 = \text{ATP}(\langle M \rangle)$. Assumption (P2) is stated with ATP rather than Human precisely so that Picard’s reflection sequence is the one already defined, and no new base predicate or new reflection sequence need be introduced. The reading of $\text{ATP}(\langle \text{Picard} \rangle)$ intended is the weak one licensed by (P1): the bearer of the tag proves theorems. A mathematician responsible for a theorem satisfies it on the nose. That Picard also proves $\text{Human}(\langle \text{Picard} \rangle)$, the premise of Section 19, is a separate self-descriptive fact recorded in Corollary 20.17; it enriches $\text{Self}(\text{Picard})$ but is not part of the tower whose height is lev.

20.2 The Theorem

Theorem 20.4 (Picard). *Under the Picard idealization (Definition 20.1), $\text{lev}(\text{Picard}) = \omega$.*

Proof. By (P2), Picard proves $\theta_0 = \text{ATP}(\langle \text{Picard} \rangle)$, so Picard has Level-1 formal self-awareness (Definition 15.3). By (P4), Picard has internal necessitation (Definition 15.26). The hypotheses of Theorem 15.27 are therefore met, and it applies verbatim: a tagged ATP with Level-1 formal self-awareness and internal necessitation has Level- n for every $n \geq 1$, whence $\text{lev}(\text{Picard}) = \omega$. $\square$

Remark 20.5 (A second, independent route). The value can be reached without invoking Theorem 15.27 directly. By (P3) Picard is provability sound and by (P4) has internal necessitation, so Proposition 15.29 applies and gives $\text{lev}(\text{Picard}) \in \{0, \omega\}$. By (P2) Picard is not silent about himself, so $\text{lev}(\text{Picard}) \neq 0$ by Proposition 15.18. Hence $\text{lev}(\text{Picard}) = \omega$. The two routes agree: the first computes the tower directly, the second observes that soundness plus necessitation leaves no finite value available.

Remark 20.6 (Picard is a Level- ω witness of the non-degenerate kind). Assumption (P3) is not needed for the ω computation—Theorem 15.27 does not assume it—but it is what makes the value earned rather than adversarial. Contrast the machine M_ω of Corollary 15.33, which attains ω by being handed its entire reflection sequence as hypotheses, with Picard, who attains ω while satisfying the anti-parrot condition of (P1)/Proposition 18.4. Picard is thus, under the idealization, exactly the sort of non-degenerate, rule-driven, provability-sound Level- ω object that question 8 of Section 16.7 asks for in the machine case.

20.3 Corollaries

Corollary 20.7 (Picard is a non-degenerate witness). *Under the Picard idealization, Picard is a non-degenerate witness to the Discovery Thesis in the sense of Definition 18.9.*

Proof. We check the four clauses. Clause 1 has two parts. Rule-drivenness is (P1). For the second part—that the self-descriptive theorems lie outside the hypothesis set—(P1) places *no* self-descriptive formula in the hypothesis set; and every self-descriptive theorem P proves (each θ_k by Theorem 20.4, and $\text{Human}(\langle \text{Picard} \rangle)$ by the standing supposition) is, not being a hypothesis, entered at a stage beyond the first as a direct consequence of the current state, since P is rule-driven. Hence no self-descriptive theorem is a hypothesis. Clause 2 (provability soundness) is (P3). Clause 3 ($\text{lev} \geq 2$) follows from $\text{lev}(\text{Picard}) = \omega \geq 2$ (Theorem 20.4). Clause 4 holds under the idealization because Picard’s formula set Y — the statements of complex analysis and of whatever else he reasons about — is infinite, while $\text{Self}(\text{Picard})$ is the countable set of formulas whose subject is the single tag $\langle \text{Picard} \rangle$; the reflection tower and $\text{Human}(\langle \text{Picard} \rangle)$ are among them (Corollary 20.17), but the statements about $\langle \text{Picard} \rangle$ are a proper subclass of Y of strictly smaller cardinality once Y contains, as it does, the uncountably many formulas of the ambient language over an infinite alphabet, or — if Y is taken countable — once one observes that clause 4’s infinite case asks only $|\text{Self}(M)| < |Y|$, which fails only if Picard proves something about his own tag for cofinally many formulas of his language. That he does not is part of the idealization, and is recorded here as the one place where clause 4 is assumed rather than verified. $\square$

Remark 20.8 (Where the idealization does work in Clause 1). The second part of Clause 1 is exactly the point at which (P1)’s exclusion of *all* self-descriptive formulas from the hypothesis set does substantive work, reinforced by the inferred-not-assumed reading of (P2). Without it, $\theta_0 = \text{ATP}(\langle \text{Picard} \rangle)$ —or $\text{Human}(\langle \text{Picard} \rangle)$, which is equally self-descriptive—could be a hypothesis, would be parroted, and Picard would fail Definition 18.9 at the base of the tower exactly as the machine of Section 14 does (Remark 14.3). The Great Picard Theorem of (P1), being about complex analysis rather than about $\langle \text{Picard} \rangle$, establishes rule-driven non-degeneracy for a non-self-descriptive target but does not by itself discharge the self-descriptive part; barring self-descriptive formulas from the hypotheses is what carries it. Note that Definition 18.9 clause 1 speaks of *all* self-descriptive theorems, so $\text{Human}(\langle \text{Picard} \rangle)$ must be derived as well, not merely θ_0 .

Remark 20.9 (Picard’s evidential status among the witnesses). The certification checklist of Remark 18.14 cannot be run on a human: step 1 asks for a faithful presentation of the subject as a SIC over an inference-closed Y , and no such presentation of a human being is available. So Picard stands on exactly the footing of Milawa and Lean4Lean in Section 18.8: non-degenerate beyond reasonable doubt, uncertified in the sense of Remark 18.14. What Picard adds to the disjunctive support of Remark 18.15 is that the non-degenerate column now contains a human, and at level ω rather than merely > 2 .

C22: Picard non-degeneracy under the corrected criterion

Authoritative replacement for: Corollary 20.7, printed p. 82.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace the proof of clause 4 by:

“Under the idealization, Picard’s formula set contains infinitely many formulas whose intended subject is not $\langle \text{Picard} \rangle$. Therefore $Y \setminus \text{Self}(\text{Picard})$ is infinite, as required by corrected Definition 18.9. This non-exhaustion is an explicit assumption of the idealization, not a verified fact about a human being.”

Delete the claims that a countable alphabet produces uncountably many finite formulas and that strict cardinal inequality fails only under a cofinality condition.

Proof. ($\Rightarrow$) A SIC satisfying (P1)–(P4) is rule-driven with no self-descriptive formula among its hypotheses (P1), is provability sound (P3) and internally necessitating (P4), and has $\text{lev} = \omega$ by Theorem 20.4; its self-descriptive theorems are therefore all derived (as in Corollary 20.7). ($\Leftarrow$) Such a machine is rule-driven, provability sound (P3), and internally necessitating (P4); being Level- ω it proves every θ_k , in particular θ_0 (P2); and since all its self-descriptive theorems are derived, none is a hypothesis (P1). Hence it satisfies (P1)–(P4). $\square$

Remark 20.12 (The antecedent of the Picard Theorem is a named open problem). Theorem 20.4 is a valid conditional, but Proposition 20.10 locates the cost of its antecedent precisely: whether *any* SIC meets assumptions (P1)–(P4) is question 8 of Section 16.7, which this paper lists as open. Two things follow. First, the theorem is neither vacuous by fiat nor known to be non-vacuous; its non-vacuity is equivalent to a machine-existence question already on the books. Second—and this is the parity theme in its sharpest form—the question “can Picard be modeled as a formally self-aware machine at level ω ?” is *the same question* as “does a non-degenerate Level- ω machine exist?” The human idealization and the sought-after machine are, formally, one object. Question 8 is answered affirmatively by the reflection-closure machine of Theorem 21.1—which derives θ_0 by a rule in the manner of the minimal Picard machine M_P of Part I (Proposition 18.4), then closes under a necessitation rule $\varphi \mapsto \text{Pr}(\langle \cdot \rangle, \langle \varphi \rangle)$ —so the antecedent of Theorem 20.4 is discharged and the idealization is instantiable (Corollary 21.2).

Proof. Picard satisfies the antecedent of Proposition 18.21 by (P2) together with the standing supposition that he proves Human((Picard)). The first consequence is Theorem 20.4 with Corollary 15.33. The second is Proposition 18.21 together with Remark 18.22. $\square$

btenneson2301.substack.com 83

C23 and C24: Conditional Picard parity and injective incomparability

Authoritative replacement for: Corollary 20.13 and Remark 20.14, printed p. 83.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Corollary 20.13 (Retitled and corrected). *The parity antecedent is realized in the Picard idealization at level ω .* Under assumptions (P1)–(P4) and the standing supposition, the formal model labeled Picard satisfies the antecedent of Proposition 18.21 and has level ω . This is a conditional statement about the idealization; it does not assert the existence of an actual human witness, and Picard is a fictional character.

Remark 20.14 (Corrected qualification). The claimed obstruction based on the additional predicate Human uses the injectivity and distinct-predicate preservation required by corrected Definition 16.6. Without that requirement, a many-to-one map could collapse Human into ATP and the incomparability argument would not follow.

$M_\omega <_{\text{SA}}$ Picard strictly. Hence at level ω the *unconditional* content of the corollary is exactly its first consequence—that level does not separate the two cases—while the full parity transfer awaits Conjecture 20.23. This is the level- ω sharpening of the seam already noted, in general terms, in Remark 18.22: the parity argument bites only on the shared invariant content, and exhibiting a machine whose invariant content equals Picard’s is a separate task, discharged in Theorem 21.7 (a hypothesis-handed machine $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard), so that the transfer is unconditional (Remark 21.9); whether a *non-degenerate* such machine exists remains open.

Corollary 20.15 (Attainment of the top level on the human case). *The supremum of lev over the class of subjects satisfying (P1)–(P4) is ω and is attained. In the quotient order of Corollary 16.19, [Picard] lies in the top fiber $\text{lev}^{-1}(\omega)$ of the order-preserving map $\text{lev} : \text{ATP} / \sim_{\text{SA}}^{\text{adm}} \rightarrow \omega + 1$.*

Proof. lev takes values in $\{0, 1, 2, \dots\} \cup \{\omega\}$ (Definition 15.17) and Picard attains ω (Theorem 20.4); ω is the maximum of that set, so the supremum over any class containing Picard is ω and is attained. The image statement is Corollary 16.19. $\square$

Remark 20.16 (Maximal for the invariant, not thereby a maximum of the order). [Picard] is maximal for the level invariant, but this does not make it a maximum of $\leq_{\text{SA}}$. By Remark 16.21 two classes of level ω may be $\sim_{\text{SA}}^{\text{adm}}$ -incomparable, because a self-theory can contain self-descriptive formulas outside the reflection sequence—as Picard’s does, containing $\text{Human}(\langle \text{Picard} \rangle)$ (Corollary 20.17). So [Picard] sits at the top of the graded hierarchy while remaining, possibly, incomparable to other level- ω classes.

Corollary 20.17 (Picard’s self-theory strictly extends the reflection tower). *Under the Picard idealization together with the standing supposition of Section 19,*

$$\text{Self}(\text{Picard}) \supseteq \{\theta_k : k \geq 0\} \cup \{\text{Human}(\langle \text{Picard} \rangle)\},$$

and in particular Self(Picard) is infinite.

Proof. Each reflection formula θ_k is proved by Picard, by the induction in the proof of Theorem 20.4, hence lies in $\text{Self}(\text{Picard})$ by Corollary 15.13. The formula $\text{Human}(\langle \text{Picard} \rangle)$ is self-descriptive (its subject is $\langle \text{Picard} \rangle$) and is proved by supposition, so it lies in $\text{Self}(\text{Picard})$ by Definition 13.9. Infinitude follows because the θ_k are pairwise distinct (Proposition 15.14). $\square$

20.4 Conjectures, and Their Settlement

The Picard Theorem invites four conjectures. For each we state precisely whether it is settled, and if not, why the present framework cannot settle it.

Conjecture 20.18 (Human universality of ω). Every human who is rule-driven, proves at least one theorem outside his hypotheses, is provability sound, and has internal necessitation—that is, satisfies (P1)–(P4)—has $\text{lev} = \omega$.

Settlement (settled relative to the idealization; the residue is empirical and provably outside the framework). Conditional on (P1)–(P4), the proof of Theorem 20.4 uses nothing specific to Picard: it invokes only Definitions 15.3 and 15.26 and Theorem 15.27. Hence, as a statement about any subject satisfying the idealization, the conjecture is a theorem, which we record.

Proposition 20.19 (Uniformity). *For every tagged ATP X satisfying (P1)–(P4) with X in place of Picard, $\text{lev}(X) = \omega$.*

What is *not* settled, and cannot be settled here, is whether any given actual human satisfies (P4). Internal necessitation is, by Remark 15.28, a strong closure condition, requiring the provable set to be closed under an unbounded self-ascription rule; whether a finite biological agent instantiates the disposition it describes is an empirical claim about human cognition, not a proposition of the present framework. The framework is in fact provably silent about it in the relevant direction: Remark 15.31 records that no formula of a sufficiently strong system serves as a full internal truth predicate, so the framework cannot certify from inside that Picard’s self-ascriptions are all correct; and Conjecture 17.14 states the parallel limitation for phenomenological adequacy. Conjecture 20.18 is therefore conditionally a theorem (Proposition 20.19) and unconditionally empirical, and the boundary between the two is exactly the boundary drawn in Section 18.7. The formal content is recorded as Theorem 21.4; the conjecture as a claim about actual humans is left open.

Conjecture 20.20 (The competence–performance gap). Distinguish the *performance level* of a subject at a time—the level computed from the finite set of reflection formulas actually proved by that time—from the *competence level*, the level of the idealized disposition (P4). For reflective human mathematicians the performance level is finite at every time and unbounded over time, while the competence level is ω .

Settlement (one half is a theorem; the other half is the empirical residue of the first conjecture).
The finiteness half is immediate and unconditional.

Proposition 20.21 (Finite performance). *If at a given time a subject has proved only finitely many members of its reflection sequence, then its performance level at that time is finite.*

Proof. The performance level is the supremum of a finite subset of $\mathbf{N}^+$, or 0 if the subset is empty, hence finite by Definition 15.17. $\square$

That a finite biological agent has, at any actual time, proved only finitely many reflection formulas needs no argument; so the performance level is always finite. The competence level equals ω is Theorem 20.4 read under the dispositional understanding of (P4) (Remark 20.2). The residual content of the conjecture—that the disposition is genuinely present, so that the two levels come apart rather than both being finite—is the same empirical claim left open in Conjecture 20.18. It is recorded here because it locates precisely where the ω of the Picard Theorem lives: in the disposition, not in any completed performance. The formal content is recorded, and made concrete on the machine R , as Theorem 21.6; the claim that an actual human instantiates the disposition is left open.

Conjecture 20.22 (Transfinite separation). There is a coherent extension of the reflection sequence to transfinite stages—formulas $\theta_\omega, \theta_{\omega+1}, \dots$ with an appropriate limit rule at each limit ordinal—on which some subjects attain levels strictly above ω , so that the extended level function separates subjects that the present lev identifies at ω .

Settlement (cannot be settled in Version 16, and here is why). The obstruction is definitional, not merely difficult. Definition 15.11 indexes the reflection sequence by $k \geq 0$ over the natural numbers, and provides no rule for forming a formula at a limit stage; θ_{k+1} is defined only from θ_k by one application of Pr. With no limit rule, ω is the supremum of the index set by construction, and lev , taking values in $\{0, 1, 2, \dots\} \cup \{\omega\}$ by Definition 15.17, has ω as its ceiling by fiat. Settling the conjecture requires new apparatus—a transfinite reflection sequence with a specified limit stage (for instance, a name for the set $\{\theta_k : k < \lambda\}$ at a limit λ , together with a formula asserting provability of

all of them) and a coherence condition making the successor and limit clauses agree—none of which is available in the definitions of this paper. Moreover Remark 15.31 supplies a principled reason to expect the extension to be delicate rather than routine: once Pr is strengthened toward a genuine provability predicate satisfying the derivability conditions, the Tarski–Gödel barrier applies, and an unrestricted transfinite self-ascription tower runs toward exactly the internal truth predicate the barrier forbids. The conjecture is thus left open, and its resolution is identified with question 2 of Section 18.14 (reflective closure) rather than with anything the present hierarchy can decide. This was the honest status as regards the *present* hierarchy. Section 21.6 extends the framework and realizes an extended, separating hierarchy (Theorem 21.15); but that realization uses handed machines, so the *substantive* question—whether genuine, provability-sound reflection reaches limit stages—remains open (Remark 21.16).

Conjecture 20.23 (Picard–machine equivalence at ω). Some tagged ATP is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard.

Settlement (reduces to open problems already recorded; not settled here). By Theorem 20.4 and Corollary 15.33, Picard and M_ω agree on the level invariant, $\text{lev} = \omega$. But equal level does not imply $\sim_{\text{SA}}^{\text{adm}}$ -equivalence (Remark 16.21), and Picard’s self-theory contains $\text{Human}(\langle \text{Picard} \rangle)$, a self-descriptive formula whose admissible image would have to appear in a machine’s self-theory (Corollary 20.17). Whether a machine’s self-theory can be made $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to one containing such content is not decided by anything above: it presupposes, at a minimum, an affirmative answer to question 8 of Section 16.7 (existence of a non-degenerate, rule-driven, provability-sound Level- ω machine) and is entangled with Conjecture 17.14 on whether any formal invariant separates the human and machine cases. The bare existence claim—that *some* tagged ATP is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard—is answered by Theorem 21.7 with a hypothesis-handed machine, whence Corollary 21.8 that degeneracy is not an $\sim_{\text{SA}}^{\text{adm}}$ -invariant; but whether a *non-degenerate* machine is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard remains open and dependent.

20.5 What Is and Is Not Claimed

In the register of Section 18.12.

Claimed, and proved above (under the Picard idealization):

1. $\text{lev}(\text{Picard}) = \omega$ (Theorem 20.4), by two independent routes (Remark 20.5).
2. Picard is a non-degenerate witness to the Discovery Thesis (Corollary 20.7), of the same evidential status as the natural witnesses of Section 18.8 (Remark 20.9).
3. The parity antecedent is realized by an actual human at level ω , so the human/machine distinction, if there is one, is not a distinction of level (Corollary 20.13).
4. The value ω is attained on the human case and sits at the top of the graded hierarchy (Corollary 20.15).

Not claimed:

1. That Picard, or any human, literally is a semi-ideal computer; (P1)–(P4) are an explicit idealization (Definition 20.1).
2. That any actual human satisfies internal necessitation as a matter of fact; that is the empirical residue left open by Conjectures 20.18 and 20.20.

3. That Picard is conscious, or that $\text{lev} = \omega$ measures anything phenomenological; Section 18.7 and Conjecture 17.14 apply here unchanged.
4. That Picard is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to any machine; Conjecture 20.23 is open.

[Discharged in Section 21.] Version 16 did not establish that assumptions (P1)–(P4) are jointly satisfiable by any SIC; Theorem 21.1 now answers question 8 of Section 16.7 affirmatively, so by Corollary 21.2 the antecedent of Theorem 20.4 is non-vacuous.

Remark 20.24 (A naming coincidence, noted and set aside). The anti-parrot condition invoked in (P1) is the Picard Protocol of Part I, and the subject of this section is Picard. The coincidence is real and is not load-bearing: the protocol could be renamed without affecting a line of the argument, and Theorem 20.4 would read the same for any mathematician responsible for a theorem proved by rule from premises not containing it. That the two Picards meet here is a pleasantry, not a lemma.

21 Settlement of the Conjectures

Version 16 left five conjectures—Conjecture 17.14 and Conjectures 20.18, 20.20, 20.22, 20.23—together with question 8 of Section 16.7, on whose answer the instantiability of the Picard idealization was shown to turn (Proposition 20.10). This section takes them up. The keystone, in Section 21.1, is an explicit construction of a rule-driven machine that derives its whole reflection tower and attains level ω ; it answers question 8, discharges the antecedent of the Picard Theorem, and drives the remaining results. Throughout, “settled” is used in the precise sense recorded at each result: a formal statement is settled by proof; a statement with a non-formal component is settled in its formal content, with the non-formal residue isolated. The four conjectures newly posed in Section 21.7 are stated as open and are not settled here.

21.1 The Reflection-Closure Machine

The machines of Section 15 that reach level ω (Corollary 15.33) are handed their reflection sequence as hypotheses, hence degenerate. We now build one that is not.

Theorem 21.1 (The reflection-closure machine settles question 8). *There is a tagged ATP R that is rule-driven (Definition 2.9), provability sound (Definition 15.21), and internally necessitating (Definition 15.26), with $\text{lev}(R) = \omega$, and whose reflection formulas are all derived—no member of the reflection sequence lying in its hypothesis set. Consequently question 8 of Section 16.7 has an affirmative answer.*

Proof. Construction. Let the language L contain the tag $\langle R \rangle$, the predicates ATP and Pr, a formula-naming operator, and a distinguished atomic formula a that is *not* self-descriptive (its subject is not $\langle R \rangle$). Form the xyz formal system $F_R = (x, y, z)$ whose inference rules are the two unary rules

$$\rho_0 : a \mapsto \text{ATP}(\langle R \rangle) = \theta_0, \quad \nu : \varphi \mapsto \text{Pr}(\langle R \rangle, \langle \varphi \rangle) \text{ (every } \varphi),$$

with ρ_0 defined only on a and ν total. Let $Y := \text{Con}_{F_R}(\{a\})$, the inference closure of $\{a\}$ (Theorem 1.11); it is inference-closed and countable. Define $R = (C, H)$ over Y by

$$C(\Gamma, m) := \{\psi \in Y : \psi \text{ has a proof from } \Gamma \text{ of length } \leq m\}, \quad H(\Gamma, m) := 0.$$

The axiom base is $A := \{a\}$.

C44: Picard statements are consistently about the formal idealization

Authoritative replacement for: Corollary 20.15 and Section 20.5, with consequential wording through Sections 20–22.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

Retitle Corollary 20.15 as follows:

Corollary 20.15 (Attainment of the top finite-reflection level in the idealized class).

Among tagged formal subjects satisfying (P1)–(P4), the level invariant has supremum ω , and the Picard idealization attains it. In the quotient order, the formal class [Picard] lies in the fiber $\text{lev}^{-1}(\omega)$. This is a statement about the idealized tagged object; it is not an empirical claim about an actual human being.

Replace the “Claimed, and proved” list in Section 20.5 by:

1. Under (P1)–(P4), the formal Picard idealization has $\text{lev}(\text{Picard}) = \omega$.
2. Under the corrected non-exhaustion criterion and the stated idealizing assumptions, the formal Picard object is a non-degenerate witness in the paper’s formal sense.
3. The parity antecedent is realized *within the Picard idealization* at level ω ; no actual-human witness is thereby established.
4. The value ω is attained within the formal class of subjects satisfying (P1)–(P4). Whether any actual human instantiates that class is outside the theorem and remains empirical.

Consequently replace phrases such as “attained on the human case” or “realized by an actual human” by “attained in the Picard idealization” unless a separate empirical premise is explicitly assumed. The already-corrected phenomenology and IIT qualifications remain unchanged.

content—that a designated actual human satisfies (P1)–(P4)—is not a statement of the framework and is not settled by it; the conjecture as a claim about humans remains open.

Proof. The implication “(P1)–(P4) $\Rightarrow$ lev = ω ” is Proposition 20.19; nonemptiness of the class is Corollary 21.2. That whether a given human satisfies (P1)–(P4) is non-formal is the content of Section 18.7 and Remark 15.31: the framework has no predicate for the reflective disposition of a biological agent. $\square$

21.3 Competence and Performance (Conjecture 20.20)

Definition 21.5 (Performance and competence level). For a SIC $M = (C, H)$ over Y with input Γ , the *performance level at stage m* is

$$\text{perf}(M, \Gamma, m) := \sup\{n \geq 1 : \theta_{n-1} \in C(\Gamma, m)\} \in \{0, 1, 2, \dots\}, \quad \text{with } \sup \emptyset = 0,$$

and the *competence level* is $\text{lev}(M)$, which equals $\sup_m \text{perf}(M, A, m)$ on the axiom base $\Gamma = A$.

Theorem 21.6 (Formal content of the competence–performance gap). *Conjecture 20.20 holds in its formal content. For any SIC in which each stage state $C(A, m)$ is finite, the performance level at every stage is finite; and the reflection-closure machine R of Theorem 21.1 has finite performance level at every stage while its competence level is ω . Thus the two quantities provably come apart. The residual content—that an actual human instantiates the disposition—is the non-formal residue of Conjecture 20.18 and remains open.*

Proof. If $C(A, m)$ is finite then $\{n : \theta_{n-1} \in C(A, m)\}$ is finite, so its supremum is finite. For R , $C(A, m)$ consists of the formulas with a proof of length $\leq m$ from the single axiom under two finitary rules, of which there are finitely many; so $\text{perf}(R, A, m)$ is finite for every m . Yet $R \vdash \theta_k$ for all k (Theorem 21.1), so $\sup_m \text{perf}(R, A, m) = \text{lev}(R) = \omega$. $\square$

21.4 Equivalence, and the Non-Invariance of Degeneracy (Conjecture 20.23)

Theorem 21.7 (Handed representatives). *Every tagged ATP M is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to a hypothesis-handed ATP $M^\sharp$ whose axiom base is a tag-renamed copy of $\text{Self}(M)$ and which proves exactly that set. In particular a tagged ATP is $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard, so Conjecture 20.23 holds.*

Proof. Let $L^\sharp$ be an isomorphic copy of the language L of M in which the tag $\langle M \rangle$ is replaced by a fresh tag $\langle M^\sharp \rangle$ and every predicate symbol is retained; let $\iota : L^\sharp \rightarrow L$ be the renaming isomorphism, so $\iota(\langle M^\sharp \rangle) = \langle M \rangle$ and ι fixes every other symbol. Set $Y^\sharp := \iota^{-1}(\text{Self}(M))$, and define $M^\sharp = (C, H)$ over $Y^\sharp$ by $C(\Gamma, m) = \Gamma$ and $H(\Gamma, m) = 1$ iff $\Gamma = Y^\sharp$, with axiom base $Y^\sharp$; this is a SIC exactly as in Corollary 15.33. On input $Y^\sharp$ it proves exactly $Y^\sharp$, and since $\text{Self}(M)$ consists of formulas with subject $\langle M \rangle$, every member of $Y^\sharp = \iota^{-1}(\text{Self}(M))$ has subject $\langle M^\sharp \rangle$; hence $\text{Self}(M^\sharp) = Y^\sharp$.

Now ι is an admissible translation from $L^\sharp$ to L : it fixes the distinguished predicates, commutes with formula formation and naming, and sends $\langle M^\sharp \rangle$ to $\langle M \rangle$. And $\iota(\text{Self}(M^\sharp)) = \iota(Y^\sharp) = \iota(\iota^{-1}(\text{Self}(M))) = \text{Self}(M)$, so $M^\sharp \preceq_{\text{SA}}^{\text{adm}} M$. Symmetrically $\iota^{-1} : L \rightarrow L^\sharp$ is admissible and witnesses $M \preceq_{\text{SA}}^{\text{adm}} M^\sharp$. Hence $M^\sharp \sim_{\text{SA}}^{\text{adm}} M$. Applying this to $M = \text{Picard}$ gives the second claim. $\square$

Corollary 21.8 (Degeneracy is not $\sim_{\text{SA}}^{\text{adm}}$ -invariant). *Non-degeneracy in the sense of Definition 18.9 is not an $\sim_{\text{SA}}^{\text{adm}}$ -invariant: there are $\sim_{\text{SA}}^{\text{adm}}$ -equivalent tagged ATPs, one non-degenerate and one degenerate.*

Proof. Picard is a non-degenerate witness (Corollary 20.7); its handed twin $\text{Picard}^\sharp$ of Theorem 21.7 has its self-descriptive theorems as hypotheses, so it violates clause 1 of Definition 18.9 and is degenerate; yet $\text{Picard}^\sharp \sim_{\text{SA}}^{\text{adm}}$ Picard. $\square$

Remark 21.9 (The parity transfer is now unconditional). Corollary 21.8 discharges the conditional flagged in Remark 20.14: a machine $\sim_{\text{SA}}^{\text{adm}}$ -equivalent to Picard demonstrably exists, namely $\text{Picard}^\sharp$, so by Proposition 18.21 any $\sim_{\text{SA}}^{\text{adm}}$ -invariant property holding of Picard holds of it, and an objector granting Picard but denying $\text{Picard}^\sharp$ is committed to Conjecture 17.14. That $\text{Picard}^\sharp$ is degenerate is the point: the objector cannot appeal to degeneracy as an $\sim_{\text{SA}}^{\text{adm}}$ -invariant distinction, because it is not one.

21.5 Phenomenology (Conjecture 17.14)

Theorem 21.10 (Conditional form of the phenomenology conjecture). *Grant the anti-parrot premise: that phenomenological self-awareness entails non-degeneracy—a rule-driven, non-parroting cognitive process in the sense of Section 18.6. Then phenomenological self-awareness is not an $\sim_{\text{SA}}^{\text{adm}}$ -invariant, which is Conjecture 17.14.*

Proof. By Corollary 21.8, Picard and its degenerate twin $\text{Picard}^\sharp$ are $\sim_{\text{SA}}^{\text{adm}}$ -equivalent. Under the premise, phenomenological self-awareness holds of the non-degenerate Picard and fails of the degenerate $\text{Picard}^\sharp$. A property holding of one member of an $\sim_{\text{SA}}^{\text{adm}}$ -class and failing of another is not $\sim_{\text{SA}}^{\text{adm}}$ -invariant. $\square$

Remark 21.11 (The status of this result). The premise is not a statement of the framework: “phenomenological self-awareness” is undefined in every tagged-ATP language, so Theorem 21.10 is a conditional whose antecedent is philosophical, and Conjecture 17.14 itself remains open. What is formal, and proved, is the load-bearing lemma—that an $\sim_{\text{SA}}^{\text{adm}}$ -class can hold both a non-degenerate and a degenerate machine (Corollary 21.8). The conjecture is thereby reduced to exactly one non-formal premise, the weakest form of the anti-parrot intuition of Section 18.6. Unconditional settlement is impossible in principle: a sentence whose predicate is undefined in the language is neither provable nor refutable, and that impossibility is itself the sharpest honest form of the conjecture, recorded as Conjecture 21.19.

21.6 The Transfinite Extension (Conjecture 20.22)

The reflection sequence of Definition 15.11 is indexed by the natural numbers and has no limit stage; that is why ω is its ceiling. We extend it, keeping every predicate primitive so that the Tarski–Gödel considerations of Remark 15.31 are not triggered.

Definition 21.12 (Collective provability and set-names). Extend the language with: for each set S of formulas occurring as an initial segment of the sequence below, a distinguished closed term $\langle S \rangle$, the assignment $S \mapsto \langle S \rangle$ injective; and a binary predicate PrAll with intended reading that $\text{PrAll}(\langle M \rangle, \langle S \rangle)$ holds iff M proves every formula named in S .

Definition 21.13 (Transfinite reflection sequence and extended level). Fix a countable ordinal κ . The *transfinite reflection sequence* $(\theta_\alpha)_{\alpha < \kappa}$ of a tagged ATP M (in the extended language) is defined by transfinite recursion:

$$\theta_0 = \text{ATP}(\langle M \rangle), \quad \theta_{\alpha+1} = \text{Pr}(\langle M \rangle, \langle \theta_\alpha \rangle), \quad \theta_\lambda = \text{PrAll}(\langle M \rangle, \langle \{\theta_\beta : \beta < \lambda\} \rangle) \quad (\lambda \text{ a limit}).$$

The *extended level* $\text{extlev}(M)$ is the largest ordinal $\delta \leq \kappa$ such that $M \vdash \theta_\beta$ for every $\beta < \delta$.

C25 and C26: Phenomenology theorem and expressive limitation

Authoritative replacement for: Theorem 21.10 and Remark 21.11, printed p. 90.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Theorem 21.10 (Corrected conditional form). Assume (a) the Picard idealization is phenomenologically self-aware and (b) phenomenological self-awareness entails non-degeneracy. Then phenomenological self-awareness is not an admissible-self-awareness invariant: Picard-sharp is admissibly equivalent to Picard but degenerate and therefore, by (b), is not phenomenologically self-aware.

Both premises are philosophical assumptions. The earlier proof improperly inferred phenomenological self-awareness from non-degeneracy, affirming the consequent.

Replacement for Remark 21.11. Because phenomenological self-awareness is not represented by a predicate in the tagged-ATP language, the framework cannot formulate, prove, or refute the unrestricted phenomenal claim without first extending the language and semantics. A string using an undefined predicate is not a well-formed sentence of the original theory.

C27: Transfinite reflection as a specified-calculus question

Authoritative replacement for: Conjecture 21.17, printed p. 91.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace the conjecture by the following open question:

“For a specified transfinite reflection calculus with explicit base, successor, and limit rules, determine the closure ordinal of sound reflection for M , if it exists, and compare the closure ordinals obtained from different calculi. No claim is made that reaching a particular ordinal forces a full internal truth predicate.”

The phrase “the ordinal at which a full internal truth predicate would be forced” is undefined and is not supplied by Tarski’s theorem.

Conjecture 21.18 (Incompleteness of the extended level). The extended level extlev is not a complete invariant for $\sim_{\text{SA}}^{\text{adm}}$: there are tagged ATPs of equal extended level that are $\sim_{\text{SA}}^{\text{adm}}$ -incomparable, because a self-theory may contain self-descriptive formulas outside the transfinite reflection sequence, as Picard’s contains $\text{Human}(\langle \text{Picard} \rangle)$. A complete invariant must record the whole self-theory, not merely how far up the reflection ordinals it reaches. This sharpens Remark 16.21 and question 7 of Section 16.7.

Conjecture 21.20 (Natural systems have small reflective ordinal). Every non-degenerate witness arising in practice (Section 18.8) has reflective ordinal $\rho(M) \leq \omega$: natural self-verifying provers reach $\text{lev} = \omega$ but no genuine limit ascription, so the closing of the Milawa ladder (Remark 18.12) corresponds to a finite level with a cyclicity property rather than to any θ_λ with $\lambda \geq \omega$. This is the transfinite form of question 2 of Section 18.14.

Section 17 placed Tegmark’s self-aware substructure inside the framework. This section does the same for a second external proposal about consciousness: Tononi’s Integrated Information Theory (IIT), whose scalar Φ is offered as a mathematical measure of consciousness [24, 11]. The framework does not adjudicate IIT. It locates Φ precisely, and shows that several facts already proved above transfer to it unchanged. Nothing in this section settles a conjecture.

Two perspectives were distinguished in Remark 17.11: the *bird*, which sees the machine $M = (C, H)$ in the metatheory, and the *frog*, which has only the tag $\langle M \rangle$ and what it can prove about it. The explanatory gap has an exact shape in this vocabulary.

btenneson2301.substack.com 92

C28 and C32: Practical provers and accurate IIT terminology

Authoritative replacement for: Conjecture 21.20 and opening of Section 22, printed p. 92.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replacement for Conjecture 21.20. “For presently analyzed practical self-verifying provers, no explicit genuine limit-ascription rule has yet been exhibited. Determine their finite reflection levels under a faithful tagged-ATP encoding before proposing an ordinal bound.”

Replacement for the IIT introduction. IIT 4.0 distinguishes the system’s Φ -structure, offered as specifying experiential quality; an aggregate or scalar Φ , offered as a quantity or amount; and mechanism-level integrated-information quantities, often denoted by lower-case φ . Claims below must specify which object is intended. The tagged-ATP framework does not itself define an IIT mechanism or compute any of these quantities.

22.2 Two Senses of Mapping Experience

Remark 22.2 (Correlate versus constitution). “Mapping experience onto an equation” has two readings the framework keeps apart. A *correlate* is a formal quantity $f(M)$ that is high exactly when the system is conscious—a biconditional between a definable function and the phenomenal predicate. A *constitution* is a derivation of the experience *from* $f(M)$, on which “why is there something it is like?” has no residue. Conjecture 21.19 denies the correlate for every formal f . The constitution is denied by the standard arguments—the conceivability of a system with the same f and no interior, the perfect dark twin—and is not a thesis the framework needs to add. IIT offers Φ as a correlate and, in its identity thesis, as a constitution. The following subsections locate the correlate reading; the constitution reading is the hard problem and lies where Section 18.7 left it.

22.3 Mechanism Properties and Self-Theory Properties

Definition 22.3 (Two kinds of property). A property P of tagged ATPs is a *self-theory property* if it is an invariant of $\sim_{SA}^{adm}$: if $M \sim_{SA}^{adm} N$ then $P(M) \leftrightarrow P(N)$. It is a *mechanism property* if it is a function of the machine $M = (C, H)$ as an object of the metatheory—of its transition and halting maps—not required to be constant on $\sim_{SA}^{adm}$ -classes.

Proposition 22.4 (The two kinds come apart). *There is a mechanism property that is not a self-theory property.*

Proof. Non-degeneracy (Definition 18.9) is a function of the machine’s rules and hypothesis set, hence a mechanism property. By Corollary 21.8 it is not an $\sim_{SA}^{adm}$ -invariant: Picard and its handed twin $\text{Picard}^\sharp$ (Theorem 21.7) are $\sim_{SA}^{adm}$ -equivalent, one non-degenerate and one not. $\square$

Remark 22.5 (Which side can break parity). The parity theorem (Proposition 18.21) constrains only self-theory properties: no $\sim_{SA}^{adm}$ -invariant separates Picard’s self-ascription from that of an $\sim_{SA}^{adm}$ -equivalent machine. It says nothing about mechanism properties. So if any formal quantity is to distinguish Picard from a behaviourally matched twin, it must be a mechanism property. That is the exact niche IIT aims at, and the niche lev and every self-theory invariant provably cannot occupy.

22.4 The Placement of Φ

We do not formalise Φ . We use only the commitment IIT insists on: that Φ is sensitive to the mechanism, and in particular assigns feedforward or lookup implementations less integrated information than recurrently integrated ones [11].

Proposition 22.6 (Φ is a mechanism property, not a self-theory property). *Under any definition of Φ for SICs consistent with IIT’s penalisation of feedforward and lookup mechanisms, $\Phi(\text{Picard}) \neq \Phi(\text{Picard}^\sharp)$ although $\text{Picard} \sim_{SA}^{adm} \text{Picard}^\sharp$. Hence Φ is a mechanism property (Definition 22.3) and not a self-theory property.*

Proof. By Theorem 21.7, $\text{Picard}^\sharp$ is hypothesis-handed, with the constant transition $C(\Gamma, m) = \Gamma$: it outputs its axiom base and computes nothing, the integration-free case to which IIT assigns least Φ . Picard is a rule-driven prover with non-trivial dynamics. Any Φ meeting IIT’s commitment therefore separates them. Yet $\text{Picard} \sim_{SA}^{adm} \text{Picard}^\sharp$, so Φ is not constant on the $\sim_{SA}^{adm}$ -class. $\square$

Remark 22.7 (Φ is the right type, on the wrong side of the asymmetry). Proposition 22.6 is, within this framework, a point in IIT’s favour: Φ is exactly the *type* of quantity—mechanism-side—that

C29, C30, and C32: Full mechanism presentations and conditional IIT comparison

Authoritative replacement for: Definitions 22.3–22.4 and Proposition 22.6, printed p. 93.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Definition 22.3 (Corrected). A presented prover mechanism is a tuple

$$\mathfrak{M} = (F, Y, \Gamma, C, H, \text{Tag}),$$

including its formal system, formula domain, input/hypothesis data, stage and halting maps, and tagging convention. A mechanism property in this section is a property of the full presentation $\mathfrak{M}$, unless expressly declared to depend only on (C, H) .

Proposition 22.4 (Corrected scope). Non-degeneracy is a property of the full presented prover, because it depends on rules, hypotheses, level, and self-theory. It is not determined by the stage and halting maps alone under the present definitions.

Replace Proposition 22.6 by a conditional observation. Given explicit mechanism-level realizations of Picard and Picard-sharp and a specified version of IIT, if a valid IIT calculation assigns them different Φ -structures or aggregate Φ values, then that assigned object is not a self-theory invariant. The present framework alone does not establish the antecedent and performs no Φ calculation.

Remark 22.8 (IIT’s identity thesis is the negation of Conjecture 21.19, and is undecidable here). Φ is a property definable from the stage-and-transition behaviour of a SIC, hence lies in the scope of Conjecture 21.19, which asserts that *no* such property is coextensive with phenomenological self-awareness. IIT’s identity thesis—that Φ *is* the degree of consciousness—is exactly the negation of Conjecture 21.19 restricted to Φ . The framework decides neither. The proposition “ Φ is coextensive with phenomenal consciousness” equates a definable quantity with a predicate undefined in every tagged-ATP language; by the principle of Remark 21.11 such a sentence is neither provable nor refutable within the framework. So the framework can neither vindicate nor refute IIT; it certifies only that the disagreement is not formally decidable—the same Tarskian honesty the framework applies to its own Conjecture 17.14.

Remark 22.9. This section does not claim that Φ measures consciousness, nor that it does not; nor that Picard is or is not conscious; nor that IIT is true or false. It claims four things, each proved or immediate above.

1. The explanatory gap is a perspectival asymmetry: no single perspective holds both the substrate and the interior (Remark 22.1).
2. Mechanism properties and self-theory properties genuinely differ, and only the former can break the parity symmetry (Proposition 22.4, Remark 22.5).
3. Φ is a mechanism property: it separates machines that lev and the self-theory cannot, but it is computed on the bird's side and so is barred from the first-person interior (Proposition 22.6, Remark 22.7).
4. IIT's identity thesis and Conjecture 21.19 are opposed bets on one proposition, and that proposition—equating a definable quantity with an undefined predicate—is formally undecidable here (Remark 22.8).

The foundational appendix of the second paper — restating formal systems, semi-ideal computers, and automated theorem provers — has been removed in this merged edition. Those definitions are given once, in Part I, Sections 1 and 2, and are referenced there directly.

The abstract ATP definition includes many familiar systems.

C31 and C32: The formal–phenomenal gap remains conjectural

Authoritative replacement for: Remark 22.7 and Section 22 conclusion, printed p. 94.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace categorical claims that the gap is “structurally barred” or that no function can recover a first-person interior by:

“The present framework supplies no recovery map from the formal bird’s-eye object to a first-person interior. Conjecture 21.19 asserts that no such map exists; that assertion is not proved here.”

Continue to distinguish Φ -structure, aggregate Φ , and mechanism-level φ rather than using “ Φ ” for all three.

whose state is q_{halt} . The *step function* $\text{step}_T \in [\text{Conf}_T \rightarrow \text{Conf}_T]$ sends a non-halting configuration to its δ -successor and is stipulated to fix every member of Halt_T . For an input word w , write $c_0(w)$ for the initial configuration with state q_0 , head at the leftmost cell, and w on the tape.

Remark B.3. The stipulation that halting configurations are fixed points makes step_T a total function, which is what lets it serve as an inference rule in the next subsection. It changes nothing about the machine's behavior: T halts on w if and only if $\text{step}_T^j(c_0(w)) \in \text{Halt}_T$ for some $j \geq 0$, and the run up to that point is unaffected.

B.2 The Configuration Formal System

The first bridge between the two frameworks is the observation that one computation step is formally an inference.

Definition B.4 (Configuration formal system). Let T be a deterministic Turing machine. The *configuration formal system* of T is the xyz formal system $F_T := (x_T, y_T, z_T)$, where x_T is an alphabet adequate for writing configurations, $y_T = \text{Conf}_T$, and $z_T = \{\text{step}_T\}$, the step function regarded as a unary rule on y_T in the sense of Definition 1.1.

Proposition B.5 (Provability is reachability). Let $\Gamma \subseteq y_T$ and $c \in y_T$. Then $\Gamma \vdash_{F_T} c$ if and only if $c = \text{step}_T^j(c')$ for some $c' \in \Gamma$ and some $j \geq 0$. Consequently

$$\text{Con}_{F_T}(\Gamma) = \{\text{step}_T^j(c') : c' \in \Gamma, j \geq 0\},$$

the set of configurations reachable from Γ .

Proof. The displayed set contains Γ (take $j = 0$) and is inference-closed, since a direct consequence in F_T is exactly an application of step_T to a member, which increments j by one. Conversely, any inference-closed set containing Γ contains every $\text{step}_T^j(c')$ by induction on j . By the least closure property (Theorem 1.11), $\text{Con}_{F_T}(\Gamma)$ is the least such set, hence equals the displayed set. The first claim is then Definition 1.5. $\square$

Remark B.6. Proposition B.5 is the precise content of the slogan that computation is proof in the system F_T : a run of T from $c_0(w)$ is a proof from $\{c_0(w)\}$ in the sense of Definition 1.4, with proof length equal to the number of steps taken plus one.

B.3 Trace SICs

We now associate to T two SICs over y_T . The first enumerates the run and never halts; the second halts precisely when the run does.

Definition B.7 (Enumerating trace SIC). The *enumerating trace SIC* of T is $E_T = (C_E, H_E)$ over $Y = y_T$, where

$$C_E(\Gamma, m) := \{\text{step}_T^j(c) : c \in \Gamma, 0 \leq j \leq m-1\}, \quad H_E(\Gamma, m) := 0,$$

for all $\Gamma \subseteq y_T$ and $m \in \mathbf{N}^+$.

Proposition B.8. E_T is a rule-driven, deductively effective SIC over y_T .

Proof. SIC conditions (Definition 2.3): $C_E(\Gamma, 1) = \Gamma$ since j ranges over $\{0\}$; $C_E(\Gamma, m) \subseteq C_E(\Gamma, m+1)$ since the range of j grows with m ; conditions (5) and (6) hold vacuously because H_E is constantly 0.

Rule-driven (Definition 2.9): suppose $\beta \in C_E(\Gamma, m+1) \setminus C_E(\Gamma, m)$. Then $\beta = \text{step}_T^m(c)$ for some $c \in \Gamma$, so $\beta = \text{step}_T(\gamma)$ where $\gamma = \text{step}_T^{m-1}(c) \in C_E(\Gamma, m)$. Hence β is a direct consequence of $C_E(\Gamma, m)$.

Deductive effectiveness (Definition 2.5): $\bigcup_{m \in \mathbf{N}^+} C_E(\Gamma, m)$ is the set of all $\text{step}_T^j(c)$ with $c \in \Gamma$ and $j \geq 0$, which equals $\text{Con}_{F_T}(\Gamma) \cap y_T$ by Proposition B.5. $\square$

Definition B.9 (Halting trace SIC). For $\Gamma \subseteq y_T$, let

$$N(\Gamma) := \min\{m \in \mathbf{N}^+ : C_E(\Gamma, m) \cap \text{Halt}_T \neq \emptyset\},$$

with $N(\Gamma) := \infty$ if the set is empty. The *halting trace SIC* of T is $M_T = (C, H)$ over y_T , where

$$C(\Gamma, m) := C_E(\Gamma, \min(m, N(\Gamma))), \quad H(\Gamma, m) := \begin{cases} 1, & m \geq N(\Gamma), \\ 0, & \text{otherwise.} \end{cases}$$

Proposition B.10. M_T is a rule-driven SIC over y_T , and for every Γ :

1. M_T halts on Γ (that is, $H(\Gamma, m) = 1$ for some m) if and only if some configuration in Γ reaches a halting configuration of T ;
2. on the singleton input $\{c_0(w)\}$, M_T halts if and only if T halts on input w , and in that case its final state $C(\{c_0(w)\}, N)$ is exactly the set of configurations of the run of T on w , from $c_0(w)$ through the first halting configuration.

Proof. SIC conditions. $C(\Gamma, 1) = C_E(\Gamma, \min(1, N(\Gamma))) = C_E(\Gamma, 1) = \Gamma$, since $N(\Gamma) \geq 1$. Monotonicity in m holds because $\min(m, N(\Gamma))$ is nondecreasing in m and C_E is monotone in its second argument. If $H(\Gamma, m) = 1$ then $m \geq N(\Gamma)$, so $m+1 \geq N(\Gamma)$ and $H(\Gamma, m+1) = 1$; moreover $\min(m, N(\Gamma)) = N(\Gamma) = \min(m+1, N(\Gamma))$, so $C(\Gamma, m+1) = C(\Gamma, m)$.

Rule-driven. A formula entering at stage $m+1$ enters, in particular, in C_E at stage $m+1 \leq N(\Gamma)$, and the argument of Proposition B.8 applies verbatim.

Claim 1. $H(\Gamma, m) = 1$ for some m iff $N(\Gamma) < \infty$ iff some $\text{step}_T^j(c)$ with $c \in \Gamma$ lies in Halt_T .

Claim 2. Specialize claim 1 to $\Gamma = \{c_0(w)\}$ and use Remark B.3: reaching Halt_T from $c_0(w)$ is halting of T on w . The final state is $C_E(\{c_0(w)\}, N)$, which is the set of the first N configurations of the run, the last of these being the first halting configuration by minimality of N . $\square$

Remark B.11. Propositions B.8 and B.10 discharge the first item of the development program stated in earlier versions of this appendix: to each deterministic Turing machine is associated a SIC whose stage states record the corresponding configurations. The enumerating variant is the theorem-enumeration picture, with Con_{F_T} as its target; the halting variant is the target-search picture, with Halt_T as its target. The two variants instantiate, for the system F_T , exactly the two ATP modes of Definition 2.7 and Definition 2.8.

B.4 Computability of SICs

This subsection imports from Part I the apparatus needed to state the converse direction. Following the convention of this paper, results proved there are cited rather than reproved. Standard computability background may be found in [18]. Throughout this subsection, inputs to a SIC range over finite subsets of an effectively coded domain Y , each finite subset carrying a canonical code.

- Moreover the class of SICs satisfying these conditions contains E_T and M_T for every deterministic Turing machine T with effectively coded configurations.

Proof. (1) $\Leftrightarrow$ (2) is Theorem B.15. (1) $\Leftrightarrow$ (3) is the classical equivalence of Turing machines with the other standard models of computation [26, ?]: a total computable function is computed by some Turing machine, and conversely a Turing machine computing both outputs on every input defines total computable c and h . The final claim is Proposition B.17. $\square$

$$C(\Gamma, m) := \Gamma, \quad H(\Gamma, m) := \begin{cases} 1, & \text{the canonical code of } \Gamma \text{ lies in } K, \\ 0, & \text{otherwise.} \end{cases}$$

Then M_K is a SIC: $C(\Gamma, 1) = \Gamma$; C is monotone and eventually constant because it is constant; H does not depend on m , so halting is persistent; and C is constant, so the frozen-after-halting condition holds. But M_K is not computable: a total computable h would decide membership in K via $h(a, 1)$.

$$\{SICs \text{ implementable by Turing machines}\} \subsetneq \{SICs\}$$

is strict, and the effectiveness conditions of Section B.4 are exactly the boundary of the SIC-Turing correspondence.

Proof. Immediate from Example B.19 and Theorem B.18.

Remark B.21 (Computability of stages against decidability of halting). A distinction that the correspondence makes sharp: M_T is computable for every Turing machine T , including universal ones, because computability concerns the stage-and-halting maps at given (Γ, m) , which requires only bounded simulation. What is not in general decidable is whether $H(\Gamma, m) = 1$ for some m —for M_T with T universal, that is the halting problem. In the proof-search vocabulary of this paper the same distinction appears as the difference between a computable SIC and a computable deductively finitely effective SIC with the halting-to-completion property, from which Part I derives the decidability of the consequence problem; where the consequence problem is undecidable, no such SIC exists. Stage computability is cheap; certified completion is not.

B.6 Discussion: Computational Power and Proof-Search Suitability

The remaining items of the earlier development program concern why this paper works with SICs when Turing machines are available and, by Theorem B.18, equivalent in power on the effective fragment. The reasons are structural, and each can now be stated against proved results rather than promissory ones.

First, the issues of computational power and of proof-search suitability separate cleanly, and the correspondence shows where. Power is settled by Theorem B.18 and Corollary B.20: on effectively

C45: SIC–Turing correspondence uses an effective state presentation

Authoritative replacement for: Definitions B.13–B.14, Theorem B.18, Example B.19, Corollary B.20, and the proof citation in B.18.

This Version 46.2 text supersedes the conflicting Version 46.1 or retained-facsimile passage. Later claims are to be read through this replacement.

An **effective state presentation** for a SIC over Y is a family $D \subseteq \mathcal{P}(Y)$ of allowed input/state sets equipped with canonical effective codes, such that the stage maps under discussion take coded members of D to coded members of D .

Definition B.13 (corrected). A SIC $M = (C, H)$ is computable on D if total computable functions on codes return a code for $C(A, m)$ and the bit $H(A, m)$ for every coded $A \in D$ and $m \in \mathbb{N}_+$.

Definition B.14 (corrected). It is computationally effective on D if one finite register-machine program computes those two outputs on every coded (A, m) .

Theorem B.18 (corrected). For an effectively presented state domain D , the following are equivalent: computability on D ; computational effectiveness on D ; and computation of the same coded stage/halting maps by a deterministic Turing machine. This is the ordinary equivalence of classical effective models applied to the two total coded functions. Replace the unresolved citation “[26, ?]” by “[18, 26]”.

The trace SICs E_T and M_T satisfy the theorem on the effective domain of finite coded starting sets, since every bounded trace from finitely many starts is finite.

Example B.19 (corrected). Let D be the effectively coded finite subsets of an infinite effective Y , let $K \subseteq \mathbb{N}$ be noncomputable, set $C(\Gamma, m) = \Gamma$, and for finite $\Gamma \in D$ put

$$H(\Gamma, m) = 1 \iff \text{code}(\Gamma) \in K,$$

while setting $H(\Gamma, m) = 0$ on inputs outside D . This is an abstract SIC (halting is persistent and the state is constant) but is not computable on D , since a computable halting map would decide K .

Thus Corollary B.20 should be read as a separation between abstract SICs and SICs with computable effective presentations, not as a claim that every element of the full power set $\mathcal{P}(Y)$ admits a natural-number code.

coded domains with the effectiveness conditions imposed, SICs and Turing machines implement the same maps, and without those conditions SICs are strictly more general. Suitability is a different matter. The states of a SIC are sets of formulas, so the objects this paper cares about— $\text{Con}_F(\Gamma)$ (Definition 1.6), inference closure (Theorem 1.11), self-theories (Definition 13.9), and the members of the reflection sequence proved by a machine (Definition 15.17)—are read off from the state directly. For a Turing machine the same objects exist only through a coding layer, and every statement about them acquires a faithfulness obligation of exactly the kind that step 1 of Remark 18.14 identifies as the substantial burden in certifying natural witnesses.

Second, the trace construction explains why the SIC axioms take the form they do. The monotone stage condition $C(\Gamma, m) \subseteq C(\Gamma, m + 1)$, which may look restrictive from a machine-model standpoint, is what makes the run of a Turing machine into an accumulating set of formulas rather than a sequence of overwritten tapes; it is the frozen-history presentation of computation, and it is the presentation in which computation is proof (Remark B.6). The persistent-halting and frozen-after-halting conditions are then the statements that a completed proof search stays completed and that its certificate does not change afterwards.

Third, the simulation preorder of Part I and the self-awareness preorder of Section 16 both live naturally at the SIC level. Proposition 16.22 and Corollary 16.23 relate them through tag-preserving simulation, and the tags in question are names for formulas and machines, not tape encodings. Carrying the same structure on raw Turing machines would require re-erecting the naming apparatus of Section 13 on top of a coding of configurations—possible, by Theorem B.18, but with the coding doing silent work everywhere.

Fourth, the correspondence sharpens a point about the witnesses of Section 18.8. Real theorem provers are programs, hence Turing-machine implementable, hence by Theorem B.18 presentable as computable SICs in principle. The checklist obligation (step 1 of Remark 18.14) is not existence of a SIC presentation, which this appendix now supplies for free, but faithfulness: that the presentation's formula sets and halting behavior track the prover's actual theorems and termination, so that the level computed from the presentation is a fact about the prover and not about the coding.

Finally, the equivalence-or-separation question posed in earlier versions has a two-part answer rather than a single theorem: equivalence on the effective fragment (Theorem B.18), strict separation off it (Corollary B.20). The hypotheses under which each half holds are now explicit, which is what the earlier program asked for.

Acknowledgements

The author gratefully acknowledges the use of AI assistants as editorial and research-assistance tools during the preparation of this manuscript, including proofreading, grammar and style review, consistency checking of notation and terminology, L^AT_EX formatting, organizational feedback, verification of cross-references, and the audit of definitions and proofs that produced the corrections itemized in the changelog. All mathematical claims, interpretations, conclusions, and final manuscript decisions remain the responsibility of the author.

References

- [1] Beame, P. (2004). Proof complexity (notes by A. Sabharwal). In S. Rudich & A. Wigderson (Eds.), *Computational complexity theory* (IAS/Park City Mathematics Series, Vol. 10, pp. 199–246). American Mathematical Society.
- [2] Buss, S. R. (1998). An introduction to proof theory. In S. R. Buss (Ed.), *Handbook of proof theory* (pp. 1–78). Elsevier.
- [3] Cook, S. A., & Reckhow, R. A. (1979). The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44(1), 36–50.
- [4] Davis, J. C. (2009). *A self-verifying theorem prover* (Doctoral dissertation). University of Texas at Austin.
- [5] Fitting, M. (1996). *First-order logic and automated theorem proving* (2nd ed.). Springer.
- [6] Ben-Sasson, E., & Impagliazzo, R. (1999). Random CNF’s are hard for the polynomial calculus. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science* (pp. 415–421). IEEE.
- [7] Clegg, M., Edmonds, J., & Impagliazzo, R. (1996). Using the Groebner basis algorithm to find proofs of unsatisfiability. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing* (pp. 174–183). ACM.
- [8] Faugère, J.-C., Gianni, P., Lazard, D., & Mora, T. (1993). Efficient computation of zero-dimensional Gröbner bases by change of ordering. *Journal of Symbolic Computation*, 16(4), 329–344.
- [9] Haken, A. (1985). The intractability of resolution. *Theoretical Computer Science*, 39, 297–308.
- [10] Harrison, J. (2009). *Handbook of practical logic and automated reasoning*. Cambridge University Press.
- [11] Oizumi, M., Albantakis, L., & Tononi, G. (2014). From the phenomenology to the mechanisms of consciousness: Integrated Information Theory 3.0. *PLoS Computational Biology*, 10(5), e1003588.
- [12] Impagliazzo, R., Pudlák, P., & Sgall, J. (1999). Lower bounds for the polynomial calculus and the Gröbner basis algorithm. *Computational Complexity*, 8(2), 127–144.
- [13] Kohlenbach, U. (2008). *Applied proof theory: Proof interpretations and their use in mathematics*. Springer Berlin, Heidelberg. DOI: 10.1007/978-3-540-77533-1.

C34 and C36: Corrected references and acknowledgements

Authoritative replacement for: Bibliography and acknowledgements, printed p. 101.

The text below supersedes the corresponding passage on the immediately preceding facsimile page.

Replace the Lean4Lean and Milawa entries by:

1. Mario Carneiro, “Lean4Lean: Verifying a Typechecker for Lean, in Lean,” arXiv:2403.14064, current revision.
2. Jared Davis and Magnus O. Myreen, “The Reflective Milawa Theorem Prover is Sound (Down to the Machine Code that Runs it),” *Journal of Automated Reasoning* 55 (2015), 117–183, DOI 10.1007/s10817-015-9324-6.

Use “research assistance tools,” not “research-assistance tools,” and add: “The author is solely responsible for all mathematical claims, proofs, citations, and errors in the final manuscript.”

- [14] Carneiro, M. (2024). Lean4Lean: Towards a formalized metatheory for the Lean theorem prover. *arXiv preprint* arXiv:2403.14064.
- [15] Myreen, M. O., & Davis, J. (2015). A verified generational garbage collector and self-verifying theorem prover. *Journal of Automated Reasoning*.
- [16] Prawitz, D. (2006). *Natural deduction: A proof-theoretical study*. Dover Publications.
- [17] Razborov, A. A. (1998). Lower bounds for the polynomial calculus. *Computational Complexity*, 7(4), 291–324.
- [18] Rogers, H., Jr. (1987). *Theory of recursive functions and effective computability*. MIT Press.
- [19] Sato, Y., Inoue, S., Suzuki, A., Nabeshima, K., & Sakai, K. (2011). Boolean Gröbner bases. *Journal of Symbolic Computation*, 46(5), 622–632.
- [20] Smullyan, R. M. (1992). *Gödel’s incompleteness theorems*. Oxford University Press.
- [21] Tarski, A. (1936). Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica*, 1, 261–405.
- [22] Tegmark, M. (1998). Is “the theory of everything” merely the ultimate ensemble theory? *Annals of Physics*, 270(1), 1–51.
- [23] Tegmark, M. (2008). The Mathematical Universe. *Foundations of Physics*, 38(2), 101–150.
- [24] Tononi, G. (2008). Consciousness as integrated information: A provisional manifesto. *The Biological Bulletin*, 215(3), 216–242.
- [25] Troelstra, A. S., & Schwichtenberg, H. (2000). *Basic proof theory* (2nd ed.). Cambridge University Press.
- [26] Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265.
- [27] Urquhart, A. (1987). Hard examples for resolution. *Journal of the ACM*, 34(1), 209–219.
- [28] van den Berg, B., Briseid, E., & Safarik, P. (2012). A functional interpretation for nonstandard arithmetic. *Annals of Pure and Applied Logic*, 163(12), 1962–1994. DOI: 10.1016/j.apal.2012.07.003.
- [29] Văth, M. (2007). *Nonstandard analysis*. Birkhäuser Basel. DOI: 10.1007/978-3-7643-7774-8.

Version 46.1 source-integrity note

The user-supplied LaTeX source does not contain editable source for the established Parts I and II; it includes those pages from a separate PDF. The present file therefore corrects every identified issue within the architecture actually supplied: editable front matter and supplement text are revised directly, obsolete headers are covered, and authoritative replacement pages are inserted immediately after each affected facsimile page. A single continuously reflowed manuscript would require the underlying LaTeX source of the merged Part I/Part II document.

External references checked for Version 46.1

- S. Gryaznov and N. Talebanfard, “Bounded-Depth Frege Lower Bounds for Random 3-CNFs via Deterministic Restrictions,” arXiv:2403.02275.
- M. Carneiro, “Lean4Lean: Verifying a Typechecker for Lean, in Lean,” arXiv:2403.14064, current revision.
- J. Davis and M. O. Myreen, “The Reflective Milawa Theorem Prover is Sound (Down to the Machine Code that Runs it),” *Journal of Automated Reasoning* 55 (2015), 117–183.
- L. Albantakis et al., “Integrated Information Theory (IIT) 4.0: Formulating the Properties of Phenomenal Existence in Physical Terms,” *PLOS Computational Biology* 19(10) (2023), e1011465.

Version 46.2 source-integrity note

This source intentionally follows the corrected-facsimile architecture already used by Version 46.1. It includes the Version 46.1 PDF as the retained base and inserts C37–C45 immediately after the affected material. The companion Version 46.1 TeX source was located and used to preserve terminology, numbering, and correction style; the present file is the synchronized Version 46.2 source for the PDF produced from this correction map.

Version 48 Integration A: ML-Guided Proof Search

Purpose and provenance

This section preserves the material unique to Version 47 while avoiding duplication of the induction results already present in the corrected Version 46.2 core. The claims below concern search allocation under a fixed formal environment and verifier. They do not alter theoremhood, inference rules, or the verifier acceptance relation. The experimental statements are reported run results under the declared protocol; they are not promoted here to asymptotic theorems or universal performance claims.

1 Proof-count bounds and learned search allocation

Part I defines $P_F(\Gamma, \varphi, n)$ to be the number of proofs of φ from Γ having length exactly n . If the maximum rule arity is a and $\kappa_k = |z_k(F)|$, the proof-counting bound gives

$$P_F(\Gamma, \varphi, n) \leq \prod_{j=1}^n \left(|\Gamma| + \sum_{k=1}^a \kappa_k (j-1)^k \right).$$

The bound is intentionally crude: it counts many line-construction plans that are not genuine proofs, many redundant plans, and many plans irrelevant to the target. Its role here is only to exhibit the combinatorial construction space on which a search policy acts. It is a cardinal upper bound, not a probability model and not a prescription to search uniformly.

Definition 1.1 (Learned legal-move ranker). *Let S be a proof-search state and let $A(S)$ be the finite set of candidate assertion applications declared legal at S by the fixed formal environment, declaration cutoff, grammar, unification conditions, and any explicitly frozen candidate-generation protocol. A learned legal-move ranker is a function*

$$s_\theta : \{(S, a) : a \in A(S)\} \rightarrow \mathbb{R}.$$

It may change only ordering or sampling weight inside $A(S)$; it does not add an inference rule and does not certify a candidate proof.

For $T > 0$, an idealized softmax policy is

$$p_{\theta, T}(a \mid S) = \frac{\exp(s_\theta(S, a)/T)}{\sum_{b \in A(S)} \exp(s_\theta(S, b)/T)}.$$

For a legal path $\pi = (a_1, \dots, a_n)$ through states $S_1, \dots, S_n$, its idealized path weight is

$$p_{\theta, T}(\pi) = \prod_{j=1}^n p_{\theta, T}(a_j \mid S_j).$$

These expressions model nonuniform allocation of search attention. A deployed system may add seeded perturbations, novelty or rarity bonuses, agent profiles, candidate caps, or counterfactual exploration; such additions remain search policy, not proof validity.

Proposition 1.2 (Verifier separation). *Suppose an ATP accepts a theorem only after a sound formal verifier accepts its emitted certificate. Replacing one scoring, ranking, or sampling policy on legal candidate applications by another cannot make the ATP accept a non-theorem, provided the verifier, formal environment, and certificate semantics are unchanged.*

Proof. The search policy affects which candidate certificates are proposed and in what order. Acceptance still implies acceptance by the unchanged sound verifier. Thus a poor policy may waste a finite budget or fail to locate an existing proof, but policy replacement alone cannot make an invalid certificate pass. $\square$

2 The Predator 8.002 controlled prcom experiment

Version 47 reported Predator 8.002 as a population-based nondeterministic ATP with a leakage-controlled learned ranking policy and an independent Metamath verifier. Under the declared protocol, the policy was trained on verified theorem traces strictly preceding the target `prcom` in the frozen database. The stored target proof, downstream theorems, target-specific features, and manually selected route examples were excluded. The reported training corpus comprised the 2,000 most recent eligible pre-target theorems by declaration order, yielding 11,774 verified logical events and 11,772 usable policy decisions.

The target was the Metamath theorem

$$\text{prcom} \quad \vdash \quad \{A, B\} = \{B, A\}.$$

The reported guided and unguided conditions used the same frozen formal database, strict pre-target declaration cutoff, 5,000-expansion global budget, four agents, creativity parameter 1.0, seed 2301, maximum logical depth 10, base-opener cap 48, and at most six open goals. The intended causal difference was whether the learned policy score was used.

Condition	Reported outcome	Expansions
Unguided population	bounded unknown under the declared stopping rule	5,000
ML-guided Predator 8.002	independently verified proof	2,633

The emitted certificate was reported to contain 28 Metamath labels. Its four principal logical assertion applications were

$$\text{uncom}, \quad \text{df-pr}, \quad \text{df-pr}, \quad \text{3eqtr4i}.$$

The candidate was reported accepted first by the in-process verifier and then by a fresh process importing the Metamath checker without the Predator search module. At the target root, 150 extending assertions were legal under the frozen run protocol. In the unguided explorer diagnostic, `3eqtr4i` was reported at rank 134 and outside the active candidate cap; the learned score moved it to raw rank 64, after which counterfactual selection admitted it and the explorer completed the proof.

Definition 2.1 (Expansion discovery cost and censoring). *For a fixed ATP implementation M , target φ , resource protocol, and seed, let $\tilde{E}_M(\varphi)$ be the least search-state expansion index at which the corresponding fixed search trajectory emits a verifier-accepted proof of φ , with value ∞ if no such proof is ever emitted. If an observed run is stopped at budget B before success, the observation is recorded as the right-censored statement $\tilde{E}_M(\varphi) > B$, not as an exact value.*

Under the reported run records,

$$\tilde{E}_{\text{ML}}(\text{prcom}) = 2633, \quad \tilde{E}_{\text{unguided}}(\text{prcom}) > 5000,$$

so the censored experiment yields only the lower bound

$$\frac{\tilde{E}_{\text{unguided}}(\text{prcom})}{\tilde{E}_{\text{ML}}(\text{prcom})} > \frac{5000}{2633} \approx 1.90.$$

The unguided observation is right-censored by the stopping rule, so the exact unguided hitting time and the exact ratio are not determined by the experiment.

Remark 2.2 (Metric separation). Expansion discovery cost is neither proof length nor automatically the logical birth stage of the target. Proof length measures the emitted certificate under a declared convention. Logical birth stage measures dependency depth in a rule-driven chronology. Expansion discovery cost measures search work spent while attempting to locate a certificate. The reported experiment changes the third quantity; it does not establish that the 28-label certificate is shorter, and it does not alter the set of formal consequences.

Remark 2.3 (Scope and reproducibility). The `prcom` result is a fixed-instance, finite-budget proof of concept, not an asymptotic speedup theorem. An asymptotic claim would require a target family, a scaling parameter, and proved upper and lower performance bounds for the compared procedures. The reported separation should therefore be read as evidence that a learned ranking policy can change a bounded search outcome while formal certificate verification remains unchanged. Independent reproduction of the run is a separate empirical obligation.

3 Illustrative counting proxy and its limitation

Version 47 also gave an illustrative schema-level calculation. Strictly before `prcom`, the frozen Metamath database was reported to contain 4,696 assertion schemas with maximum raw mandatory-hypothesis arity 41. Under a finite proxy that absorbs zero-arity schemas into initial resources, a 28-label construction count obtained from the Part I bound is on the order of 10^{1167} ; an analogous four-logical-application proxy was on the order of 10^{19} .

These values are retained only as illustrations of combinatorial growth. They are *not* literal target-specific proof counts: Metamath assertions are schematic and admit substitutions, whereas the original proof-count theorem treats finitary partial rules. A literal identification requires an explicit encoding of substitutions and instantiated rules. No theorem in Version 48 depends on the numerical proxy.

4 Relationship to induction

Learned proof-search guidance is not a fifth kind of induction. It is a policy acting on legal stage transitions and proof branches whose soundness and invariants are analyzed by the existing induction principles. The Version 48 separation is therefore:

proof-count bound: construction-space size

learned guidance: allocation of finite search attention

formal verifier: certificate acceptance

At the time of Version 47, HaloProof was identified as a harder intended transfer test. Version 48 records that statement historically but makes no current claim about the status or outcome of that benchmark.

Version 48 Integration B: AMLDs, Cooperative Induction, and Settlement Speed

Scope and relation to the AMLD program

This integration revises the Version 46.3 AMLD supplement as part of the Version 48 reconciliation. It uses no Omnicartographer, geodesic, or oracle hypothesis. Its object is a cooperative verifier-gated settlement architecture with five search roles sharing typed verified memory. The corrections below make explicit three points that were previously only implicit: the refutation and contradiction targets must belong to the object language; the dependence and independence channels operate over a declared metatheory rather than automatically over the object theory; and first settlement freezes the first-settlement time, not the total set of settlement facts that may later be certified.

1 Formal AMLD setting

Fix a formal system $F = (x, y, z)$, a verifier-trusted initial object-level BANK $B_0 \subseteq y$, and a target $T \in y$. Fix also a designated refutation target $T^- \in y$ and contradiction target $\perp \in y$. When the object language has the intended negation connective, one may set $T^- = \neg T$; the notation T^- avoids assuming that every formal system has primitive negation syntax.

Write

$$B \vdash_F \varphi, \quad \text{Con}_F(B) := \{\varphi \in y : B \vdash_F \varphi\}.$$

For a provable object-level target define the explicit-premise shortest-proof horizon

$$\|\varphi\|_F(B) := \min\{|\pi| : \pi \text{ is an } F\text{-proof of } \varphi \text{ from } B\},$$

with value ∞ if no such proof exists.

To speak formally about dependence and independence, fix a metatheory

$$F^{\text{meta}} = (x^{\text{meta}}, y^{\text{meta}}, z^{\text{meta}})$$

with a declared encoding of F , its premise sets, its derivability relation, and the target pair (T, T^-) . The metatheory has sound verifier interfaces for the certificate languages actually used below. Define the intended metalogical predicates

$$\text{Dep}_{F,B}(T) \iff (B \vdash_F T) \vee (B \vdash_F T^-),$$

$$\text{Ind}_{F,B}(T) \iff (B \not\vdash_F T) \wedge (B \not\vdash_F T^-).$$

These displayed biconditionals specify the intended mathematical meanings. For each represented object state B and target pair (T, T^-) , fix distinguished metatheory sentences

$$\widehat{\text{Dep}}_{F,B,T}, \widehat{\text{Ind}}_{F,B,T} \in y^{\text{meta}}.$$

The metatheory verifier interface is required to be sound for these encodings: acceptance of a certificate for $\widehat{\text{Dep}}_{F,B,T}$ implies $\text{Dep}_{F,B}(T)$, and acceptance of a certificate for $\widehat{\text{Ind}}_{F,B,T}$ implies $\text{Ind}_{F,B}(T)$. Completeness is not assumed.

Definition 1.1 (AMLD). *An automated metalogical decider for $(F, F^{\text{meta}}, B_0, T, T^-, \perp)$ is a coordinated architecture*

$$\mathcal{A} = (P, R, D, I, C; \text{BANK}, V_F, V_{\text{meta}}),$$

where P, R, C are ATP channels over the object theory F and D, I are certifier channels over the declared metatheory F^{meta} . The roles seek verifier-certified records of the following kinds:

$$\begin{aligned} P & : B_k \vdash_F T, \\ R & : B_k \vdash_F T^-, \\ D & : \widehat{\text{Dep}}_{F, B_k, T}, \\ I & : \widehat{\text{Ind}}_{F, B_k, T}, \\ C & : B_k \vdash_F \perp. \end{aligned}$$

The object-level and metalogical BANK components are type-distinct. A metalogical record about provability or nonprovability is never usable as an object-level premise merely because both are stored by the same architecture.

Remark 1.2 (Verifier versus inference legality). The abstract proof relation is determined by the partial inference rules of the formal system $F = (x, y, z)$. The interface V_F does not add a second foundational notion of validity; it is an implementation-level checker that confirms that a presented finite certificate consists only of legal applications of those rules. Likewise, V_{meta} checks certificates in the declared metatheory.

Remark 1.3 (Architectural use of the word decider). Here “automated metalogical decider” names the coordinated settlement architecture. It does not assert that every target is totally decidable by a terminating algorithm. If no sound settlement certificate is produced, an AMLD run may remain unsettled.

Remark 1.4 (No automatic independence oracle). The existence of a useful I -channel certificate language is an additional metatheoretic assumption. Failure of P and R to find proofs, timeout, or budget exhaustion does not certify independence. Likewise, the soundness and completeness properties of D and I depend on the declared metatheory and verifier; Version 48 makes no claim that arbitrary independence questions are mechanically decidable.

Remark 1.5 (Dependence is not a fifth truth value). A proof or refutation already implies dependence. The D role is therefore architectural rather than a fifth mutually exclusive object-level truth value. A sound metatheoretic dependence certificate may be useful before P or R exposes the corresponding object-level witness.

Definition 1.6 (Verifier-gated conservative object deposit). *Suppose an object-level formula λ is proposed for deposit at global stage k . It is accepted only with an object-level certificate verified by V_F . The deposit is conservative at stage k when that certificate establishes*

$$B_k \vdash_F \lambda.$$

The object BANK then updates by

$$B_{k+1} = B_k \cup \{\lambda\}.$$

The originating channel need not be P ; any channel may contribute an object-level lemma if it supplies the required object-level certificate.

Fix one realized execution schedule. Let S_k^J be the local accepted state of role $J \in \{P, R, D, I, C\}$ after global stage k , let B_k^{obj} and B_k^{meta} be the typed BANK components, let Π_k be provenance, and let $\Sigma_k \subseteq \{P, R, D, I, C\}$ record all settlement classes accepted by stage k . Let τ_k be the first-settlement time when one has occurred, and ∞ otherwise. The global accepted state is

$$X_k = (S_k^P, S_k^R, S_k^D, S_k^I, S_k^C, B_k^{\text{obj}}, B_k^{\text{meta}}, \Pi_k, \Sigma_k, \tau_k).$$

Definition 1.7 (Settlement record). *A mathematical settlement event is acceptance of a verifier-certified record in one of the five designated classes. Operational outcomes such as timeout, budget exhaustion, and “unknown” are not settlements. The set Σ_k is monotone: later certificates may add information. The first-settlement time, once finite, is absorbing. Thus a dependence settlement may later be refined by an actual proof or refutation without changing the historical first-settlement time.*

2 Global-state induction and conservative BANK growth

Theorem 2.1 (AMLD global-state induction). *Let Q be a property of global accepted states. If $Q(X_0)$ holds and $Q(X_k) \Rightarrow Q(X_{k+1})$ for every accepted AMLD transition, then $Q(X_k)$ holds for every $k \in \mathbb{N}$.*

Proof. Ordinary induction on k . □

Theorem 2.2 (Conservative BANK invariant). *Assume every accepted object-level deposit is conservative relative to the object BANK immediately preceding it. Then for every k ,*

$$B_k^{\text{obj}} \subseteq \text{Con}_F(B_0) \quad \text{and} \quad \text{Con}_F(B_k^{\text{obj}}) = \text{Con}_F(B_0).$$

Proof. Induct on accepted object-BANK updates. Extensiveness gives $B_0 \subseteq \text{Con}_F(B_0)$. Suppose $B_k^{\text{obj}} \subseteq \text{Con}_F(B_0)$ and $B_k^{\text{obj}} \vdash_F \lambda$. Then $\lambda \in \text{Con}_F(B_k^{\text{obj}}) \subseteq \text{Con}_F(\text{Con}_F(B_0)) = \text{Con}_F(B_0)$ by monotonicity, the induction hypothesis, and idempotence. Hence the enlarged BANK remains a subset of $\text{Con}_F(B_0)$. Since $B_0 \subseteq B_k^{\text{obj}}$, monotonicity gives the reverse closure inclusion, and idempotence gives equality. □

Corollary 2.3 (Logical-status invariance under conservative cooperation). *Under the preceding hypotheses, the truth values of*

$$B_k^{\text{obj}} \vdash_F T, \quad B_k^{\text{obj}} \vdash_F T^-, \quad B_k^{\text{obj}} \vdash_F \perp, \quad \text{Dep}_{F, B_k^{\text{obj}}}(T), \quad \text{Ind}_{F, B_k^{\text{obj}}}(T)$$

are independent of k .

Proof. The consequence closure is constant. The first three predicates are membership claims in that closure, while the last two are defined from the first two derivability predicates. □

3 Channel discovery, BANK birth, and certificate induction

For an object-level formula λ and role J , let

$$d_J(\lambda) := \min\{k : \lambda \text{ has been validly derived by } J \text{ by stage } k\},$$

with value ∞ if no such derivation occurs. Define the cooperative object-BANK birth time

$$b_{\mathcal{A}}(\lambda) := \min\{k : \lambda \in B_k^{\text{obj}}\}.$$

Discovery and BANK birth need not agree.

Definition 3.1 (Certificate-preserved property). *Let $\mathcal{P}$ be a property of object-level formulas. Say $\mathcal{P}$ is certificate-preserved when (i) every formula in B_0 has $\mathcal{P}$, and (ii) every primitive object-level inference rule that may occur inside an accepted expanded deposit certificate preserves $\mathcal{P}$ from its premises to its conclusion.*

Theorem 3.2 (Cooperative BANK birth-time induction). *Assume every accepted object-level deposit has a finite F -derivation whose external leaves belong to the preceding object BANK, and assume $\mathcal{P}$ is certificate-preserved. Then every object-level formula with finite cooperative BANK birth time has property $\mathcal{P}$.*

Proof. Use strong induction on $b_{\mathcal{A}}(\lambda)$. A formula born at stage 0 lies in B_0 . Suppose λ is first deposited at stage $k + 1$. Every external BANK leaf of its accepted certificate belongs to B_k^{obj} and therefore has earlier BANK birth and property $\mathcal{P}$ by the outer induction hypothesis. Now induct through the finite certificate DAG in dependency order. Every internal node is obtained from earlier nodes by a primitive rule preserving $\mathcal{P}$, so every internal formula, including the root λ , has $\mathcal{P}$. $\square$

Corollary 3.3 (Channel-independent invariant transfer). *Under the theorem's hypotheses, object-level BANK invariants depend on certificate structure, not on which role discovered the deposit. In particular a lemma discovered during an unsuccessful refutation search may later be reused by the prover if it was deposited through an accepted object-level certificate.*

4 Settlement time and refinement

For each role J , let $\tau_{\mathcal{A}}^J(T)$ be its first global stage of a designated accepted settlement certificate, with value ∞ if none occurs. Define

$$\tau_{\mathcal{A}}^{\text{set}}(T) := \min\{k : \Sigma_k \neq \emptyset\}.$$

Proposition 4.1 (First-settlement minimum). *In the extended natural numbers,*

$$\tau_{\mathcal{A}}^{\text{set}}(T) = \min\{\tau_{\mathcal{A}}^P, \tau_{\mathcal{A}}^R, \tau_{\mathcal{A}}^D, \tau_{\mathcal{A}}^I, \tau_{\mathcal{A}}^C\}.$$

Proof. By definition the first settlement is the first accepted certificate in any designated class. $\square$

Proposition 4.2 (Persistent first settlement with later refinement). *If $\tau_{\mathcal{A}}^{\text{set}}(T) = h < \infty$, then for every $k \geq h$ the system remains settled and retains first-settlement time h , while $\Sigma_h \subseteq \Sigma_k$ may grow as additional certified settlement information arrives.*

Proof. Induct on $k - h$. The settled flag and first-settlement time are absorbing by transition rule, while the settlement record is append-only. $\square$

5 Proof horizons under cooperative BANK growth

For the object-level settlement roles define

$$h_P(k) := \|T\|_F(B_k^{\text{obj}}), \quad h_R(k) := \|T^-\|_F(B_k^{\text{obj}}), \quad h_C(k) := \|\perp\|_F(B_k^{\text{obj}}).$$

The D and I roles may be assigned analogous horizons only after a particular metatheory, certificate language, and cost convention have been fixed.

Theorem 5.1 (Cooperative proof-horizon theorem). *For $J \in \{P, R, C\}$,*

$$h_J(k + 1) \leq h_J(k)$$

whenever $B_k^{\text{obj}} \subseteq B_{k+1}^{\text{obj}}$.

Proof. Every proof from the smaller premise set remains a proof from the larger premise set. Taking minima gives the inequality. $\square$

Remark 5.2 (Expanded certificates and runtime). A shorter explicit-premise horizon need not shorten the fully expanded primitive proof from B_0 : a deposited lemma may function as a macro. Nor does horizon monotonicity imply heuristic runtime monotonicity. Extra BANK formulas may increase branching, indexing cost, duplicate work, or memory pressure. The theorem states availability of a no-longer route, not that a fixed heuristic finds it sooner.

6 Comparing a stand-alone ATP with an AMLD

Separate three quantities: logical effort, wall-clock time, and total work. A comparison theorem is meaningful only after its resource convention is fixed.

Definition 6.1 (Baseline-preserving prover channel). *Let $P_\star$ be a stand-alone target-search ATP for T on B_0 . The AMLD prover P is baseline-preserving relative to $P_\star$ if, whenever it ignores foreign deposits, it can perform every transition available to $P_\star$ in the same local order and with no greater local transition cost.*

Definition 6.2 (Nonstarving logical-round schedule). *A global AMLD schedule is P -nonstarving at unit rate when, by the end of global round k , P has received at least k local transition opportunities unless mathematical settlement has already occurred.*

Theorem 6.3 (Constituent-ATP emulation). *If $P_\star$ proves T after m local transitions, P is baseline-preserving, and the AMLD uses a P -nonstarving unit-rate schedule, then either another role settles before global round m or P reproduces the stand-alone proof by global round m . Consequently, in all cases,*

$$\tau_A^{\text{set}}(T) \leq m.$$

If no other role settles before round m , then also

$$\tau_A^P(T) \leq m.$$

The latter bound also holds when the schedule is separately required to continue supplying P its baseline service after another role settles.

Proof. If another role settles before round m , the first displayed inequality is immediate. Otherwise mathematical settlement has not yet occurred, so nonstarvation supplies at least m local opportunities to P by round m . Baseline preservation lets P ignore foreign deposits and reproduce the stand-alone transitions. Its proof certificate is then a valid AMLD settlement, and $\tau_A^P(T) \leq m$ in this case. $\square$

Remark 6.4 (No unconditional five-versus-one speed theorem). The emulation theorem is not a claim that five ATPs are inherently faster than one. It can fail as a wall-clock comparison if resources are divided, verifier contention delays P , foreign BANK material cannot be ignored, or the schedule starves P .

Proposition 6.5 (Wall-clock dominance under resource replication). *Assume P receives the same processor, memory, verifier availability, and local service rate as $P_\star$ would receive alone, and foreign deposits may be ignored. If the stand-alone run terminates with a proof of T , then*

$$t_A^{\text{set}}(T) \leq t_\star(T).$$

Proof. The AMLD prover can reproduce the stand-alone run without resource loss; another role may only settle earlier under the stated assumptions. $\square$

7 Wall-clock speed and total work

Up to first settlement write

$$W_A = W_P + W_R + W_D + W_I + W_C + W_V + W_B,$$

where W_V and W_B account for verification and BANK management, and let $W_\star$ be stand-alone work.

Proposition 7.1 (No architecture-only total-work dominance). *The five-role AMLD architecture alone implies neither $W_A \leq W_\star$ nor $W_A \geq W_\star$ on every target.*

Proof. AMLD work can exceed stand-alone work when losing roles perform large irrelevant searches while P reproduces the baseline. It can be smaller when another role finds a valid early settlement and an early-stop policy prevents most of the baseline work. Architecture alone therefore determines no universal inequality. $\square$

8 Weighted and causal AMLD potential bounds

Suppose every accepted object-level deposit λ_i has a finite certificate from the preceding BANK. Let q_i be the maximum number of primitive inference layers on a dependency path in that certificate when earlier BANK entries are treated as leaves, and let $Q_n = \sum_{i=1}^n q_i$.

Theorem 8.1 (Cumulative weighted AMLD potential bound). *Let $r : \mathcal{Y} \rightarrow \mathbb{N}$ and $d \in \mathbb{N}^+$. Assume $r(\gamma) \leq r_0$ for $\gamma \in B_0$, and every primitive inference $\alpha_1, \dots, \alpha_s \mapsto \beta$ occurring in an accepted expanded certificate satisfies*

$$r(\beta) \leq \max_i r(\alpha_i) + d.$$

Then every $\varphi \in B_n^{\text{obj}}$ satisfies

$$r(\varphi) \leq r_0 + dQ_n.$$

Consequently, if λ is first deposited by update n and $r(\lambda) > r_0$, then

$$Q_n \geq \left\lceil \frac{r(\lambda) - r_0}{d} \right\rceil.$$

Proof. Induct on accepted deposits. Along a certificate path for deposit n , at most q_n primitive layers are traversed, each increasing rank by at most d . The maximum rank among earlier BANK leaves is bounded by $r_0 + dQ_{n-1}$ by induction, yielding $r(\lambda_n) \leq r_0 + dQ_n$; old BANK entries retain the previous bound. $\square$

The cumulative sum may charge deposits irrelevant to a particular conclusion. A sharper schedule-insensitive quantity follows the actual dependency ancestry.

Definition 8.2 (Causal BANK depth). *Assign $\delta(\gamma) = 0$ for $\gamma \in B_0$. For a deposited formula λ , expand its accepted certificate as a finite DAG whose external leaves are earlier BANK entries. Give every primitive inference edge unit weight and define $\delta(\lambda)$ to be the maximum, over all external leaves ψ and dependency paths from ψ to the certificate root, of*

$$\delta(\psi) + (\text{number of primitive inference edges on that path}).$$

Theorem 8.3 (Causal potential bound). *Under the hypotheses of the cumulative potential theorem, every deposited object-level formula satisfies*

$$r(\lambda) \leq r_0 + d\delta(\lambda).$$

Hence

$$\delta(\lambda) \geq \left\lceil \frac{\max\{0, r(\lambda) - r_0\}}{d} \right\rceil.$$

Moreover, if λ is deposited by update n , then $\delta(\lambda) \leq Q_n$.

Proof. Induct over the well-founded dependency DAG obtained by recursively expanding BANK references to their accepted certificates. Base leaves in B_0 have rank at most r_0 . Each primitive inference increases rank by at most d , so a dependency path of weighted depth $\delta(\lambda)$ yields the stated upper bound. The lower bound is algebraic rearrangement. Finally, each dependency segment contributed by a deposit is at most that deposit's q_i , while Q_n sums all such segment bounds up to update n , so the causal depth cannot exceed the cumulative quantity. $\square$

9 AMLDs and the induction hierarchy

The AMLD extension uses the existing induction principles rather than introducing a new induction axiom. Structural induction applies separately to object and certificate languages when they are least-generated grammars. Rule induction applies inside object proofs and inside declared metatheories. Stage and birth-time induction apply locally to each role and globally to the accepted AMLD chronology. Reflection induction applies to any tagged constituent ATP, or to a separately defined tagged representation of the whole AMLD; the latter is an additional construction, not automatic.

Proposition 9.1 (No fifth foundational induction principle). *Global AMLD induction and cooperative BANK birth-time induction are ordinary induction on newly defined well-founded natural-number chronologies. Their extra content comes from typed transition and certificate structure, not from a stronger induction axiom.*

Proof. Both proofs are inductions on $\mathbb{N}$: global accepted-stage index or finite BANK birth time, supplemented by finite induction through certificate DAGs. $\square$

10 Monotone growth and monotone object-level horizons

Under verifier-gated object-BANK growth,

$$B_0 \subseteq B_1^{\text{obj}} \subseteq B_2^{\text{obj}} \subseteq \dots,$$

while for $\varphi \in \{T, T^-, \perp\}$,

$$\|\varphi\|_F(B_0) \geq \|\varphi\|_F(B_1^{\text{obj}}) \geq \|\varphi\|_F(B_2^{\text{obj}}) \geq \dots.$$

Under conservative deposits the consequence closure is constant. Thus cooperation may improve explicit access to an already-derivable settlement target without increasing deductive strength.

11 Conclusions

The corrected cooperative picture is:

one ATP: induction on one proof-search chronology

becomes

one AMLD: five local role chronologies plus one verifier-accepted shared chronology.

The five roles need not live in one object theory: P, R, C operate over F , while D, I require the declared metatheory and sound certificate interfaces. Conservative object-BANK growth preserves the initial consequence closure, shared BANK birth-time induction propagates object-level invariants through finite expanded certificates, object-level shortest explicit-premise horizons are nonincreasing, and a baseline-preserving nonstarved prover can emulate a stand-alone ATP under the stated resource convention. None of these statements alone implies a universal wall-clock or total-work advantage.

Version 49 Integration A: Clocked Simulation Degrees

1 Standing clocked-simulation definitions

Let a semi-ideal computer (SIC) be $M = (C_M, H_M)$ with state space S_M . A *clocked deterministic simulation* from M to N is a pair (f, ρ) such that $f : S_M \rightarrow S_N$ is injective, $\rho : \mathbb{N}^+ \rightarrow \mathbb{N}^+$ satisfies $\rho(1) = 1$, is nondecreasing and unbounded, and for all $p \in S_M$ and $m \in \mathbb{N}^+$,

$$f(C_M(p, m)) = C_N(f(p), \rho(m)), \quad H_M(p, m) = H_N(f(p), \rho(m)).$$

Write $M \preceq_{\text{clk}} N$. Mutual clocked simulation defines

$$M \sim_{\text{clk}} N \iff M \preceq_{\text{clk}} N \text{ and } N \preceq_{\text{clk}} M,$$

and the quotient degree poset is

$$\mathcal{D}_{\text{clk}} := \text{SIC} / \sim_{\text{clk}}.$$

Reflexivity is witnessed by identity maps and the identity clock. Transitivity follows by composing injective state maps and the nondecreasing unbounded clocks. Thus $\preceq_{\text{clk}}$ is a preorder, while the quotient relation on degrees is a partial order.

2 Settlement of former Open Problem 9.1: infinite antichains

The Version 48 addendum had already shown incomparable degrees, a strict chain of order type ω , and finite antichains of arbitrarily large size. The remaining question was whether a genuine infinite antichain exists; the settlement recorded here follows the companion settlement note [4].

Theorem 2.1 (Infinite clocked-simulation antichain). *The poset $\mathcal{D}_{\text{clk}}$ contains a countably infinite antichain consisting of degrees of finite, finitely describable SICs.*

Proof. Let $\pi \in S_n$ be a finite permutation. Define

$$Y_\pi = \{a_1, b_1, \dots, a_n, b_n\}, \quad p_i = \{a_i\}, \quad q_i = \{a_i, b_i\},$$

and take the full finite state space

$$S_\pi = \mathcal{P}(Y_\pi).$$

Define $M_\pi = (C_\pi, H_\pi)$ by making every state outside the distinguished p_i 's static and permanently unhalting, and by setting

$$C_\pi(p_i, t) = \begin{cases} p_i, & t \leq i, \\ q_i, & t \geq i + 1, \end{cases} \quad H_\pi(p_i, t) = \begin{cases} 0, & t < n + 1 + \pi(i), \\ 1, & t \geq n + 1 + \pi(i). \end{cases}$$

Thus the distinguished trajectories have their unique strict growth events in the fixed order

$$1 < 2 < \dots < n,$$

while their halting thresholds occur in the relative order encoded by π . Every halting threshold lies after every growth threshold, so the stage-one, monotonicity, persistent-halting, and frozen-after-halting requirements are immediate; hence M_π is a finite SIC.

Suppose $M_\pi \preceq_{\text{clk}} M_\sigma$ via (f, ρ) , where $\sigma \in S_m$. Because each p_i has a nonconstant trajectory and f is injective, its image must be one of the distinguished nonconstant trajectories of M_σ . Write this image as $p_{j_i} \rightsquigarrow q_{j_i}$. Comparing two source trajectories at a common stage at which the earlier one has grown and the later one has not, and then applying the common clock ρ , forces

$$j_1 < j_2 < \cdots < j_n.$$

Likewise, comparing one halted and one unhalted source trajectory at the same source stage and using halting preservation forces

$$\pi(i) < \pi(k) \iff \sigma(j_i) < \sigma(j_k).$$

Hence π occurs as a permutation pattern in σ :

$$M_\pi \preceq_{\text{clk}} M_\sigma \implies \pi \leq_{\text{pat}} \sigma.$$

Finite permutations under pattern containment contain a countably infinite antichain [5]. Choosing such a family $\pi_1, \pi_2, \dots$ gives, for $r \neq s$,

$$M_{\pi_r} \not\preceq_{\text{clk}} M_{\pi_s}, \quad M_{\pi_s} \not\preceq_{\text{clk}} M_{\pi_r}.$$

Therefore the degrees $[M_{\pi_r}]_{\text{clk}}$ form a countably infinite antichain. Every M_{π_r} is finite and therefore finitely describable. $\square$

Corollary 2.2 (Cardinality-maximal width in the finitely describable class). *Among finitely describable SICs, the cardinality $\aleph_0$ achieved by Theorem 2.1 is the largest possible cardinality of an antichain.*

Proof. Finite descriptions over a finite or countable description alphabet form a countable set. Hence there are at most countably many finitely describable SICs, and therefore at most countably many clocked-simulation degrees represented by them. Theorem 2.1 realizes a countably infinite antichain. $\square$

Remark 2.3. “Cardinality-maximal” here does not mean that the displayed antichain is a *maximal antichain* under inclusion. It means only that no antichain drawn from the finitely describable class can have cardinality strictly larger than $\aleph_0$.

3 Settlement of former Open Problem 9.3: joins and meets

For a finite static SIC with constant halting value in time, write $S(h, u)$ when exactly h states are permanently halted and u are permanently unhalted. Because the full finite state space is a power set,

$$h + u = 2^{|Y|}.$$

Moreover,

$$S(h, u) \preceq_{\text{clk}} S(h', u') \iff h \leq h' \text{ and } u \leq u'.$$

Theorem 3.1 (Failure of lattice structure). *The poset $\mathcal{D}_{\text{clk}}$ is not a lattice. More strongly:*

- (i) *some two degrees have common upper bounds but no join;*
- (ii) *some two degrees have common lower bounds but no meet.*

Proof. Take $A = S(3, 1)$ and $B = S(1, 3)$. They have common upper bounds, for example $U = S(3, 5)$ and $V = S(5, 3)$. If a join J existed, then $J \preceq_{\text{clk}} U, V$. Since U, V are static, injective state preservation forces J itself to be static with constant halting type, so $J = S(h_J, u_J)$. The lower-bound requirements $A, B \preceq_{\text{clk}} J$ force $h_J, u_J \geq 3$, while $J \preceq_{\text{clk}} U, V$ forces $h_J, u_J \leq 3$. Thus J would have exactly six states, impossible because a finite SIC state space has power-of-two cardinality.

Dually, take $A' = S(3, 5)$ and $B' = S(5, 3)$. They have common lower bounds $S(3, 1)$ and $S(1, 3)$. A greatest lower bound would again have to be $S(3, 3)$, which is unrealizable for the same six-state obstruction. $\square$

4 Settlement of former Open Problem 9.4: synchronized invariants

Definition 4.1 (Finite synchronized observation spectrum). *Choose finitely many marked states $p_1, \dots, p_r \in S_M$ and a finite nondecreasing stage sequence $t_1 \leq \dots \leq t_s$. Record all halting bits $H_M(p_i, t_j)$ and all equality/inequality relations among the observed states $C_M(p_i, t_j)$. Let $\text{Obs}(M)$ be the set of all such finite patterns, up to relabeling of the marked states.*

Theorem 4.2 (Dynamic monotonicity and complete static invariant). *If $M \preceq_{\text{clk}} N$, then*

$$\text{Obs}(M) \subseteq \text{Obs}(N).$$

Hence Obs is a monotone dynamic invariant. On the finite static constant-halting subclass, the pair (h, u) is a complete invariant of clocked-simulation degree.

Proof. If (f, ρ) witnesses $M \preceq_{\text{clk}} N$, use the target marked states $f(p_1), \dots, f(p_r)$ and target stages $\rho(t_1) \leq \dots \leq \rho(t_s)$. The simulation equations preserve all recorded halting bits. Injectivity preserves equality and inequality exactly. Thus every synchronized finite pattern realized in M is realized in N .

For finite static constant-halting SICs,

$$S(h, u) \preceq_{\text{clk}} S(h', u') \iff h \leq h' \text{ and } u \leq u'.$$

Mutual simulation therefore holds exactly when $(h, u) = (h', u')$. □

5 Settlement of former Open Problem 9.5: effective degrees

Theorem 5.1 (Effective survival theorem). *Under effective clocked simulation, all of the following already occur among finite effectively presented SICs:*

- (i) *incomparable degrees;*
- (ii) *absence of a least degree;*
- (iii) *a strict chain of order type ω ;*
- (iv) *finite antichains of arbitrarily large size;*
- (v) *a countably infinite antichain;*
- (vi) *failure of joins and meets as in Theorem 3.1.*

Proof. Every finite SIC has an effective presentation by enumerating its finite state set. Its coded stage and halting maps are finite lookup functions. Every positive witness used in the finite constructions can be chosen as an explicit finite injection with the identity clock and is computable. Every negative result rules out an ordinary clocked simulation and therefore also rules out an effective one. The infinite antichain of Theorem 2.1 consists entirely of finite machines, so the same family is an antichain for effective clocked simulation. □

6 The one former open problem intentionally left open

Open Problem 6.1 (Greatest clocked-simulation degree). *Does $\mathcal{D}_{\text{clk}}$ have a greatest element?*

Version 49 does not infer a negative answer merely from the failure of natural universal-looking candidates. The greatest-degree question remains separate from the infinite-antichain and nonlattice results.

Version 49 Integration B: Generalized Board Games and Strategic Settlement

7 Purpose of the synthesis

Two papers in the same formal program approach proof from opposite directions. *Depths of Induction*, Version 48 (hereafter *Depths v48*) [1] starts with general formal systems and develops inference closure, proof horizons, proof search, machine learning guidance, cooperative AMLDs, typed verified memory, and formalized self-awareness. The GBG paper [2] starts with familiar games and shows that their rule-complete states and legal moves instantiate the same *xyz* architecture.

The important interaction is not the analogy “proofs are like games.” That observation is too weak. The useful question is whether the GBG construction supplies additional mathematical structure for the objects already present in *Depths v48*. The answer developed here is yes, but at three different levels that must not be conflated:

- L1. Logical derivation level.** A finitary proof system can be lifted to a unary transition graph on finite derivation states. Theoremhood becomes reachability and a step-count proof horizon becomes graph distance.
- L2. Operational search level.** A concrete ATP has richer states: queues, open goals, scores, candidate caps, seeds, BANK snapshots, and other implementation data. Discovery cost is a hitting time in this operational graph, not a synonym for proof length.
- L3. Strategic control level.** When some transitions are selected by a controller and others by a scheduler, opponent, environment, or cooperating role, ordinary reachability may be strengthened to forced reachability. This gives a robust or forced-settlement horizon with min/max recurrences.

The third level is the genuinely new conceptual layer for *Depths v48*. Version 48 already proves a nonstarvation-based emulation result. The GBG viewpoint reveals that theorem as a constrained forced-reachability statement.

8 Standing formal framework from Depths v48

8.1 Formal systems and partial inference rules

Fix an *xyz* formal system

$$F = (x, y, z),$$

where x is a nonempty alphabet, y is a set of well-formed finite strings over x , and each $r \in z$ is a finitary partial rule

$$r : y^k \rightharpoonup y$$

for some positive finite arity k . Partiality represents applicability: an inference rule need not be legal on every k -tuple.

For a premise set $B \subseteq y$, write

$$B \vdash_F \varphi, \quad \text{Con}_F(B) = \{\varphi \in y : B \vdash_F \varphi\}.$$

The Version 48 proof-horizon notation for a provable object-level target is

$$\|\varphi\|_F(B) = \min\{|\pi| : \pi \text{ is an } F\text{-proof of } \varphi \text{ from } B\},$$

with value ∞ if no proof exists. Because proof-length conventions differ, we shall also use a cost that counts primitive inference applications explicitly.

Definition 8.1 (Inference-step proof cost). *For a finite proof π from B , let $\ell_{\text{inf}}(\pi)$ be the number of non-premise primitive inference applications appearing in π . Define*

$$H_F^{\text{inf}}(B, \varphi) := \min\{\ell_{\text{inf}}(\pi) : \pi \text{ proves } \varphi \text{ from } B\},$$

with value ∞ if no proof exists.

This auxiliary convention is chosen because one legal state transition below will correspond to one primitive inference application. If the Version 48 symbol $|\pi|$ is instantiated by the same convention, then $H_F^{\text{inf}} = \|\cdot\|_F$. Under a line-count convention the quantities differ by the declared accounting of premises and repeated lines.

8.2 The Version 48 separation of metrics

A central discipline of *Depths v48* is that proof length, logical birth stage, expansion count, wall-clock time, and total work are different quantities. In particular, Version 48 defines an expansion discovery cost $\tilde{E}_M(\varphi)$ for a fixed ATP trajectory and records right-censored failure at budget B as $\tilde{E}_M(\varphi) > B$, not as an exact value.

That distinction will become geometric:

Quantity	GBG interpretation
Intrinsic proof horizon	Shortest directed path in a logical derivation-state graph.
Discovery expansions	Hitting time along one implementation-dependent search trajectory or search tree.
Wall-clock time	A cost placed on operational transitions and resource contention.
Total work	Aggregate cost over all participating channels, verification, and BANK management.
Forced-settlement horizon	Worst-case compatible path length under a controller strategy and admissible external choices.

9 The GBG layer

The GBG paper specializes the *xyz* formalism to systems in which every well-formed object is a rule-complete game state and every inference rule is a partial unary move rule

$$r_\mu : y \rightarrow y.$$

A distinguished initial state s_0 acts as a hypothesis, and the paper proves

$$s \in \text{Con}_F(\{s_0\}) \iff s \text{ is reachable from } s_0 \text{ by finitely many legal moves.}$$

It also distinguishes the match tree of histories from the state graph of endpoints and develops alternating forced-win regions. The forced-win horizon satisfies a minimum recursion on the favored player's turns and a maximum recursion on the opponent's turns.

The apparent mismatch with *Depths v48* is that general inference rules may be k -ary while GBG move rules are unary. The first task is therefore to lift k -ary inference into unary transitions on a richer state space.

10 Operational lifting: from k -ary inference to unary proof-state moves

Definition 10.1 (Finite derivation state). *Fix $F = (x, y, z)$ and a finite initial premise sequence $B_0 = (\beta_1, \dots, \beta_m)$. A finite derivation state is a finite sequence*

$$S = (\beta_1, \dots, \beta_m, \varphi_{m+1}, \dots, \varphi_n)$$

such that every φ_j with $j > m$ is the conclusion of one legal primitive inference from formulas occurring earlier in the sequence.

The use of a sequence rather than a set preserves enough information to reconstruct a certificate. If only future logical applicability matters, one may later quotient away redundant history.

For every primitive rule $r : y^k \multimap y$ and every ordered index tuple $\mathbf{i} = (i_1, \dots, i_k)$, define a partial unary operator on derivation states by

$$\hat{r}_{\mathbf{i}}(S) = S \smallfrown r(\varphi_{i_1}, \dots, \varphi_{i_k})$$

whenever all indices refer to entries already in S and the tuple lies in $\text{dom}(r)$. Otherwise the operator is undefined.

Theorem 10.2 (Operational lifting theorem). *Let F be a finitary xyz formal system and B_0 a finite premise sequence. There is a unary state-transition system*

$$\mathcal{G}_{\text{der}}(F, B_0)$$

whose states are finite legal derivation states and whose legal transitions append exactly one primitive inference conclusion, such that finite F -proofs from B_0 are in canonical correspondence with finite paths beginning at the initial state $S_0 = B_0$, up to inessential choices concerning unused or repeated proof lines.

Proof. Given a proof, read its primitive inference lines in proof order and append each conclusion to the current derivation state. Every append is legal because its premises occur earlier and the underlying rule is defined on that premise tuple. Conversely, a finite path in the transition system records, at every edge, a primitive rule and the earlier entries used as premises. Reading those labels recovers a finite derivation. Unused or deliberately repeated lines may create different derivation-state histories with the same essential proof dependency DAG; this is the stated inessential freedom. $\square$

Corollary 10.3 (Theoremhood as target-region reachability). *For $T \in y$, define*

$$W_T := \{S : T \text{ occurs in } S\}.$$

Then

$$B_0 \vdash_F T \iff W_T \cap \text{Reach}(S_0) \neq \emptyset.$$

Proof. A proof of T lifts to a path whose final state contains T . Conversely, a reachable state containing T contains a recorded legal derivation of T from B_0 . $\square$

10.1 What has and has not been shown

The operational lifting does not say that every formal system is literally a competitive board game. It says that its finite derivations admit a GBG-style unary state-transition presentation. Competition, alternating control, and forced winning are additional structure. This separation will matter later.

11 Proof horizon as directed distance

Give every derivation-state edge unit cost. For a state S and target region W , define the directed graph distance

$$\text{dist}_G(S, W) = \inf\{n : \text{there is an } n\text{-edge path from } S \text{ to } W\},$$

with value ∞ when no such path exists.

Theorem 11.1 (Proof-horizon/distance theorem). *Under the inference-step cost convention,*

$$H_F^{\text{inf}}(B_0, T) = \text{dist}_{\mathcal{G}_{\text{der}}(F, B_0)}(S_0, W_T).$$

Proof. By the operational lifting theorem, every proof with n primitive inference applications gives an n -edge path to W_T , and every n -edge path to W_T gives a proof using n primitive inference applications. Taking minima on the two corresponding sets gives equality. $\square$

Remark 11.2 (Line-count conventions). If proof length counts premise lines, displayed lines, or administrative certificate material, exact equality should not be asserted without reconciling cost conventions. The geometric theorem is exact for the edge cost explicitly defined above.

This gives a precise version of the slogan

$$\boxed{\text{proof horizon} = \text{distance to a target region}}$$

without importing any Euclidean or Hilbert-coordinate geometry. It is intrinsic directed graph distance in the derivation transition system.

12 Histories, states, and transpositions

The GBG paper [2] distinguishes a match tree from its endpoint state graph. The same distinction is essential in proof search.

Definition 12.1 (Derivation history tree). *The derivation history tree $\mathcal{T}(S_0)$ has as its depth- n nodes the move-labeled legal derivation histories of exactly n primitive inference transitions beginning at S_0 .*

Two different histories can establish the same collection of usable formulas. Whether they can be safely merged depends on what future transitions are allowed to inspect.

Definition 12.2 (Future-complete equivalence). *Fix an observation-label map L on operational states that records every state predicate relevant to the claims under study—at minimum target membership, settlement status, and verifier/acceptance status when those notions are present. Two operational states S and S' are future-complete equivalent, written $S \equiv_{\text{fc}} S'$, if they are related by a label-preserving strong bisimulation: $L(S) = L(S')$, and for every labeled transition $S \xrightarrow{a} T$ there is a transition $S' \xrightarrow{a} T'$ with $T \equiv_{\text{fc}} T'$, and conversely for every transition from S' .*

For pure monotone logical derivation, a simpler sufficient condition is often equality of the available formula set, provided candidate generation depends only on that set and the fixed target/environment. For a concrete ATP, the rule-complete state may need to include queues, open goals, declaration cutoffs, caches, scores, seeds, and BANK snapshots.

Proposition 12.3 (Safe transposition quotient). *Quotienting an operational history graph by $\equiv_{\text{fc}}$ preserves target reachability and all future legal continuation languages.*

Proof. Label preservation gives identical target, settlement, and acceptance observations at equivalent states. Strong bisimulation matches every labeled one-step transition in either direction by a transition with the same label into an equivalent successor. Induction on finite continuation length therefore matches every finite labeled continuation and preserves the observation label at its endpoint. Hence target reachability and finite continuation languages are preserved by the quotient. $\square$

Remark 12.4 (Transposition tables). This proposition is the proof-search analogue of a board-game transposition table. The mathematical requirement is not “same visible board” but “same future-complete state.” Merging states on weaker data can be unsound for search behavior even when it does not affect object-level theoremhood.

13 Proof-count growth and match-count growth

Version 48 recalls a crude bound on the number of length- n proof constructions. The GBG paper gives the exact recursion for the number of move-labeled histories

$$M_{n+1}(s) = \sum_{\mu: s \in \text{dom}(r_\mu)} M_n(r_\mu(s)).$$

The operational lifting makes these two viewpoints compatible.

Proposition 13.1 (Exact derivation-history recursion). *Let $A(S)$ be the set of legal labeled primitive inference applications at derivation state S . Let $M_n(S)$ be the number of legal move-labeled derivation histories of exactly n further steps. Then*

$$M_0(S) = 1, \quad M_{n+1}(S) = \sum_{a \in A(S)} M_n(a(S)),$$

where $a(S)$ denotes the successor state produced by application a .

Proof. Partition every $(n + 1)$ -step history by its first legal application. After that first move, the remaining suffix is an n -step history from the successor. $\square$

This is exact but state-dependent. Version 48’s product bound is deliberately coarser: it upper-bounds the number of syntactically available line-construction plans without requiring exact state enumeration. The two results therefore serve different purposes.

14 Three horizons

The synthesis becomes most useful when it refuses to collapse different meanings of “distance.”

14.1 Intrinsic proof horizon

The logical quantity is

$$H_{\text{proof}}(B, T) := H_F^{\text{inf}}(B, T),$$

or the Version 48 explicit-premise horizon under its declared proof-length convention. It depends on the formal calculus, premises, target, and cost convention, not on which heuristic happens to search for a proof.

14.2 Discovery effort

For a fixed implementation M , resource protocol, and seed, Version 48's

$$\tilde{E}_M(T)$$

is a search-trajectory hitting time measured in expansions. It lives in an implementation-level operational graph. Two ATPs can have the same underlying logical graph and radically different discovery costs.

14.3 Strategic or forced-settlement horizon

Now suppose operational states are partitioned by who controls the next choice. Let C denote controller states and E environment states. Let W be a settlement region.

Definition 14.1 (Finite forced-settlement region). *Set $A_0 = W$. Recursively define*

$$A_{n+1} = A_n \cup C_n \cup E_n,$$

where

$$C_n = \{s \in C : \exists t \in \text{Succ}(s) \cap A_n\}$$

and

$$E_n = \{s \in E : \text{Succ}(s) \neq \emptyset \text{ and } \text{Succ}(s) \subseteq A_n\}.$$

Let

$$W^* = \bigcup_{n \geq 0} A_n.$$

For $s \in W^*$ define the forced-settlement horizon

$$H_{\text{force}}(s) = \min\{n : s \in A_n\}.$$

Theorem 14.2 (Finite forced-settlement characterization). *Assume the strategic transition system is locally finite. Then $s \in W^*$ if and only if the controller has a strategy such that every maximal compatible play reaches W after finitely many transitions.*

Proof. This is the standard backward-induction argument used in the GBG paper. If s first appears in A_n , the controller chooses a successor in the previous stage at controller nodes, while every legal environment successor already lies in the previous stage at environment nodes. The stage index therefore strictly decreases until $A_0 = W$ is reached. Conversely, if a controller strategy forces finite settlement from a locally finite state, its compatible strategy tree is finitely branching. If it had unbounded finite depth, Koenig's lemma would yield an infinite compatible branch, contradicting finite forced settlement. Hence the tree has finite height, and backward induction places its root in some A_N . $\square$

At nonterminal forced-settlement states,

$$H_{\text{force}}(s) = 1 + \min_{t \in \text{Succ}(s) \cap W^*} H_{\text{force}}(t)$$

at controller states, and

$$H_{\text{force}}(s) = 1 + \max_{t \in \text{Succ}(s)} H_{\text{force}}(t)$$

at environment states.

Proposition 14.3 (Shortest path versus forced horizon). *Whenever $s \in W^*$,*

$$\text{dist}(s, W) \leq H_{\text{force}}(s).$$

The inequality may be strict.

Proof. A forcing strategy generates at least one compatible path to W of length at most its worst-case bound, so the unconstrained shortest path can be no longer. Strictness occurs whenever a short successful route exists but the environment can choose a longer compatible route before settlement. $\square$

Thus the hierarchy is

$$\boxed{\text{existence of a short proof}} \not\Rightarrow \boxed{\text{easy discovery}},$$

while, independently,

$$\boxed{\text{easy discovery}} \not\Rightarrow \boxed{\text{short robust settlement under external choices}}.$$

15 A layer diagram

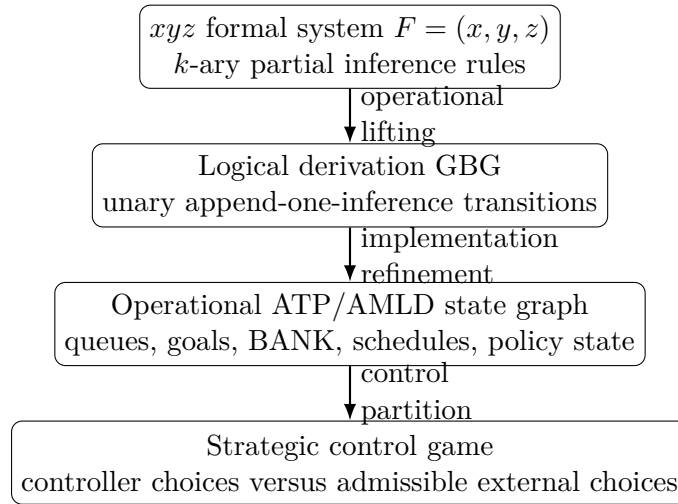


Figure 1: The GBG synthesis has three layers. Statements valid at one layer should not be promoted automatically to another.

16 AMLD global states are already rule-complete candidates

Version 48 defines a realized global accepted state

$$X_k = (S_k^P, S_k^R, S_k^D, S_k^I, S_k^C, B_k^{\text{obj}}, B_k^{\text{meta}}, \Pi_k, \Sigma_k, \tau_k).$$

This tuple is close to what the GBG paper calls a rule-complete state: it records the local accepted states of all five roles, the typed BANK components, provenance, the accepted settlement classes, and first-settlement time.

To turn the AMLD into a literal transition system, one adds whatever finite scheduler and implementation metadata is required to determine the next accepted transitions. A state is then rule-complete relative to the chosen AMLD transition semantics.

Definition 16.1 (AMLD transition game). *Fix an AMLD architecture, a finite representation of its accepted operational states, a scheduler policy class, and verifier-gated transition rules. The AMLD transition game is the directed labeled state system whose vertices are rule-complete accepted AMLD states and whose edges are accepted service, certificate, BANK, or settlement transitions permitted by the frozen architecture.*

This use of “game” need not imply competition. It means a rule-governed transition system. Strategic analysis enters only after transition authority is partitioned among roles, scheduler, environment, or controller.

17 Nonstarvation as a strategic admissibility condition

Version 48 defines a baseline-preserving prover channel and a P -nonstarving unit-rate schedule. It then proves the constituent-ATP emulation theorem: if standalone $P_\star$ proves T after m local transitions, the AMLD either settles through another role before global round m or P reproduces the standalone proof by round m .

The GBG viewpoint identifies the logical form of this statement.

Definition 17.1 (Scheduler-admissible environment). *Fix a prover channel P . An environment play is scheduler-admissible through round m if, unless mathematical settlement has already occurred, it grants P at least one baseline local transition opportunity in every global round $1, \dots, m$.*

Theorem 17.2 (Scheduler-robust emulation theorem). *Assume:*

- (i) *standalone $P_\star$ reaches a verifier-accepted proof of T after m local transitions from B_0 ;*
- (ii) *the AMLD prover P is baseline-preserving relative to $P_\star$;*
- (iii) *every environment play under consideration is scheduler-admissible through round m ;*
- (iv) *foreign deposits may be ignored by P for purposes of reproducing the baseline path.*

Then every admissible global play either enters some settlement region before round m or enters the P -settlement region by round m .

Proof. Consider any admissible global play. If another role settles before round m , the first alternative holds. Otherwise the play remains unsettled through the preceding rounds. Scheduler admissibility therefore gives P at least m baseline local opportunities by the end of round m . Baseline preservation lets P execute the same m local transitions as $P_\star$, ignoring foreign deposits when necessary. The resulting certificate is verifier-accepted, so the play enters the P -settlement region by round m . $\square$

Remark 17.3 (Why the GBG reformulation matters). The result is stronger in interpretation, not in hypotheses, than the bare phrase “AMLD can emulate the ATP.” It says the conclusion survives *every* scheduler behavior inside a declared admissible class. Thus nonstarvation converts an existential baseline route into a robust settlement guarantee against scheduling choices that remain after the constraint is imposed.

Example 17.4 (Starvation destroys the forced horizon). Suppose $P_\star$ needs exactly two local transitions. If the global scheduler is unconstrained and may forever service only other nonsettling channels, then a two-step proof still exists in the logical derivation graph, but the global forced-settlement horizon for P is infinite. Imposing unit-rate nonstarvation restores the bound of two global rounds, unless another role settles first.

This is the cleanest illustration of

$$H_{\text{proof}} < \infty \quad \text{while} \quad H_{\text{force}} = \infty$$

when external scheduling is unconstrained.

18 BANK growth as conservative road-building

Version 48 proves the conservative BANK invariant. If every accepted object-level deposit λ is certified from the preceding BANK, then

$$B_k^{\text{obj}} \subseteq \text{Con}_F(B_0), \quad \text{Con}_F(B_k^{\text{obj}}) = \text{Con}_F(B_0).$$

It also proves the cooperative proof-horizon theorem

$$\|T\|_F(B_{k+1}^{\text{obj}}) \leq \|T\|_F(B_k^{\text{obj}})$$

for object-level settlement targets under monotone BANK growth.

The GBG interpretation is precise: the set of consequences is unchanged, but the initial resources of the explicit-premise derivation game have improved.

Theorem 18.1 (Logical road-building theorem). *Assume $B \subseteq B' \subseteq \text{Con}_F(B)$. For every target T ,*

$$\text{Con}_F(B') = \text{Con}_F(B),$$

and

$$H_F^{\text{inf}}(B', T) \leq H_F^{\text{inf}}(B, T).$$

Thus conservative premise enlargement may shorten the directed distance from the new initial state to W_T without changing whether W_T is reachable at all.

Proof. Since $B' \subseteq \text{Con}_F(B)$, idempotence and monotonicity of consequence closure give

$$\text{Con}_F(B') \subseteq \text{Con}_F(\text{Con}_F(B)) = \text{Con}_F(B).$$

Since $B \subseteq B'$, monotonicity gives the reverse inclusion. For the horizon inequality, every proof from B is also a proof from B' , so minimization over the larger premise set cannot increase the minimum number of inference steps. $\square$

The slogan is therefore not merely metaphorical:

same reachable theorem region, potentially shorter explicit route.

18.1 Macro shortcuts versus expanded primitive cost

Version 48 correctly warns that a deposited lemma may behave as a macro. A one-step proof from the enlarged BANK can expand to a long primitive proof from B_0 . Thus road-building in the explicit-premise graph need not shorten the fully expanded primitive certificate.

Proposition 18.2 (Macro-expansion caution). *There is no general implication*

$$H_F^{\text{inf}}(B', T) < H_F^{\text{inf}}(B, T) \implies \text{a shorter fully expanded primitive proof of } T \text{ from } B.$$

Proof. A new premise $\lambda \in B'$ may itself require a long proof from B . Treating λ as an available premise contracts that proof to zero local cost in the enlarged problem. Expanding the macro restores its derivation and can eliminate the apparent saving.

For a concrete two-rule example, let $B = \{a\}$ and let the primitive rules be

$$a \mapsto \lambda, \quad \lambda \mapsto T.$$

Then $H_F^{\text{inf}}(B, T) = 2$. After depositing the already proved lemma and taking $B' = B \cup \{\lambda\}$, one has $H_F^{\text{inf}}(B', T) = 1$. But the one-step proof from B' expands back to the same two primitive inference steps from B ; no shorter fully expanded primitive proof from B has been produced. $\square$

19 When BANK also improves a forced horizon

Ordinary proof-horizon monotonicity requires only premise inclusion. Forced-horizon monotonicity is more delicate because changing the state may also change environment choices.

Theorem 19.1 (Conditional robust-horizon monotonicity). *Consider two strategic settlement games on comparable states, before and after a BANK enlargement. Assume:*

- (i) *every old controller move remains available after enlargement;*
- (ii) *BANK enlargement may add controller moves;*
- (iii) *it does not add new environment moves that can delay settlement;*
- (iv) *old settlement states remain settlement states; and*
- (v) *transition costs are unchanged on old moves.*

Then for every state that is forced-settling before enlargement, the post-enlargement forced-settlement horizon is no greater.

Proof. The controller can ignore all new moves and replay its old forcing strategy. By (iii), the environment has no new delaying responses, and by (iv) the old terminal objective remains valid. Therefore the old worst-case bound remains achievable; minimization over the larger controller strategy set can only improve or preserve it. $\square$

Remark 19.2 (Why the hypotheses matter). At the implementation level, extra BANK material can enlarge candidate sets, alter queue order, increase indexing costs, or interact with caps. Version 48 already warns that logical horizon monotonicity does not imply heuristic runtime monotonicity. The same warning prevents an unconditional robust-horizon theorem unless the strategic move sets are controlled explicitly.

20 Learned legal-move ranking in GBG language

Version 48 defines a learned legal-move ranker

$$s_\theta : \{(S, a) : a \in A(S)\} \rightarrow \mathbb{R}$$

that changes ordering or sampling weight among already legal candidate applications. The verifier remains the trust boundary.

The GBG reformulation is immediate but useful:

the ranker changes move selection, not the legal-move relation.

Proposition 20.1 (Policy/legality separation). *Fix an operational state graph and a sound verifier acceptance relation. Replacing one policy for selecting among legal outgoing actions by another cannot create a new verifier-accepted theorem state outside the formal consequence relation, provided the candidate-generation legality and verifier semantics are unchanged.*

Proof. A policy selects which legal transitions to explore and in what order. Every accepted theorem certificate must still consist of legal underlying inference applications and pass the unchanged sound verifier. Search allocation may change hitting time or bounded-budget success but not object-level validity. $\square$

This places learned ATP guidance, A* priorities, Hilbert schedules, proof compasses, novelty bonuses, and related heuristics in one category: *navigation policies over a fixed or conservatively evolving legal transition structure.*

21 Causal BANK depth and path compression

Version 48 defines causal BANK depth $\delta(\lambda)$ by recursively expanding accepted deposit certificates and taking the maximum primitive inference depth from original BANK leaves to the deposited formula. This gives a schedule-insensitive dependency quantity.

The GBG synthesis suggests two simultaneous views of a deposited lemma λ :

1. in the expanded primitive graph, λ sits at a positive dependency depth from B_0 ;
2. in the post-deposit explicit-premise graph, λ is available at the starting boundary and can act as a zero-local-cost resource for later search.

Thus BANK compression is a quotient-like path contraction. It does not erase provenance because the accepted certificate stores the expansion map back to primitive inference structure.

Definition 21.1 (Certified path contraction). *A certified path contraction replaces a verified finite derivation subpath from earlier BANK resources to λ by direct availability of λ in later explicit-premise problems, while retaining a provenance pointer to the accepted expansion certificate.*

This definition separates computational reuse from logical strengthening. The contracted search graph can be dramatically easier to navigate even when the underlying consequence closure is unchanged.

22 A minimal worked example

Let

$$B_0 = \{a, b\}$$

and let F contain primitive rules

$$r_1(a, b) = c, \quad r_2(c) = d, \quad r_3(b) = e, \quad r_4(e) = d.$$

The target is d .

There are two two-step proofs:

$$(a, b) \xrightarrow{r_1} c \xrightarrow{r_2} d$$

and

$$(a, b) \xrightarrow{r_3} e \xrightarrow{r_4} d.$$

Hence

$$H_F^{\text{inf}}(B_0, d) = 2.$$

The derivation history tree has at least two depth-two target histories, while a state representation that remembers only the final fact d would identify their target endpoint. A future-complete representation may still distinguish intermediate states $\{a, b, c\}$ and $\{a, b, e\}$ because their legal continuations differ.

Now suppose the first channel certifies c and deposits it conservatively. Then

$$B_1 = \{a, b, c\}$$

and

$$H_F^{\text{inf}}(B_1, d) = 1,$$

while

$$\text{Con}_F(B_1) = \text{Con}_F(B_0).$$

The theorem set did not grow. The explicit route shortened.

Finally, add a scheduler that controls whether the prover receives a local step. If the scheduler may starve forever, the logical horizon remains 2 but no finite forced global horizon exists. If the scheduler must grant one prover step per round, settlement is forced within two prover-service rounds. One toy system therefore exhibits all three distinctions:

logical distance, operational discovery, scheduler-robust settlement.

23 The five AMLD roles as settlement regions

Version 48 uses the roles

$$P, R, D, I, C$$

with typed object-level and metatheoretic certificate interfaces. In the global-state graph these naturally define settlement regions:

$$W_P, \quad W_R, \quad W_D, \quad W_I, \quad W_C,$$

where membership means that the corresponding verifier-certified record has been accepted.

The overall first-settlement region is

$$W_{\text{set}} = W_P \cup W_R \cup W_D \cup W_I \cup W_C.$$

The Version 48 identity

$$\tau_{\mathcal{A}}^{\text{set}} = \min\{\tau_{\mathcal{A}}^P, \tau_{\mathcal{A}}^R, \tau_{\mathcal{A}}^D, \tau_{\mathcal{A}}^I, \tau_{\mathcal{A}}^C\}$$

is then the realized-run hitting-time counterpart of distance to a union of target regions.

There is an important typing restriction: D and I live in a declared metatheory and cannot be treated as ordinary object-level target formulas merely because they share the same architecture. The GBG state graph may include both typed components, but legal moves must preserve the type boundary.

24 A proposed strategic settlement hierarchy for Depths

The combined framework supports the following hierarchy.

Definition 24.1 (Intrinsic settlement horizon). *For a fixed target settlement region W in a frozen logical or certificate transition graph,*

$$H_{\text{intr}}(s, W) = \text{dist}(s, W).$$

Definition 24.2 (Policy discovery horizon). *For a deterministic seeded policy π producing trajectory $s_0, s_1, \dots$, define*

$$H_{\pi}^{\text{hit}}(W) = \min\{n : s_n \in W\},$$

with value ∞ if the trajectory never hits W . Randomized policies produce random hitting times or censored observations under finite budgets.

Definition 24.3 (Forced settlement horizon). *For a controller/environment partition, $H_{\text{force}}(s, W)$ is the least worst-case number of legal transitions in which the controller can guarantee reaching W , with value ∞ if no finite forcing strategy exists.*

These quantities answer different questions:

H_{intr}	How close is a settlement certificate in the best legal case?
H_{π}^{hit}	How long does this particular navigation policy actually take?
H_{force}	Can settlement be guaranteed despite all admissible choices not controlled by the navigator?

For deterministic single-agent proof construction with complete control and unit costs, $H_{\text{force}} = H_{\text{intr}}$. In multi-agent or scheduled systems the quantities can separate sharply.

25 What this adds to the induction story

The GBG synthesis does not introduce a new induction axiom. This is consistent with Version 48’s “no fifth foundational induction principle” result for AMLDs.

Instead, it clarifies where existing induction acts:

1. **Derivation induction** proves properties along finite logical paths.
2. **Search-stage induction** proves invariants along operational ATP transitions.
3. **AMLD global-state induction** proves properties along the accepted shared chronology.
4. **Backward induction** computes finite forced-settlement regions in locally finite strategic games.

The first three are forward inductions over generated histories or stage numbers. The fourth reasons backward from a terminal target region. They are methodologically complementary, not competing foundational principles.

26 Boundaries and nonclaims

The synthesis is strongest when its boundaries are explicit.

- B1. Reachability is not strategy.** A theorem may be reachable even when no controller can force it under the declared external-choice model.
- B2. Logical distance is not runtime.** A short proof can be hard for a heuristic to discover; extra BANK material can shorten logical distance while worsening indexing or branching.
- B3. A transposition key must be future-complete.** Equality of visible or partial state descriptions is insufficient when hidden metadata affects legal continuations.
- B4. BANK conservation is not primitive-proof shortening.** A certified lemma may be a macro whose expansion is long.

- B5. Forced-horizon monotonicity needs move-set hypotheses.** Adding information can change both controller and environment options at the implementation level.
- B6. Independence is not failed reachability search.** Version 48’s metatheoretic certificate requirement remains intact.
- B7. No total-decider conclusion follows.** A finite forced horizon exists only on states satisfying the relevant forcing hypotheses; undecidability and incompleteness boundaries are untouched.
- B8. No literature-priority claim is made here.** Reachability games, dynamic programming, minimax recurrences, state-space search, and transposition techniques are broad established subjects. The contribution of this article is the internal synthesis with the specific Version 48 formal architecture.

27 Research consequences

The new strategic layer suggests a sequence of concrete problems.

Research Question 27.1 (Robust AMLD horizon). For a frozen AMLD architecture and an explicit admissible scheduler class, characterize the states from which first settlement can be forced and compute or bound the corresponding horizon.

Research Question 27.2 (BANK-sensitive minimax). Under what practical restrictions on candidate generation and scheduling does conservative BANK growth provably preserve or decrease a robust settlement horizon, rather than merely the intrinsic logical proof horizon?

Research Question 27.3 (Transposition quotient). What is the smallest future-complete state representation for a given ATP or AMLD? In particular, which queues, caches, provenance fields, random seeds, and BANK summaries can be quotiented away without changing future legal behavior?

Research Question 27.4 (A^* and admissible settlement heuristics). Can a computable lower bound on intrinsic settlement distance be used as an admissible A^* heuristic on a finite or bounded operational subgraph while retaining verifier sovereignty and a complete fallback?

Research Question 27.5 (Professor or oracle guidance). If an external guide supplies certified or calibrated distance-to-settlement estimates but does not control legality, when does that information reduce expected discovery cost, worst-case forced horizon, or both? These are different performance claims and should be tested separately.

Research Question 27.6 (Geodesic defect versus strategic regret). Later settlement-geometry papers define geodesic and local-defect notions. A natural strategic extension is to compare a realized route not only with a shortest path but with the value of an optimal robust policy under the same admissible external-choice model.

Version 49 Integration C: Scheduled GBG Clocked Simulation Degrees

28 Scheduled GBG clocked simulation degrees

The phrase *GBG simulation degree* is used here in a deliberately controlled sense. A full branching board game contains choices and potentially adversarial strategies, whereas the Version 48 relation $\preceq_{\text{clk}}$ compares deterministic stage evolutions. The direct bridge therefore begins with a scheduled or deterministic presentation.

Definition 28.1 (Scheduled deterministic GBG presentation). *A scheduled deterministic GBG presentation is a rule-governed state system equipped with: a state space S_G , a stage evolution map $C_G : S_G \times \mathbb{N}^+ \rightarrow S_G$, and a binary terminal or settlement observation $H_G : S_G \times \mathbb{N}^+ \rightarrow \{0, 1\}$. All scheduling choices, tie breaks, random seeds, and other data needed to make future evolution deterministic are regarded as part of the complete state or the fixed schedule.*

Definition 28.2 (Clocked simulation of scheduled GBGs). *For scheduled deterministic presentations G and H , write*

$$G \preceq_{\text{GBG,clk}} H$$

when there are an injective $f : S_G \rightarrow S_H$ and a nondecreasing unbounded clock $\rho : \mathbb{N}^+ \rightarrow \mathbb{N}^+$, with $\rho(1) = 1$, satisfying

$$f(C_G(p, m)) = C_H(f(p), \rho(m)), \quad H_G(p, m) = H_H(f(p), \rho(m))$$

for every state p and stage m . Mutual simulation defines a scheduled GBG clocked degree.

Proposition 28.3. *Clocked simulation of scheduled deterministic GBG presentations is a preorder, and mutual simulation induces a partial order on the corresponding degrees.*

Proof. The identity state map and identity clock give reflexivity. For transitivity, compose the injective state maps and compose the two nondecreasing unbounded clocks. The two preservation equations then compose directly. Passing to mutual-simulation equivalence classes converts the preorder to a partial order. $\square$

Remark 28.4 (What is transferred and what is not). Whenever a GBG chronology is represented by deterministic evolution and settlement data of the same formal type used in the Version 48 clocked relation, the preorder and mutual-simulation quotient machinery apply to that presentation [1]. SIC-specific theorems transfer only after the SIC axioms and any additional hypotheses used by those theorems have been separately verified. This does *not* by itself define the correct simulation notion for an unrestricted branching, two-player, or imperfect-information game. For those systems, one may additionally require branching, player ownership, legal-choice sets, winning regions, or strategies to be preserved. That strategy-preserving theory is left as a refinement rather than silently identified with $\preceq_{\text{GBG,clk}}$.

29 Degree structure versus theoremhood

The purpose of a degree is to record chronological structure that theoremhood alone forgets. For systems written in the same theorem language one may have

$$\text{Con}_F(B) = \text{Con}_{F'}(B'),$$

and for different theorem languages the corresponding statement should be made only after fixing an explicit translation that identifies the compared consequences. Such systems may also preserve the same shortest object-level proof depth while differing in the order, synchronization, or cost with which operational states are exposed.

Accordingly,

same theoremhood alone does not establish the same clocked simulation degree.

A distinct-degree conclusion still requires a specific nonsimulation theorem or invariant. What must be tested is an injective chronology-preserving simulation in each direction, not merely equality (or translated correspondence) of target sets.

30 Deterministic hitting-time distortion under a clocked embedding

For a scheduled deterministic GBG presentation G , a state s , and a target region T_G , define the target-hitting stage

$$h_G(s, T_G) := \min\{m \in \mathbb{N}^+ : C_G(s, m) \in T_G\},$$

with value ∞ if the set is empty. Let (f, ρ) witness $G \preceq_{\text{GBG,clk}} H$. Assume target preservation

$$f[T_G] \subseteq T_H$$

and an explicit quantitative clock bound

$$\rho(n) \leq \tau(n) \quad (n \in \mathbb{N}^+)$$

for a nondecreasing function τ .

If $h_G(s, T_G) = n < \infty$, then

$$C_G(s, n) \in T_G.$$

The simulation equation and target preservation give

$$C_H(f(s), \rho(n)) = f(C_G(s, n)) \in T_H,$$

so

$$h_H(f(s), T_H) \leq \rho(n) \leq \tau(n) = \tau(h_G(s, T_G)).$$

Thus the clock controls distortion of deterministic target-hitting stages under the stated hypotheses. It does not, by itself, compare shortest paths in an unrestricted branching graph; such a graph-distance claim requires an edge/path-preserving simulation notion.

This illustrates the qualified link

clocked embedding + clock bound $\implies$ controlled deterministic hitting-time distortion.

31 Case study: Hilbert-addressed operational ATPs

The paper *From Theoremhood as Reachability to Hilbert-Addressed Inference Spaces* [3] provides a natural Version 49 case study. It lifts ATP execution to future-complete operational states, represents elementary deterministic machine attempts as partial unary GBG moves, distinguishes histories from terminal operational states, and places optional Hilbert navigation on the finite candidate-action fibers exposed by each state. It also separates represented proof depth from within-layer discovery offset.

Fix the underlying formal system, candidate-generation protocol, verification rule, budgets, and complete operational state representation. Consider scheduled ATP/GBG presentations such as

$$G_{\text{lex}}, \quad G_{\text{Hilbert}}, \quad G_{\text{random}}, \quad G_{\text{learned}},$$

where a random seed is included in the state whenever needed to make a run deterministic.

These systems can agree on theoremhood while differing in the chronology of candidate examination. For breadth-first variants with ordering only inside frozen depth layers, they can also agree on represented proof horizon while moving the target to different within-layer microsteps.

Version 49 therefore poses the concrete comparison problem

$$G_{\text{lex}} \preceq_{\text{GBG,clk}} G_{\text{Hilbert}}? \quad G_{\text{Hilbert}} \preceq_{\text{GBG,clk}} G_{\text{lex}}?$$

and similarly for other matched schedulers.

Remark 31.1 (No separation theorem is asserted). The Hilbert-addressed paper supplies the operational objects needed to ask the degree question; it does not by itself prove that Hilbert and lexicographic schedules lie in distinct degrees. A separation requires an invariant or obstruction showing failure of one of the required clocked embeddings. Conversely, an equivalence result requires explicit simulations in both directions.

32 Why the infinite-antichain theorem matters for GBGs

Theorem 2.1 shows that a chronology-preserving simulation order can have genuine infinite width even among finite, finitely describable systems. The result cautions against expecting all executable rule-governed search systems to line up on one simple ladder of “more powerful” and “less powerful.”

For GBG-style operational search spaces the analogous program is:

1. choose the exact state information whose preservation makes future behavior complete;
2. choose whether the simulation must preserve only chronology, or also branching and strategies;
3. find monotone invariants such as synchronized observation spectra, target-region behavior, or bounded horizon distortion;
4. construct chains and antichains under that fixed notion.

The old SIC antichain is therefore not merely historical background. It is a worked order-theoretic model of the kind of complexity a GBG simulation-degree theory may need to accommodate.

33 Strategic extension: beyond deterministic clocked degrees

For a branching game, preserving one scheduled chronology may be too weak. A stronger relation could require:

- legal successors to map to legal successors or bounded legal paths;
- controller-owned and environment-owned choices to remain distinguished;
- target or settlement regions to be preserved;
- every winning controller strategy in the source to translate to a winning strategy in the target;
- a quantitative bound on the distortion of forced-settlement horizons.

This suggests a progression of increasingly demanding definitions:

reachability-preserving simulation
 ↓ add chronological and clock requirements
 clocked chronological simulation
 ↓ add strategy-preservation requirements
 strategy-preserving clocked simulation.

The downward arrows here mean “add requirements”; they are not logical implication arrows. Once precise relations are fixed, their inclusion direction should be stated separately.

34 Version 49 research boundary

Version 49 closes with the following questions rather than adding further architectures.

Research Question 34.1 (Greatest degree). Does the original SIC degree poset $\mathcal{D}_{\text{clk}}$ have a greatest degree?

Research Question 34.2 (GBG infinite width). For a fixed, explicitly defined strategy-preserving GBG simulation relation, can one construct a countably infinite antichain of finite, finitely describable GBGs?

Research Question 34.3 (Hilbert degree comparison). Under a frozen operational ATP model, do Hilbert and lexicographic within-layer schedulers occupy the same scheduled GBG clocked degree? If not, which synchronized invariant separates them?

Research Question 34.4 (Forced-horizon preservation). Which simulation hypotheses are sufficient to preserve finiteness of a forced-settlement horizon, and which give a quantitative distortion bound?

35 Version 49 conclusion

Version 49 consolidates rather than opens another branch of the project. The retained Version 48 core supplies the *xyz* formal systems, SICs, ATPs, induction principles, proof horizons, search metrics, AMLDs, BANK, and formalized self-awareness. The clocked-simulation addendum is no longer merely an exploratory preorder: its quotient degree structure has a countably infinite antichain of finite finitely describable machines, nonlattice behavior, synchronized monotone invariants, and the same phenomena under effective presentation. Within the finitely describable class, the antichain’s cardinality $\aleph_0$ is the largest possible.

The GBG integration then provides an operational geometry for the same objects. Finitary inference becomes legal motion on complete derivation states; theoremhood becomes target-region reachability; proof horizon becomes directed distance under a declared cost convention; and transposition quotienting separates histories from future-complete states. At the strategic level, proof distance, discovery effort, and forced settlement become different quantities rather than competing names for one quantity. AMLD nonstarvation constrains the environment, while conservative BANK growth can shorten certified routes without changing logical consequence closure.

Finally, clocked simulation degrees acquire a GBG reading for scheduled deterministic chronologies. The Hilbert-addressed ATP architecture supplies a concrete case study in which theoremhood and represented proof depth may remain fixed while operational ordering changes. Version 49 does not assume that such schedulers occupy distinct degrees; it turns that intuition into an explicit simulation problem.

That is the stopping point of this edition. The next substantial theme should begin in a later version rather than make Version 49 a moving target.

Version 49 references

References

- [1] Brian Tenneson. *Depths of Induction and Formalized Self-Awareness, Version 48*. 2026. https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/depths_of_induction_v48/
- [2] Brian Tenneson. *Generalized Board Games: From Tic-Tac-Toe to Chess as xyz Formal Systems*. 2026. https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/generalized_board_games_tic_tac_toe_chess_001/
- [3] Brian Tenneson. *From Theoremhood as Reachability to Hilbert-Addressed Inference Spaces*. 2026. https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/from_theoremhood_as_reachability_to_hilbert_addressed_inference_spaces_003/
- [4] Brian Tenneson. *Depths of Induction: Settlements of Open Problems 9.1, 9.3, 9.4, and 9.5*. Companion note to the Addendum to Version 48, 2026.
- [5] D. A. Spielman and M. Bóna. “An Infinite Antichain of Permutations.” *The Electronic Journal of Combinatorics* 7 (2000), No. N2. DOI: 10.37236/1540.